Reinforcement Learning with Pacman

From Markov decision processes to PPO and expert iteration

Course written for the repository CL-BZH/Pacman

November 8, 2026

Contents

How to read this course **vi**

1 Introduction **1**
1.1 The agent–environment loop . . . 1
1.2 Where reinforcement learning is used . . . 1
1.3 A map of the methods . . . 2
1.4 The running example: a Pacman game . . . 2
1.5 The rewards . . . 4
1.6 What the agent observes . . . 4
1.7 Why reinforcement learning? . . . 5
1.8 Game v2 . . . 5
1.9 Plan of the course . . . 5

2 Multi-armed bandits **6**
2.1 The problem . . . 6
2.2 Greedy and ε-greedy . . . 7
2.3 Upper confidence bounds . . . 7
2.4 Thompson sampling . . . 9
2.5 Gradient bandits . . . 10
2.6 Contextual bandits . . . 10
2.7 Summary . . . 10

3 Markov decision processes **12**
3.1 Markov decision processes . . . 12
3.2 Returns . . . 13
3.3 Policies and value functions . . . 14
3.4 The Bellman expectation equations . . . 15
3.5 Optimal policies . . . 17
3.6 Other criteria . . . 19
3.7 Summary . . . 19

4 Dynamic programming **20**
4.1 Policy evaluation . . . 20
4.2 Policy improvement . . . 21
4.3 Policy iteration . . . 22
4.4 Value iteration . . . 22
4.5 Linear programming . . . 23
4.6 Pacman solved exactly . . . 24
4.7 The curse of dimensionality . . . 25

5 Monte Carlo and temporal-difference learning **26**
5.1 Monte Carlo prediction . . . 26
5.2 Temporal-difference learning . . . 27
5.3 n-step returns and λ-returns . . . 28
5.3.1 Eligibility traces . . . 29
5.4 Control: SARSA and Q-learning . . . 30
5.5 Off-policy Monte Carlo and importance sampling . . . 32
5.6 The maximisation bias . . . 33
5.7 Summary . . . 33

6 Value function approximation **34**
6.1 The prediction objective . . . 34
6.2 Gradient Monte Carlo and semi-gradient TD . . . 35
6.3 The TD fixed point . . . 35
6.4 Least-squares TD . . . 37
6.5 Off-policy divergence . . . 37
6.6 Control with function approximation . . . 38
6.7 Summary . . . 38

7 Deep Q-networks **39**
7.1 Q-learning with function approximation . . . 39
7.1.1 The deadly triad . . . 39
7.2 DQN . . . 40
7.3 Improvements . . . 41
7.3.1 Double DQN . . . 41
7.3.2 Dueling architecture . . . 41
7.3.3 n-step targets . . . 42
7.3.4 Soft target updates and other details . . . 42
7.4 Rainbow and beyond . . . 42
7.4.1 Prioritised experience replay . . . 43
7.4.2 Distributional reinforcement learning . . . 43
7.4.3 Noisy networks . . . 44
7.4.4 Beyond Rainbow . . . 44
7.5 The DQN agent of the repository . . . 44
7.6 Summary . . . 45

8 Policy gradients and actor-critic methods **46**
8.1 Parametrised policies . . . 46
8.2 The policy gradient theorem . . . 46
8.3 Baselines and advantages . . . 48
8.4 Actor-critic methods . . . 48
8.4.1 Generalised advantage estimation . . . 49
8.5 Continuous actions: Gaussian policies . . . 50
8.6 The natural policy gradient . . . 50
8.7 A2C . . . 51
8.8 The actor-critic network of the repository . . . 52
8.9 Summary . . . 53

9 Trust regions: TRPO and PPO **54**
9.1 How good is a new policy? . . . 54
9.2 Trust region policy optimisation . . . 56
9.3 The clipped objective . . . 56
9.4 The algorithm . . . 57
9.5 Results on Pacman . . . 58
9.6 Summary . . . 59

10 Continuous actions: DDPG, TD3 and SAC **60**
10.1 Deterministic policy gradients . . . 60
10.2 DDPG . . . 61
10.3 TD3 . . . 61
10.4 Soft actor-critic . . . 61
10.5 Summary . . . 63

11 Model-based reinforcement learning and planning **64**
11.1 Models and background planning . . . 64
11.2 Decision-time planning . . . 65
11.2.1 Rollouts . . . 65
11.2.2 Two ways of choosing the move . . . 66
11.3 Monte Carlo tree search . . . 67
11.3.1 AlphaZero . . . 68
11.3.2 MuZero . . . 68
11.4 What does the search know about the ghosts? . . . 69
11.5 Expert iteration . . . 70
11.6 What is left? . . . 71
11.7 Summary . . . 71

12 Imitation learning and offline RL **72**
12.1 Behaviour cloning . . . 72
12.2 DAgger . . . 73
12.3 Inverse reinforcement learning . . . 73
12.4 Offline reinforcement learning . . . 74
12.5 Summary . . . 76

13 Exploration **77**
13.1 Why dithering fails . . . 77
13.2 Optimism in the face of uncertainty . . . 77
13.3 Counts in large state spaces . . . 78
13.4 Posterior sampling and deep exploration . . . 79
13.5 Summary . . . 80

14 Multi-agent reinforcement learning **81**
14.1 Markov games . . . 81
14.2 Two-player zero-sum games . . . 81
14.3 Learning in cooperative teams . . . 82
14.4 Summary . . . 84

15 Reinforcement learning from human feedback **85**
15.1 A language model is a policy 85
15.2 Reward models from preferences 85
15.3 RL fine-tuning with a KL penalty 86
15.4 Direct preference optimisation 87
15.5 Feedback from AI 88
15.6 Summary 88

Conclusion **89**

How to read this course

This course presents reinforcement learning (RL) from its foundations up to the algorithms used in the repository `CL-BZH/Pacman`: deep Q-networks (DQN), advantage actor-critic (A2C) with generalised advantage estimation (GAE), proximal policy optimisation (PPO), look-ahead search and expert iteration. Every notion is illustrated on the Pacman game of the repository, and the results quoted were obtained with its code.

The boxes follow a colour code:

- **blue**: definitions;
- **red**: theorems, lemmas and corollaries;
- **amber**: propositions;
- **green**: examples on the Pacman game;
- **grey**: remarks;
- **violet**: algorithms.

Proofs are given in full. The mathematical prerequisites are elementary probability (conditional expectation, Markov chains), linear algebra and the basics of neural networks and gradient descent.

Chapter 1

Introduction: reinforcement learning and the Pacman example

Reinforcement learning studies how an *agent* can learn to act in an *environment* so as to collect as much *reward* as possible, using only the experience it gathers by acting. Nobody tells the agent which action is correct: it only observes the consequences of its actions, which may come much later. This chapter introduces the vocabulary of RL, gives a map of its methods, and states precisely the rules and rewards of the Pacman game of the repository, which the following chapters use as a running example whenever an algorithm applies to it.

1.1 The agent–environment loop

At each time step $t = 0, 1, 2, \dots$ the agent observes the state S_t of the environment, chooses an action A_t, and the environment answers with a reward R_{t+1} and a new state S_{t+1} (Fig. 1.1). The interaction produces a *trajectory*

$$S_0,\ A_0,\ R_1,\ S_1,\ A_1,\ R_2,\ S_2,\ A_2,\ R_3,\ \dots \tag{1.1}$$

which stops when a *terminal* state is reached; the trajectory is then called an *episode*. For Pacman an episode is a game.

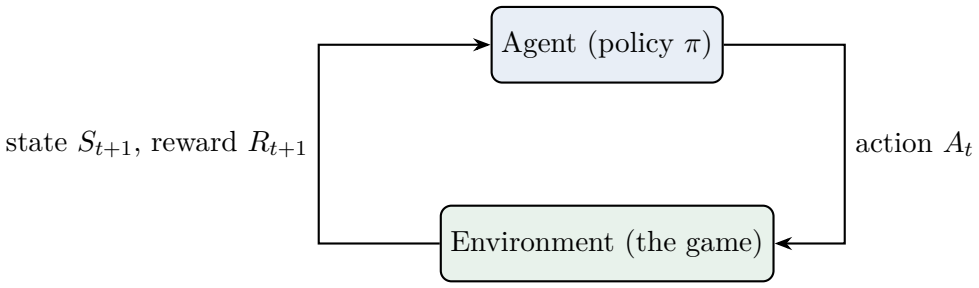


Figure 1.1: The agent–environment loop.

The behaviour of the agent is described by a *policy* π, which gives for each state a probability distribution over the actions; the goal of RL is to find a policy that maximises the expected sum of the rewards (Chapter 3 makes this precise).

1.2 Where reinforcement learning is used

The framework covers any problem of sequential decisions under uncertainty in which the quality of a behaviour can be measured by rewards. Some landmarks:

- **Games**: backgammon (TD-Gammon, 1992), Atari video games from pixels (DQN, 2015), Go, chess and shogi (AlphaGo and AlphaZero, 2016–2018), StarCraft and Dota 2 (2019).
- **Robotics and control**: locomotion, manipulation, the control of plasma in a tokamak, the cooling of data centres; actions are then continuous (Chapter 10).
- **Recommendation and operations**: choosing what to show to a user, pricing, routing; often formulated as bandits (Chapter 2).
- **Language models**: the alignment of large language models with human preferences (Chapter 15).

1.3 A map of the methods

RL algorithms can be classified along a few independent axes, which organise this course.

- **What is learnt.** *Value-based* methods learn the value of states or actions and act greedily with respect to it (Q-learning, DQN, Chapters 5 and 7). *Policy-based* methods learn the policy directly (policy gradients, Chapter 8). *Actor-critic* methods learn both (A2C, PPO, SAC, Chapters 8 to 10). *Model-based* methods learn or use a model of the environment to plan (Chapters 4 and 11).

- **Which data.** *On-policy* methods learn from the data of the current policy (SARSA, A2C, PPO); *off-policy* methods can learn from data generated by other policies (Q-learning, DQN, SAC); *offline* methods learn from a fixed dataset without interacting with the environment (Chapter 12).

- **How targets are formed.** *Monte Carlo* methods use complete returns; *temporal-difference* methods *bootstrap* from their own estimates (Chapter 5).

- **Which actions.** Discrete actions (Pacman's five moves) or continuous ones (the torques of a robot, Chapter 10).

- **How many agents.** One agent in a fixed environment, or several agents learning together (Chapter 14).

Two further themes cut across all of them: *exploration*, the need to try actions whose value is uncertain (Chapters 2 and 13), and *generalisation*, the need to represent values and policies with parametrised functions when there are too many states for a table (Chapters 6 and 7).

1.4 The running example: a Pacman game

The game of the repository is a simplified Pacman written from scratch to test RL algorithms. We describe the original version of the game (called `v1` in the code); Section 1.8 lists the changes of version `v2`.

The board. The maze `pacman_map3` is a grid of 11×15 cells whose borders are walls; 77 cells are open (Fig. 1.2). At the start of a game every open cell holds a cookie, except the cell of Pacman. Two of the cookies are replaced by *power cookies*. Pacman and the two ghosts start on random cells, with the ghosts at a distance at least 2 from Pacman.

The actions. Pacman has five actions: `E`, `N`, `W`, `S` (move by one cell to the east, north, west or south) and `I` (stay idle). Moving into a wall leaves Pacman where he is.

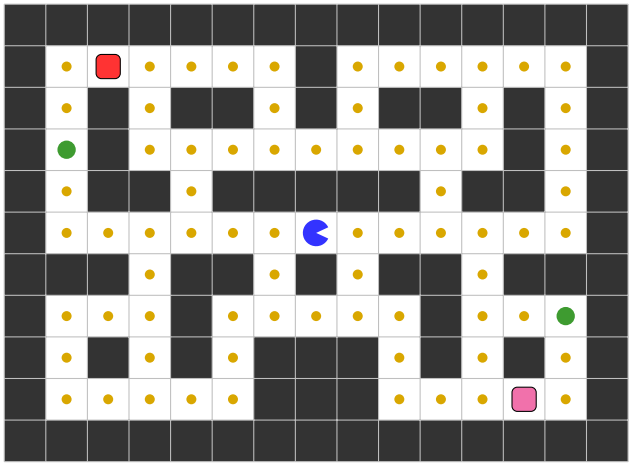


Figure 1.2: A position of game `v1` on the maze `pacman_map3`: Pacman (blue), the two ghosts, the two power cookies (green) and the cookies (yellow). The walls are black.

One step of the game. A call to `step(action)` plays one move of everybody, in this order:

1. ghost 0 moves;
2. Pacman moves; his power credit (see below) decreases by one; he eats the cookie of his new cell, if any;
3. the other ghosts move.

After each of these moves, if Pacman and a ghost are on the same cell, the ghost eats Pacman (who loses the game) if Pacman has no power, and Pacman eats the ghost otherwise (the ghost leaves the game).

The power. Eating a power cookie sets Pacman's *power credit* to $C = 12$. The credit decreases by one at each move of Pacman, and Pacman can eat ghosts while it is positive. Since the credit is decreased *before* the collision test, Pacman with a credit of 1 who moves onto a ghost has no power any more at the moment of the collision, and is eaten: we will meet this "off-by-one" situation again in Chapter 11.

The end of a game. Pacman wins when he has eaten all the cookies (normal and power ones), and loses when a ghost eats him. Games longer than a fixed number of moves are *aborted* (1000 moves during training): this limit is not part of the game itself, and Chapter 3 explains why it must be treated differently from a real end.

The ghosts. The ghosts are the random part of the environment. Ghost i has an *aggressiveness* a_i ($a_0 = 0.74$, $a_1 = 0.64$). At each of its moves, when Pacman has no power:

- with probability a_i the ghost *chases* Pacman: it moves to the next cell of a shortest path to Pacman (computed with the A* algorithm);
- otherwise it moves to a cell chosen uniformly among the cells it can reach in one move, including its own cell.

When Pacman has power, the ghosts *flee*: a ghost moves to a reachable cell chosen with a probability proportional to its Euclidean distance to Pacman, and never to Pacman's cell.

Remark 1.1. (Random moves of the ghosts)

The code of the ghosts intends to give a weight 1.5 to the move that keeps the current direction of a ghost, but the comparison it uses (a position with a direction letter) is never true: the random moves are uniform, as described above. A measurement over many games

confirms it (a ghost keeps its direction 31.7% of the time, against 31.2% for a uniform choice). As a consequence, the ghosts' moves depend only on the current positions and on Pacman's power, which matters in Chapter 3. Game `v2` (Section 1.8) fixes the comparison: its ghosts keep their direction 39.6% of the time in the same measurement, as predicted by the weight 1.5, while the games of `v1` are unchanged.

1.5 The rewards

The reward R_{t+1} of a step is the sum of the rewards of the events of the step, listed in Table 1.1. Its design reflects what the author of the game wanted Pacman to learn: eat the cookies, use the power to eat the ghosts, and avoid dying.

Event	Reward	Condition
Living cost (each move of Pacman)	-0.15	Pacman has no power, or no ghost is left
Staying in place	-0.75	instead of the living cost, if Pacman has power or no ghost is left
Eating a cookie	$+0.8$	Pacman has no power, or no ghost is left
Eating a power cookie	see (1.2)	$+0.8$ if no ghost is left
Eating a ghost	$+20$	
Being eaten	-35	
Winning	$+30 - 5\,g$	g = number of ghosts left

Table 1.1: Rewards of game `v1` (`pacman_training.py`).

The reward for a power cookie depends on the situation. If c is Pacman's power credit when he eats it (after the decrease of his move) and $d \geq 1$ is the Manhattan distance to the nearest ghost,

$$r_{\text{power}} = 1.3 \left(1 - \frac{c}{C}\right) \frac{C}{d}. \tag{1.2}$$

Eating a power cookie is thus worth more when Pacman has little power left and when a ghost is close: $r_{\text{power}} = 1.3 \times 12 = 15.6$ with no power and a ghost at distance 1, but only 1.3 with a ghost at distance 12.

Pacman 1.2. (An optimistic estimate of the score)

A game won without power and without eating a ghost, in n moves, has a total reward of about $0.8 \times 76 - 0.15\,n + 30 - 10$. With $n = 100$ this is $60.8 - 15 + 20 = 65.8$. Each ghost eaten adds $20 + 5$ (its reward and the penalty avoided), which is why the trained agents hunt the ghosts: the reward per game of the agent of the repository is about 115 at the end of its training (Chapter 9).

1.6 What the agent observes

The neural networks of the agents do not receive the Python objects of the game but an image of it, the *feature planes* of `PacmanEnvironment.feature_image`: an array of shape $(5+n_g) \times 9 \times 13$ for n_g ghosts (the borders are removed), with one binary plane for each kind of object:

1. the walls;
2. the cookies;
3. the power cookies;
4. Pacman;

5–6. one plane per ghost (they have different aggressiveness);
7. a constant plane equal to Pacman's power $c/C \in [0, 1]$.

The agents stack the last two images (*frame stacking*), so that the input has $2 \times 7 = 14$ channels. Chapter 3 discusses why the image is (almost) a complete description of the state of the game, and what the second frame adds.

1.7 Why reinforcement learning?

Pacman illustrates the three difficulties that distinguish RL from supervised learning.

- **No teacher.** There is no dataset of correct moves: the agent only receives rewards, and must discover by itself that, for example, waiting near a power cookie for a ghost to come close is a good idea.
- **Delayed consequences.** A move into a dead end can be fatal five moves later; the reward -35 arrives long after the mistake. The agent must assign the credit (or the blame) of a reward to the actions that caused it.
- **Exploration.** The data depend on the agent's own behaviour: an agent that never eats a power cookie never learns that eating ghosts pays. The agent must balance the exploitation of what it knows with the exploration of what it does not.

1.8 Game v2

On game `v1` the trained agents win more than 96% of the games and usually eat both ghosts early, after which the game is only about cookies. Game `v2` makes the whole game matter: a bigger maze (`pacman_map4`, 17×19 cells, 163 open), three ghosts, four power cookies, a power of 10 moves, a reward of 5 for eating a ghost, ghosts that start in a *ghost house* in the middle of the maze and *come back* to it when eaten (once Pacman's power has ended, each ghost in the house leaves it with probability 0.1 at each move), and ghosts whose random moves prefer their current direction (Remark 1.1). Unless stated otherwise, the examples of this course use game `v1`, on which all the results were obtained.

1.9 Plan of the course

- **Foundations.** Chapter 2 studies exploration in the simplest setting, bandits. Chapter 3 gives the framework (Markov decision processes, Bellman equations), Chapter 4 solves it when the model is known, Chapter 5 learns from experience instead.
- **Function approximation.** Chapter 6 studies linear value functions and the stability of TD learning; Chapter 7 moves to deep networks (DQN and its extensions).
- **Policy optimisation.** Chapter 8 derives policy gradients and actor-critic methods, Chapter 9 trust regions (TRPO, PPO), Chapter 10 continuous actions (DDPG, TD3, SAC).
- **Planning and data.** Chapter 11 covers model-based RL and search (Dyna, MCTS, AlphaZero, MuZero), Chapter 12 learning from demonstrations and from fixed datasets.
- **Further topics.** Chapter 13 returns to exploration in large environments, Chapter 14 to several agents, Chapter 15 to learning from human preferences.

The Pacman examples use the code of the repository; code excerpts show how the equations are implemented. The chapters on continuous actions, multi-agent RL and human feedback use other examples, as these settings do not arise in the game.

Chapter 2

Multi-armed bandits

Before facing sequential decisions, we study the exploration problem in its purest form. A *multi-armed bandit* is an MDP with a single state: each action ("arm") gives a random reward drawn from an unknown distribution, and the agent must find the best arm while collecting as much reward as possible along the way. Every reward spent trying an arm is a reward not collected from the best one; every arm not tried may be the best. This *exploration–exploitation dilemma* reappears in all of RL (the ε-greedy policy of DQN, the entropy bonus of PPO, Chapter 13), and the bandit algorithms of this chapter are used directly in planning: Monte Carlo tree search chooses its simulations with the UCB rule (Chapter 11).

2.1 The problem

Definition 2.1. (Stochastic multi-armed bandit)

A bandit with K arms is a family of reward distributions $\nu_1, \dots, \nu_K$ on $\mathbb{R}$ with means $\mu_1, \dots, \mu_K$. At each round $t = 1, 2, \dots, T$ the agent chooses an arm A_t and receives a reward $R_t \sim \nu_{A_t}$, independent of the past given A_t. Let $\mu^\star = \max_a \mu_a$, $\Delta_a = \mu^\star - \mu_a$ the *gap* of arm a, and $N_a(t)$ the number of times arm a was played in the first t rounds. The *regret* after T rounds is

$$\mathcal{R}_T = T\mu^\star - \mathbb{E}\left[\sum_{t=1}^{T} R_t\right]. \tag{2.1}$$

In words. $T\mu^\star$ is what an oracle who knew the best arm from the start would collect on average in T rounds; $\mathbb{E}[\sum_t R_t]$ is what the agent collects. The regret is the price of not knowing which arm is best. It compares with the best arm's *mean*, not with the luckiest rewards, so it measures bad decisions, not bad luck. If $\mathcal{R}_T$ grows linearly in T, the agent keeps losing a fixed amount per round: it never learns. If it grows sublinearly, the loss per round $\mathcal{R}_T/T$ tends to 0: the agent ends up playing the best arm.

Proposition 2.2. (Regret decomposition)

$\mathcal{R}_T = \sum_{a=1}^{K} \Delta_a \, \mathbb{E}\left[N_a(T)\right]$.

Proof. Given A_t, the expected reward is μ_{A_t}, so $\mathbb{E}[\sum_t R_t] = \mathbb{E}[\sum_t \mu_{A_t}] = \mathbb{E}[\sum_a \mu_a N_a(T)]$. Since $\sum_a N_a(T) = T$, $T\mu^\star - \sum_a \mu_a N_a(T) = \sum_a (\mu^\star - \mu_a) N_a(T)$; take expectations. □

In words. The regret is a bill: each play of arm a costs Δ_a on average, and the bill is the number of plays of each arm times its price. Playing an almost-optimal arm costs little; playing a bad arm costs a lot.

A good algorithm therefore plays each suboptimal arm rarely, and the more rarely the larger its gap. The estimate of an arm after n plays is the sample mean $\hat{\mu}_a$ of its rewards, which can be updated incrementally (Proposition 5.1).

2.2 Greedy and ε-greedy

The *greedy* algorithm plays each arm once, then always the arm of largest $\hat{\mu}_a$. If the best arm happens to give a bad first reward, it may never be played again: the regret grows linearly with T with positive probability. The *ε-greedy* algorithm plays a uniformly random arm with probability ε and the greedy arm otherwise.

Proposition 2.3. (Constant exploration has linear regret)

With a constant $\varepsilon > 0$, the regret of ε-greedy satisfies $\mathcal{R}_T \geq \frac{\varepsilon T}{K} \sum_a \Delta_a$.

Proof. At each round, with probability at least ε/K arm a is played as a random arm, so $\mathbb{E}[N_a(T)] \geq \varepsilon T/K$; apply Proposition 2.2. □

Decreasing ε_t (e.g. $\varepsilon_t \propto 1/t$) can give a logarithmic regret, but the right schedule depends on the unknown gaps. The next algorithms explore *where it is needed*: arms that are clearly worse are abandoned quickly, arms that might be the best are explored more.

2.3 Upper confidence bounds

The principle of *optimism in the face of uncertainty*: play the arm whose mean *could* be the largest, given the data. It relies on a concentration inequality.

Lemma 2.4. (Hoeffding's inequality (stated without proof))

Let $X_1, \dots, X_n$ be independent random variables with values in $[0,1]$ and mean μ, and $\bar{X}_n = \frac{1}{n}\sum_i X_i$. For every $c > 0$,

$$\mathbb{P}\left(\bar{X}_n \geq \mu + c\right) \leq e^{-2nc^2}, \qquad \mathbb{P}\left(\bar{X}_n \leq \mu - c\right) \leq e^{-2nc^2}. \tag{2.2}$$

In words. A sample mean rarely strays far from the true mean, and the probability of a deviation c falls exponentially in nc^2. Turned around: with probability at least $1-\delta$, $\left|\bar{X}_n - \mu\right| \leq \sqrt{\ln(2/\delta)/(2n)}$. Deviations of order $1/\sqrt{n}$ are typical: four times more data halve the uncertainty. The lemma gives an error bar around each estimate.

Definition 2.5. (UCB1)

For rewards in $[0,1]$, UCB1 (Auer, Cesa-Bianchi and Fischer, 2002) plays each arm once, then at round t the arm maximising

$$\hat{\mu}_a + \sqrt{\frac{2\ln t}{N_a(t-1)}}. \tag{2.3}$$

The bonus is large for arms played rarely, and decreases as their estimate becomes reliable; the slow growth of $\ln t$ ensures that every arm is eventually retried.

In words. The index of an arm is the most optimistic mean that is still plausible given its data: the estimate plus its error bar. The width of the bar comes from Hoeffding's inequality: choosing c so that $e^{-2nc^2} = t^{-4}$ gives $c = \sqrt{2\ln t/n}$, so the true mean exceeds the index only with a tiny probability. The rule corrects itself. An arm with a large index is either good, and playing it is right, or poorly known, and playing it shrinks its error bar until the arm is seen for what it is. Exploration stops by itself where it is no longer needed.

Theorem 2.6. (Regret of UCB1)

For rewards in $[0,1]$ and every suboptimal arm a ($\Delta_a > 0$),

$$\mathbb{E}\left[N_a(T)\right] \le \frac{8\ln T}{\Delta_a^2} + 1 + \frac{\pi^2}{3}, \qquad \text{hence} \qquad \mathcal{R}_T \le \sum_{a:\Delta_a>0} \left(\frac{8\ln T}{\Delta_a} + \left(1 + \frac{\pi^2}{3}\right)\Delta_a \right). \tag{2.4}$$

Proof. Fix a suboptimal arm a and an optimal arm $\star$. Let $c_{t,s} = \sqrt{2\ln t/s}$, let $\hat{\mu}_{a,s}$ be the mean of the first s rewards of arm a, and $\ell = \lceil 8\ln T/\Delta_a^2 \rceil$. Arm a is played at most ℓ times while $N_a < \ell$, so

$$N_a(T) \le \ell + \sum_{t=K+1}^{T} \mathbb{1}\left[A_t = a,\ N_a(t-1) \ge \ell\right].$$

If $A_t = a$, the index of a is at least that of $\star$: $\hat{\mu}_{\star,s} + c_{t,s} \le \hat{\mu}_{a,s_a} + c_{t,s_a}$ for the current counts $s = N_\star(t-1)$ and $s_a = N_a(t-1) \ge \ell$. This inequality implies one of the three events

$$\text{(E1) } \hat{\mu}_{\star,s} \le \mu^\star - c_{t,s}, \qquad \text{(E2) } \hat{\mu}_{a,s_a} \ge \mu_a + c_{t,s_a}, \qquad \text{(E3) } \mu^\star < \mu_a + 2c_{t,s_a},$$

for if none holds, $\hat{\mu}_{\star,s} + c_{t,s} > \mu^\star \ge \mu_a + 2c_{t,s_a} > \hat{\mu}_{a,s_a} + c_{t,s_a}$. Event (E3) is impossible: $t \le T$ and $s_a \ge \ell \ge 8\ln T/\Delta_a^2$ give $2c_{t,s_a} \le 2\sqrt{2\ln T \cdot \Delta_a^2/(8\ln T)} = \Delta_a$. By Hoeffding's inequality, for fixed s and s_a, (E1) and (E2) have probability at most $e^{-2s\cdot 2\ln t/s} = t^{-4}$. Since the counts are random, we take a union bound over their possible values $1 \le s, s_a \le t$:

$$\mathbb{E}[N_a(T)] \le \ell + \sum_{t=K+1}^{T} \sum_{s=1}^{t} \sum_{s_a=1}^{t} 2\,t^{-4} \le \ell + 2\sum_{t\ge 1} t^{-2} = \ell + \frac{\pi^2}{3},$$

and $\ell \le 8\ln T/\Delta_a^2 + 1$. The regret bound follows from Proposition 2.2. □

In words. To tell apart two means separated by Δ_a, the error bars must be narrower than Δ_a, which takes about $1/\Delta_a^2$ samples since they shrink like $1/\sqrt{n}$. The factor $\ln T$ is the price of confidence: to be wrong with probability about $1/T$ only, the bars must be widened by $\sqrt{\ln T}$. Close arms are played often but each play costs little; the regret of arm a is $\ln T/\Delta_a$, large only when the gap is small, and it never exceeds the trivial bound $\Delta_a T$.

The regret of UCB1 grows like $\ln T$, and this is optimal up to the constant:

Theorem 2.7. (Lai–Robbins lower bound (stated without proof))

For any algorithm whose regret is $o(T^\alpha)$ for every $\alpha > 0$ on every bandit of a given parametric family, and every suboptimal arm a,

$$\liminf_{T\to\infty} \frac{\mathbb{E}[N_a(T)]}{\ln T} \ge \frac{1}{\mathrm{KL}(\nu_a \,\|\, \nu_\star)}, \tag{2.5}$$

where KL is the Kullback–Leibler divergence (Lai and Robbins, 1985).

In words. $\mathrm{KL}(\nu_a \,\|\, \nu_\star)$ measures how much evidence one reward of arm a brings that a is not the best arm. Any algorithm that does well on every bandit must gather about $\ln T$ units of this evidence against each suboptimal arm, hence play it about $\ln T / \mathrm{KL}$ times. Arms whose rewards look alike are hard to rule out.

2.4 Thompson sampling

Thompson sampling (1933) is Bayesian: maintain a posterior distribution over the mean of each arm, sample one value from each posterior, and play the arm whose sample is largest. Each arm is played with the posterior probability that it is the best.

In words. An arm that is well known has a narrow posterior, and its sample is close to its estimate. An arm that is poorly known has a wide posterior, and now and then its sample is large, so it gets tried. An arm that is surely worse almost never wins the draw. The randomness of the samples plays the role of the error bars of UCB, and it is calibrated by the data instead of by a worst-case formula.

Thompson sampling for Bernoulli rewards

$S_a \leftarrow 0$, $F_a \leftarrow 0$ for every arm ▷ successes and failures
for $t = 1, \dots, T$ **do**
  sample $\theta_a \sim \mathrm{Beta}(1 + S_a, 1 + F_a)$ for every arm
  play $A_t = \arg\max_a \theta_a$, observe $R_t \in \{0, 1\}$
  $S_{A_t} \leftarrow S_{A_t} + R_t$; $\quad F_{A_t} \leftarrow F_{A_t} + 1 - R_t$

The $\mathrm{Beta}(1 + S, 1 + F)$ distribution is the posterior of a Bernoulli mean under a uniform prior after S successes and F failures. Thompson sampling attains the Lai–Robbins bound for Bernoulli bandits (Kaufmann, Korda and Munos, 2012; Agrawal and Goyal, 2012), and it is often the best performer in practice.

Remark 2.8. (A testbed)

The script `Course/code/bandits.py` compares the algorithms on Bernoulli bandits with $K = 10$ arms whose means are drawn uniformly in $[0, 1]$, averaging the regret over 100 such bandits:

Regret after T rounds	100	1 000	10 000	100 000
Greedy	8.8	28.2	218.6	2121.9
ε-greedy, $\varepsilon = 0.1$	12.2	57.1	441.0	4177.5
ε-greedy, $\varepsilon = 0.01$	9.8	36.5	227.2	857.0
UCB1	27.6	132.8	362.1	692.9
Thompson sampling	13.9	27.4	44.6	64.1

Greedy and ε-greedy have a linear regret: each tenfold increase of T multiplies it by about 10 (by 4 for $\varepsilon = 0.01$, which is still in transition). Thompson sampling adds about 19 per tenfold increase: the logarithmic growth of the theory. UCB1 is also sublinear but has a large constant: its bonus $\sqrt{2 \ln t / n}$, designed for the worst case of rewards in $[0, 1]$, keeps exploring long after the arms are well estimated, so it is overtaken by $\varepsilon = 0.01$ at first and only passes it after 10 000 rounds. Guarantees of order $\ln T$ say nothing about the constants.

2.5 Gradient bandits

Another approach learns *preferences* $H(a)$ and plays the softmax policy $\pi(a) = e^{H(a)}/\sum_b e^{H(b)}$. After playing A_t and receiving R_t, the *gradient bandit* algorithm updates

$$H(b) \leftarrow H(b) + \alpha(R_t - \bar{R}_t)\,(\mathbb{1}[b = A_t] - \pi(b)) \qquad \text{for every arm } b, \tag{2.6}$$

where $\bar{R}_t$ is a *baseline*, for example the average of the past rewards.

> **In words.** If the reward is better than usual ($R_t > \bar{R}_t$), the played arm becomes more likely and the others less likely, each in proportion to its probability; if it is worse, the opposite. The baseline turns "was this reward good?" into "was it better than expected?". Without it, every positive reward would push up whatever arm was played.

Proposition 2.9. (The gradient bandit is stochastic gradient ascent)

Let $J(H) = \sum_a \pi(a)\mu_a$ be the expected reward. For any baseline b that does not depend on A_t, the expected update of (2.6) is $\alpha\,\nabla_H J$.

Proof. By Proposition 8.1 (proved in Chapter 8), $\partial\pi(a)/\partial H(b) = \pi(a)(\mathbb{1}[a = b] - \pi(b))$. Hence

$$\frac{\partial J}{\partial H(b)} = \sum_a \mu_a\,\pi(a)\,(\mathbb{1}[a = b] - \pi(b)) = \mathbb{E}\left[\mu_{A_t}\,(\mathbb{1}[A_t = b] - \pi(b))\right].$$

Given A_t, $\mathbb{E}[R_t] = \mu_{A_t}$, and the baseline term has mean zero: $\mathbb{E}[\mathbb{1}[A_t = b] - \pi(b)] = \pi(b) - \pi(b) = 0$. □

This is the one-state case of the policy gradient theorem (Theorem 8.2): the baseline reduces the variance without changing the expected update, exactly as in actor-critic methods.

2.6 Contextual bandits

In a *contextual bandit*, a context x_t (a user, a page, a situation) is observed before choosing the arm, and the reward depends on both. It is an MDP whose next state does not depend on the action: decisions have no long-term consequences, but the policy must generalise over contexts. LinUCB (Li et al., 2010) assumes $\mathbb{E}[R \mid x, a] = \theta_a^\top x$, estimates θ_a by ridge regression with matrix $M_a = \lambda I + \sum_{t:A_t=a} x_t x_t^\top$, and plays

$$\arg\max_a \left(\hat{\theta}_a^\top x_t + \beta\sqrt{x_t^\top M_a^{-1} x_t}\right), \tag{2.7}$$

the bonus measuring the uncertainty of the estimate in the direction of the context. Contextual bandits are the standard model of recommendation and online advertising.

> **In words.** $\sqrt{x^\top M_a^{-1} x}$ is, up to the noise level, the standard deviation of the predicted reward $\hat{\theta}_a^\top x$. It is small for contexts similar to those already seen with arm a (directions where M_a is large), and large for new kinds of contexts. It is the error bar of UCB, for a linear model.

2.7 Summary

- The regret is the sum over suboptimal arms of the gap times the number of plays; constant exploration gives a linear regret.

- UCB1 explores optimistically and has a regret in $O(\ln T)$, the optimal order (Lai–Robbins); Thompson sampling samples from the posterior, is asymptotically optimal and usually better in practice.
- The gradient bandit is a one-state policy gradient method, with a baseline.
- Exploration in MDPs is harder: an action can be informative because of where it leads, not only through its immediate reward (Chapter 13).

Chapter 3

Markov decision processes

This chapter gives the mathematical framework of reinforcement learning: Markov decision processes (MDPs), returns, policies and value functions. Its central results are the Bellman equations, which characterise the value of a policy and the value of an optimal policy as fixed points of contractions; every algorithm of this course is built on them.

Throughout the theory we assume, as in Pacman, that the sets of states and actions are finite, and that the rewards are bounded.

3.1 Markov decision processes

Definition 3.1. (Markov decision process)

A (finite) *Markov decision process* is a tuple $(\mathcal{S}, \mathcal{A}, p, \gamma)$ where

- $\mathcal{S}$ is a finite set of *states* and $\mathcal{A}$ a finite set of *actions*;
- $p(s', r \mid s, a)$ is the probability that the environment moves to state s' and gives reward r when action a is taken in state s, the rewards belonging to a finite set $\mathcal{R} \subset \mathbb{R}$:

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) = 1 \qquad \text{for all } s \in \mathcal{S},\ a \in \mathcal{A}; \tag{3.1}$$

- $\gamma \in [0, 1)$ is the *discount factor*.

The process generates random variables $S_0, A_0, R_1, S_1, A_1, R_2, \ldots$ such that, for every t and every history,

$$\mathbb{P}\left(S_{t+1} = s', R_{t+1} = r \,\middle|\, S_t = s, A_t = a, H_{t-1}\right) = p(s', r \mid s, a), \tag{3.2}$$

where $H_{t-1} = (S_0, A_0, R_1, \ldots, S_{t-1}, A_{t-1}, R_t)$ is the past. Equation (3.2) is the *Markov property*: the future depends on the past only through the current state and action.

In words. The state is a summary of the past that is enough to predict the future, like a chess position: how the pieces got there does not matter for what can happen next. The Markov property is a property of the *choice* of state as much as of the system: a process that is not Markov in some variables becomes Markov when the state includes enough of the history (the direction of the ghosts in game `v2`, Remark 3.3).

Two quantities derived from p are used constantly: the *transition probabilities* and the *expected rewards*

$$p(s' \mid s, a) = \sum_{r} p(s', r \mid s, a), \qquad r(s, a) = \mathbb{E}\left[R_{t+1} \mid S_t = s, A_t = a\right] = \sum_{s', r} r\, p(s', r \mid s, a). \tag{3.3}$$

Episodes. A game ends in a *terminal* state. It is convenient to model a terminal state as an *absorbing* state: every action keeps the process in it with reward 0. With this convention all the trajectories are infinite, and the episodic and continuing settings are handled by the same equations.

Pacman 3.2. (The game as an MDP)

A state of game `v1` is the configuration of the board: Pacman's cell, the cells of the ghosts still in the game, the set of remaining cookies, the set of remaining power cookies and Pacman's power credit $c \in \{0, \dots, 12\}$; plus two absorbing states, "won" and "lost". The actions are $\mathcal{A} = \{\texttt{E}, \texttt{N}, \texttt{W}, \texttt{S}, \texttt{I}\}$. The randomness of p comes only from the ghosts (Section 1.4): given the configuration and Pacman's action, the probability of each new configuration is a product of the probabilities of the moves of the ghosts. The reward is a deterministic function of the configuration, the action and the new configuration (Table 1.1).
The Markov property holds because the ghosts' moves depend only on the current positions and on Pacman's power (Remark 1.1): the past adds nothing once the configuration is known.

Remark 3.3. (Observations and partial observability)

The agent does not need the configuration in its Python form: the feature planes of Section 1.6 encode exactly the same information (walls, cookies, power cookies, Pacman, each ghost, the power credit). In game `v1` the image is therefore a complete state, and the second stacked frame is redundant in principle: it would be necessary if the ghosts' moves depended on their direction, which the second frame reveals.
Game `v2` differs in two ways. Its ghosts prefer to keep their direction (Remark 1.1), so the direction of each ghost is part of the state: the second frame is now needed, and it is enough, since the displacement of a ghost between the two frames is its last move. Its eaten ghosts also come back. In a first version they reappeared on a random cell 15 moves after being eaten: the time since a ghost was eaten then decided when it came back, and it appears in no plane of the image. The process seen by the agent was a *partially observable* MDP (POMDP): the image was not a state, and stacking frames does not help (the timer can be older than two frames). In game `v2` the eaten ghosts instead wait in a ghost house, visible on the board, and once Pacman's power has ended each of them leaves it with probability $p = 0.1$ at each move. The waiting time T is geometric, $\mathbb{P}(T > k) = (1-p)^k$, hence *memoryless*:

$$\mathbb{P}(T > k+m \mid T > k) = \frac{(1-p)^{k+m}}{(1-p)^k} = (1-p)^m = \mathbb{P}(T > m).$$

How long a ghost has already waited tells nothing about when it will leave, and the image (with its power plane) is again a state.

3.2 Returns

Definition 3.4. (Return)

The *return* from time t is the discounted sum of the future rewards

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}. \tag{3.4}$$

Proposition 3.5. (Properties of the return)

Let $R_{\max} = \max_{r \in \mathcal{R}} |r|$. Then
(i) the series (3.4) converges absolutely and $|G_t| \le R_{\max}/(1-\gamma)$;
(ii) $G_t = R_{t+1} + \gamma\, G_{t+1}$.

Proof. (i) $\sum_k \gamma^k |R_{t+k+1}| \le R_{\max} \sum_k \gamma^k = R_{\max}/(1-\gamma)$. (ii) Since the series converges absolutely we may split off its first term: $G_t = R_{t+1} + \sum_{k\ge 1} \gamma^k R_{t+k+1} = R_{t+1} + \gamma \sum_{j\ge 0} \gamma^j R_{t+1+j+1} = R_{t+1} + \gamma\, G_{t+1}$. □

In words. The return is the total reward from now on, where a reward k steps ahead counts only γ^k of its value. A useful reading: if at each step the game could stop for an outside reason with probability $1-\gamma$, the expected total of the rewards actually collected would be exactly $\mathbb{E}[G_t]$; discounting is uncertainty about the future, or impatience. Property (ii) is the seed of everything that follows: the future seen from now is the next reward plus the (discounted) future seen from the next step.

The discount factor has two roles. Mathematically it makes the returns finite; practically it sets a *horizon*: a reward k steps ahead counts γ^k, and $\sum_k \gamma^k = 1/(1-\gamma)$ is the effective number of steps the agent looks ahead.

Pacman 3.6. (Choosing γ)

The agents use $\gamma = 0.99$, an effective horizon of 100 moves, about the length of a game. The reward of winning (+30) obtained 80 moves later is worth $30 \times 0.99^{80} \approx 13.4$ now: still enough to make the agent prefer finishing the game over wandering, but less than eating a ghost now (+20). With $\gamma = 0.9$ (horizon 10 moves) the win would be worth $30 \times 0.9^{80} \approx 0.007$: the agent would be blind to it.

Remark 3.7. (Termination and truncation)

A game *aborted* after the maximal number of moves is not a terminal state of the game: the configuration reached still has a value, and the move counter, which caused the end, is not part of the state. Treating an aborted game as terminal would teach the agent that the states reached late in long games have value 0. All the algorithms of the repository therefore distinguish the two cases: after a real end, the return stops; after an abort, the return is completed with an estimate of the value of the last state (*bootstrapping*, Chapter 5).

3.3 Policies and value functions

Definition 3.8. (Policy)

A (stationary, Markov) *policy* is a function π that gives, for every state s, a probability distribution $\pi(\cdot \mid s)$ over $\mathcal{A}$; the agent following π chooses $A_t \sim \pi(\cdot \mid S_t)$. The policy is *deterministic* if each $\pi(\cdot \mid s)$ is concentrated on one action, which we then write $\pi(s)$.

Given an initial state, a policy and the MDP define the distribution of the whole trajectory; we write $\mathbb{E}_\pi$ for expectations under it.

Definition 3.9. (Value functions)

The *state-value function* and the *action-value function* of a policy π are

$$v_\pi(s) = \mathbb{E}_\pi\left[G_t \,|\, S_t = s\right], \qquad q_\pi(s,a) = \mathbb{E}_\pi\left[G_t \,|\, S_t = s, A_t = a\right]. \tag{3.5}$$

By the Markov property these expectations do not depend on t (the distribution of the future given $S_t = s$ is the same at all times). The value $v_\pi(s)$ answers "how good is it to be in s if I follow π?"; $q_\pi(s,a)$ answers "how good is it to play a in s and then follow π?".

Proposition 3.10. (Relations between v_π and q_π)

For all states s and actions a,

$$v_\pi(s) = \sum_a \pi(a \mid s)\, q_\pi(s,a), \tag{3.6}$$

$$q_\pi(s,a) = \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma\, v_\pi(s')\right] = r(s,a) + \gamma \sum_{s'} p(s' \mid s, a)\, v_\pi(s'). \tag{3.7}$$

Proof. (3.6) is the law of total expectation, conditioning on A_t, whose distribution given $S_t = s$ is $\pi(\cdot \mid s)$. For (3.7), write $G_t = R_{t+1} + \gamma G_{t+1}$ and condition on (S_{t+1}, R_{t+1}):

$$q_\pi(s,a) = \sum_{s',r} p(s', r \mid s, a)\left(r + \gamma\, \mathbb{E}_\pi\left[G_{t+1} \,|\, S_t = s, A_t = a, S_{t+1} = s', R_{t+1} = r\right]\right).$$

Given $S_{t+1} = s'$, the actions from $t+1$ on are drawn from π and the transitions from p, independently of what happened before (Markov property (3.2) and the definition of a Markov policy). Hence the conditional expectation equals $\mathbb{E}_\pi[G_{t+1} \mid S_{t+1} = s'] = v_\pi(s')$. □

In words. The value of a state is the average of the values of its actions, weighted by how often the policy chooses them. The value of an action is one step of reality (the reward and the next state, drawn by the environment) followed by the value of wherever the agent lands. v averages over the agent's choices, q over the environment's.

3.4 The Bellman expectation equations

Substituting (3.7) into (3.6), and conversely, gives equations that involve one value function only.

Theorem 3.11. (Bellman expectation equations)

For every policy π,

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma\, v_\pi(s')\right] \quad \text{for all } s, \tag{3.8}$$

$$q_\pi(s,a) = \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma \sum_{a'} \pi(a' \mid s')\, q_\pi(s', a')\right] \quad \text{for all } s, a. \tag{3.9}$$

Moreover v_π is the *unique* solution of (3.8) and q_π the unique solution of (3.9).

In words. The Bellman equation is a consistency condition, not a recipe: the value of a state must equal what one expects to get in one step plus the discounted value of where one lands. It links the value of each state to the values of its neighbours, so that a reward

at the end of a game is "felt" backwards, step by step, in all the states that lead to it. The equations form a linear system with one unknown per state, and Proposition 3.14 solves it.

The existence part is Proposition 3.10. For uniqueness, and for everything that follows, it pays to write the equations with operators.

Definition 3.12. (Bellman operators)

Let $\mathbb{R}^{\mathcal{S}}$ be the space of functions $v : \mathcal{S} \to \mathbb{R}$ with the sup norm $\|v\|_\infty = \max_s |v(s)|$. The *Bellman expectation operator* of π and the *Bellman optimality operator* are the maps $\mathbb{R}^{\mathcal{S}} \to \mathbb{R}^{\mathcal{S}}$

$$(\mathcal{T}^\pi v)(s) = \sum_a \pi(a \mid s) \left[r(s,a) + \gamma \sum_{s'} p(s' \mid s,a)\, v(s') \right], \tag{3.10}$$

$$(\mathcal{T}^\star v)(s) = \max_a \left[r(s,a) + \gamma \sum_{s'} p(s' \mid s,a)\, v(s') \right]. \tag{3.11}$$

With this notation, (3.8) reads $v_\pi = \mathcal{T}^\pi v_\pi$: v_π is a fixed point of $\mathcal{T}^\pi$.

Lemma 3.13. (Monotonicity and contraction)

For all $u, v \in \mathbb{R}^{\mathcal{S}}$:
(i) if $u \le v$ (pointwise) then $\mathcal{T}^\pi u \le \mathcal{T}^\pi v$ and $\mathcal{T}^\star u \le \mathcal{T}^\star v$;
(ii) $\|\mathcal{T}^\pi u - \mathcal{T}^\pi v\|_\infty \le \gamma \|u - v\|_\infty$ and $\|\mathcal{T}^\star u - \mathcal{T}^\star v\|_\infty \le \gamma \|u - v\|_\infty$.

Proof. (i) The coefficients $\pi(a \mid s)$ and $\gamma\, p(s' \mid s,a)$ are nonnegative, so each bracket is a nondecreasing function of v, and so are their weighted sum and their maximum.

(ii) For $\mathcal{T}^\pi$: for every s,

$$\begin{aligned} |(\mathcal{T}^\pi u)(s) - (\mathcal{T}^\pi v)(s)| &= \gamma \left| \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s,a)\, (u(s') - v(s')) \right| \\ &\le \gamma \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s,a)\, \|u - v\|_\infty , \end{aligned}$$

and the double sum of probabilities equals 1. For $\mathcal{T}^\star$ we use the elementary inequality, valid for any functions f, g on a finite set,

$$\left| \max_a f(a) - \max_a g(a) \right| \le \max_a |f(a) - g(a)| . \tag{3.12}$$

(Let a_f maximise f; then $\max f - \max g \le f(a_f) - g(a_f) \le \max_a |f(a) - g(a)|$, and symmetrically.) Applying it to $f(a) = r(s,a) + \gamma \sum_{s'} p(s' \mid s,a)\, u(s')$ and the corresponding g with v, the rewards cancel and we conclude as for $\mathcal{T}^\pi$. □

In words. Applying $\mathcal{T}^\pi$ to a guess v means: look one step ahead with the true rewards and dynamics, and use the guess only for the states one step later. Whatever error the guess had, it now enters only through the next states, discounted by γ: each application pushes the error one step into the future, where it counts less. Monotonicity says that a better guess for the future gives a better estimate now. The inequality (3.12) says that taking the best action cannot amplify errors: if every option is known to within δ, the best option is also known to within δ.

Proof of uniqueness in Theorem 3.11. If $v = \mathcal{T}^\pi v$ and $w = \mathcal{T}^\pi w$, then

$$\|v - w\|_\infty = \|\mathcal{T}^\pi v - \mathcal{T}^\pi w\|_\infty \le \gamma \|v - w\|_\infty ,$$

so $\|v - w\|_\infty = 0$ because $\gamma < 1$. The equation for q_π is treated in the same way on $\mathbb{R}^{\mathcal{S}\times\mathcal{A}}$. □

Proposition 3.14. (Matrix form)

Number the states, and let $r_\pi \in \mathbb{R}^{\mathcal{S}}$ and $P_\pi \in \mathbb{R}^{\mathcal{S}\times\mathcal{S}}$ be defined by $r_\pi(s) = \sum_a \pi(a \mid s)\, r(s,a)$ and $P_\pi(s,s') = \sum_a \pi(a \mid s)\, p(s' \mid s,a)$. Then $I - \gamma P_\pi$ is invertible and

$$v_\pi = (I - \gamma P_\pi)^{-1} r_\pi = \sum_{k=0}^{\infty} \gamma^k P_\pi^k\, r_\pi. \tag{3.13}$$

Proof. The Bellman equation reads $v_\pi = r_\pi + \gamma P_\pi v_\pi$. The matrix P_π is stochastic (nonnegative entries, rows summing to 1), so its operator norm for $\|\cdot\|_\infty$ is 1 and $\|\gamma P_\pi\|_\infty = \gamma < 1$. The Neumann series $\sum_k (\gamma P_\pi)^k$ then converges and is the inverse of $I - \gamma P_\pi$: indeed $(I - \gamma P_\pi)\sum_{k=0}^{n}(\gamma P_\pi)^k = I - (\gamma P_\pi)^{n+1} \to I$. The term $\gamma^k P_\pi^k r_\pi$ is the expected discounted reward collected k steps ahead. □

In words. $P_\pi^k(s,s')$ is the probability of being in s' after k steps from s, so $P_\pi^k r_\pi$ is the reward expected at step k. The value is the sum of the rewards expected at each future step, discounted: the series $\sum_k \gamma^k P_\pi^k r_\pi$ is the definition of the value, and $(I - \gamma P_\pi)^{-1}$ is its closed form, the matrix analogue of $1 + x + x^2 + \cdots = 1/(1-x)$.

3.5 Optimal policies

Value functions order policies: we say that $\pi \ge \pi'$ if $v_\pi(s) \ge v_{\pi'}(s)$ for *all* states s. It is not obvious that a policy better than all the others in every state exists; the Bellman optimality equation shows that it does.

Definition 3.15. (Optimal value functions)

The *optimal value functions* are

$$v_\star(s) = \max_\pi v_\pi(s), \qquad q_\star(s,a) = \max_\pi q_\pi(s,a), \tag{3.14}$$

the maxima being over all stationary Markov policies. A policy $\pi_\star$ is *optimal* if $v_{\pi_\star} = v_\star$.

Theorem 3.16. (Bellman optimality equation)

(i) The operator $\mathcal{T}^\star$ has a unique fixed point $v^\star$, and for every $v \in \mathbb{R}^{\mathcal{S}}$, $(\mathcal{T}^\star)^n v \to v^\star$.

(ii) Let $\pi_\star$ be a *greedy* policy with respect to $v^\star$: for every s, $\pi_\star(s)$ is an action maximising $r(s,a) + \gamma \sum_{s'} p(s' \mid s,a)\, v^\star(s')$. Then $v_{\pi_\star} = v^\star$ and $v_\pi \le v^\star$ for every policy π. Hence $v_\star = v^\star$, and a deterministic optimal policy exists.

(iii) The optimal action values are $q_\star(s,a) = r(s,a) + \gamma \sum_{s'} p(s' \mid s,a)\, v_\star(s')$, and they satisfy

$$v_\star(s) = \max_a q_\star(s,a), \qquad q_\star(s,a) = \sum_{s',r} p(s',r \mid s,a)\left[r + \gamma \max_{a'} q_\star(s',a')\right]. \tag{3.15}$$

Proof. (i) $\mathbb{R}^{\mathcal{S}}$ with $\|\cdot\|_\infty$ is complete and $\mathcal{T}^\star$ is a γ-contraction (Lemma 3.13); the Banach fixed point theorem gives the unique fixed point and the convergence of the iterates. (We recall its

proof: $\|(\mathcal{T}^\star)^{n+1}v - (\mathcal{T}^\star)^n v\|_\infty \le \gamma^n \|\mathcal{T}^\star v - v\|_\infty$, so the iterates form a Cauchy sequence; its limit is a fixed point by continuity of $\mathcal{T}^\star$, unique by the argument used for Theorem 3.11.)

(ii) By definition of the greedy policy, $\mathcal{T}^{\pi_\star}v^\star = \mathcal{T}^\star v^\star = v^\star$: $v^\star$ is a fixed point of $\mathcal{T}^{\pi_\star}$, whose unique fixed point is $v_{\pi_\star}$ (Theorem 3.11); so $v_{\pi_\star} = v^\star$. Now let π be any policy. For every v, $\mathcal{T}^\pi v \le \mathcal{T}^\star v$, because a weighted average of the brackets is at most their maximum. Hence $\mathcal{T}^\pi v^\star \le \mathcal{T}^\star v^\star = v^\star$, and by monotonicity (Lemma 3.13), by induction, $(\mathcal{T}^\pi)^n v^\star \le v^\star$ for all n: if $(\mathcal{T}^\pi)^n v^\star \le v^\star$ then $(\mathcal{T}^\pi)^{n+1}v^\star \le \mathcal{T}^\pi v^\star \le v^\star$. Since $(\mathcal{T}^\pi)^n v^\star \to v_\pi$ (Banach again, for $\mathcal{T}^\pi$), we get $v_\pi \le v^\star$.

(iii) By (3.7) applied to $\pi_\star$, the right-hand side is $q_{\pi_\star}(s,a)$. For any policy π, (3.7) and $v_\pi \le v_\star$ give $q_\pi(s,a) \le q_{\pi_\star}(s,a)$; so $q_\star = q_{\pi_\star}$. The fixed point equation $v_\star = \mathcal{T}^\star v_\star$ is the first identity of (3.15); substituting it into the definition of $q_\star$ gives the second. □

In words. This is Bellman's *principle of optimality*: whatever the first move, an optimal plan must continue optimally from wherever it lands, so the value of playing well is the best, over the first move, of the reward plus the value of playing well afterwards. $q_\star(s,a)$ answers the question "what if I play a now and play perfectly afterwards?", and the best action is the one with the best answer. Each application of $\mathcal{T}^\star$ makes the agent look one move further ahead, and the contraction says that, far enough ahead, the starting guess no longer matters.

Remark 3.17. (What the theorem says)

An optimal policy needs no randomness and no memory: in each state it is enough to play an action maximising $q_\star(s,\cdot)$. Knowing $q_\star$ is therefore enough to act optimally, *without* knowing the dynamics p; this is the starting point of Q-learning and DQN (Chapter 7). Knowing $v_\star$ only is not enough to act: choosing the greedy action requires p, i.e. a model of the environment. (One can also show that policies depending on the whole history cannot do better than $v_\star$; see Puterman, *Markov Decision Processes*, 1994, Theorem 6.2.10.)

Pacman 3.18. (To the power cookie, or away?)

Pacman is at a junction s, with a ghost two cells away. Consider two options, with simplified consequences.

- a_1, *run to the power cookie*: with probability $1-\rho$ Pacman gets there first, receives about 5 and reaches a state worth 25 (the ghost will be eaten); with probability ρ the ghost catches him (-35, game over).
- a_2, *flee down the corridor*: Pacman pays the living cost -0.15; with probability 0.6 the ghost follows and Pacman is back in a situation like s; with probability 0.4 he escapes to a state worth 10.

With $\gamma = 0.99$, (3.15) gives

$$q_\star(s,a_1) = (1-\rho)(5 + 0.99 \times 25) - 35\,\rho, \qquad q_\star(s,a_2) = -0.15 + 0.99\,(0.6\,v_\star(s) + 0.4 \times 10)\,,$$

with $v_\star(s) = \max(q_\star(s,a_1), q_\star(s,a_2))$: the value of fleeing depends on the value of the state itself.

- For $\rho = 0.3$: $q_\star(s,a_1) = 10.325$. Assuming $v_\star(s) = 10.325$ gives $q_\star(s,a_2) = 9.943 < 10.325$, which is consistent: run to the power cookie, $v_\star(s) = 10.325$.
- For $\rho = 0.4$: $q_\star(s,a_1) = 3.85$. If fleeing is optimal, $v_\star(s) = q_\star(s,a_2)$ solves $q = -0.15 + 0.99(0.6\,q + 4)$, i.e. $q = 3.81/0.406 = 9.384 > 3.85$, which is consistent: flee.

A 10% change in the risk changes the optimal decision. Learning such trade-offs from experience is what the agents of the repository do.

3.6 Other criteria

The discounted return is not the only possible objective.

- **Finite horizon.** With a fixed number H of steps and no discount, the optimal value depends on the number of steps left: v_h for h steps to go satisfies $v_0 = 0$ and $v_h = \mathcal{T}^\star v_{h-1}$ (with $\gamma = 1$), and the optimal policy is non-stationary. This is backward induction, the original form of dynamic programming (Bellman, 1957).
- **Undiscounted episodic tasks.** With $\gamma = 1$, the returns are finite if every policy reaches a terminal state with probability 1 (*proper* policies); the results of this chapter extend to this *stochastic shortest path* setting (Bertsekas and Tsitsiklis, 1996).
- **Average reward.** For continuing tasks one can maximise $\lim_{T\to\infty} \frac{1}{T}\mathbb{E}[\sum_{t=1}^{T} R_t]$. The values are then defined relative to the average reward $\bar{r}_\pi$, $v_\pi(s) = \mathbb{E}_\pi[\sum_t (R_{t+1} - \bar{r}_\pi) \mid S_0 = s]$, and satisfy $v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)[r - \bar{r}_\pi + v_\pi(s')]$. Discounting with γ close to 1 is a common approximation of this criterion.

3.7 Summary

- An MDP is described by states, actions, a transition-reward kernel p satisfying the Markov property, and a discount factor γ.
- The value functions v_π and q_π are expected returns; they are the unique fixed points of the Bellman expectation operators.
- The optimal value $v_\star$ is the unique fixed point of the Bellman optimality operator, a γ-contraction; any greedy policy with respect to $v_\star$ (or $q_\star$) is optimal.
- For Pacman, the state is the board configuration; the feature planes of game `v1` contain it entirely, while game `v2` is partially observable.

The next chapters turn these equations into algorithms: first when p is known (dynamic programming), then when it must be replaced by experience.

Chapter 4

Dynamic programming

When the dynamics p of the MDP are known, the Bellman equations of Chapter 3 can be solved by iterating the Bellman operators: this is *dynamic programming* (DP). DP is rarely applicable as such (the dynamics are often unknown, and the number of states is usually huge, as in Pacman), but every RL algorithm of this course is an approximation of one of its two schemes, policy iteration and value iteration. This chapter presents them with complete convergence proofs and solves exactly a reduced version of Pacman.

4.1 Policy evaluation

Policy evaluation computes v_π for a given policy π. By Theorem 3.11 it is the unique fixed point of the contraction $\mathcal{T}^\pi$, so the iteration $v_{k+1} = \mathcal{T}^\pi v_k$ converges to it from any starting point.

Iterative policy evaluation

Require: policy π, tolerance $\varepsilon > 0$
$v(s) \leftarrow 0$ for all s
repeat
  $\Delta \leftarrow 0$
  for each state s **do**
    $u \leftarrow \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\,[r + \gamma\, v(s')]$
    $\Delta \leftarrow \max(\Delta, |u - v(s)|)$; $v(s) \leftarrow u$
until $\Delta < \varepsilon$
return v

The algorithm above updates v *in place*: the new value of a state is used immediately for the following states of the same sweep. This variant (Gauss–Seidel) also converges, usually faster; we prove the convergence of the synchronous variant, which is a direct consequence of the contraction.

Proposition 4.1. (Convergence of policy evaluation)

Let $v_{k+1} = \mathcal{T}^\pi v_k$. Then $\|v_k - v_\pi\|_\infty \le \gamma^k \|v_0 - v_\pi\|_\infty$, and if $\|v_{k+1} - v_k\|_\infty \le \varepsilon$ then

$$\|v_{k+1} - v_\pi\|_\infty \le \frac{\gamma\,\varepsilon}{1-\gamma}. \tag{4.1}$$

Proof. The first inequality follows by induction from $\|v_{k+1} - v_\pi\|_\infty = \|\mathcal{T}^\pi v_k - \mathcal{T}^\pi v_\pi\|_\infty \le$

$\gamma \|v_k - v_\pi\|_\infty$. For the second,

$$\begin{aligned}\|v_{k+1} - v_\pi\|_\infty &= \|\mathcal{T}^\pi v_k - \mathcal{T}^\pi v_\pi\|_\infty \le \gamma \|v_k - v_\pi\|_\infty \\ &\le \gamma \left(\|v_k - v_{k+1}\|_\infty + \|v_{k+1} - v_\pi\|_\infty\right) \le \gamma\,\varepsilon + \gamma \|v_{k+1} - v_\pi\|_\infty ,\end{aligned}$$

and we solve for $\|v_{k+1} - v_\pi\|_\infty$. □

In words. A small change between two sweeps does not mean that the values are almost right: each sweep only reduces the error by the factor γ, so the remaining error can be $\gamma/(1-\gamma)$ times the last change, 99 times with $\gamma = 0.99$. Slowly changing values may still be far from the answer, and the stopping tolerance must be chosen accordingly.

4.2 Policy improvement

Knowing v_π, can we find a better policy? The natural candidate is the policy that is greedy with respect to q_π: in each state, play the action that is best if π is followed afterwards.

Theorem 4.2. (Policy improvement)

Let π and π' be two policies such that, for every state s,

$$\sum_a \pi'(a \mid s)\, q_\pi(s,a) \ge v_\pi(s). \tag{4.2}$$

Then $v_{\pi'} \ge v_\pi$. If moreover the inequality (4.2) is strict in some state s, then $v_{\pi'}(s) > v_\pi(s)$.

Proof. The left-hand side of (4.2) is $(\mathcal{T}^{\pi'} v_\pi)(s)$, by (3.7). The hypothesis is therefore $\mathcal{T}^{\pi'} v_\pi \ge v_\pi$. By monotonicity of $\mathcal{T}^{\pi'}$ (Lemma 3.13) and induction, $(\mathcal{T}^{\pi'})^n v_\pi \ge (\mathcal{T}^{\pi'})^{n-1} v_\pi \ge \cdots \ge v_\pi$ for every n, and $(\mathcal{T}^{\pi'})^n v_\pi \to v_{\pi'}$ as $n \to \infty$. Hence $v_{\pi'} \ge \mathcal{T}^{\pi'} v_\pi \ge v_\pi$, the first inequality being the limit of $(\mathcal{T}^{\pi'})^n v_\pi \ge \mathcal{T}^{\pi'} v_\pi$. If $(\mathcal{T}^{\pi'} v_\pi)(s) > v_\pi(s)$, then $v_{\pi'}(s) \ge (\mathcal{T}^{\pi'} v_\pi)(s) > v_\pi(s)$. □

In words. The hypothesis compares two plans that differ only in the first move: play π' once, then return to π, versus play π throughout. If deviating for one move never hurts, then deviating at every move does not hurt either: each deviation starts from a state where it is at least as good, and the gains add up along the trajectory.

Corollary 4.3. (Greedy improvement)

Let π' be greedy with respect to q_π: $\pi'(s) \in \arg\max_a q_\pi(s,a)$ for every s. Then $v_{\pi'} \ge v_\pi$. If $v_{\pi'} = v_\pi$, then π and π' are both optimal.

Proof. $q_\pi(s, \pi'(s)) = \max_a q_\pi(s,a) \ge \sum_a \pi(a \mid s)\, q_\pi(s,a) = v_\pi(s)$, so Theorem 4.2 applies. If $v_{\pi'} = v_\pi$, then by (3.7) $q_{\pi'} = q_\pi$, and

$$v_{\pi'}(s) = q_{\pi'}(s, \pi'(s)) = q_\pi(s, \pi'(s)) = \max_a q_\pi(s,a) = \max_a \left[r(s,a) + \gamma \sum_{s'} p(s' \mid s,a)\, v_{\pi'}(s') \right] :$$

$v_{\pi'}$ is a fixed point of $\mathcal{T}^\star$, hence equals $v_\star$ (Theorem 3.16). □

In words. To improve a policy it suffices to ask, in every state, "which action would be best if I followed my current policy afterwards?", and to play it. If the answer is always the action already played, no one-move deviation helps, and then no deviation of any kind

helps: the policy is optimal. Optimality can be checked one move at a time.

4.3 Policy iteration

Alternating evaluation and greedy improvement gives *policy iteration*.

Policy iteration

choose any deterministic policy π_0
for $k = 0, 1, 2, \dots$ **do**
  evaluation: compute v_{π_k} (Proposition 3.14 or iterative policy evaluation)
  improvement: $\pi_{k+1}(s) \leftarrow \arg\max_a \left[r(s,a) + \gamma \sum_{s'} p(s' \mid s,a)\, v_{\pi_k}(s')\right]$, keeping $\pi_k(s)$ in case of a tie
  if $\pi_{k+1} = \pi_k$ **then return** π_k, v_{π_k}

Theorem 4.4. (Convergence of policy iteration)

Policy iteration stops after a finite number of iterations, at most the number $|\mathcal{A}|^{|\mathcal{S}|}$ of deterministic policies, and returns an optimal policy.

Proof. If $\pi_{k+1} \neq \pi_k$, then because ties keep the previous action, $\pi_{k+1}(s)$ strictly improves on $\pi_k(s)$ in some state: $q_{\pi_k}(s, \pi_{k+1}(s)) > q_{\pi_k}(s, \pi_k(s)) = v_{\pi_k}(s)$. By Theorem 4.2, $v_{\pi_{k+1}} \geq v_{\pi_k}$ with strict inequality in s. The value functions $v_{\pi_0}, v_{\pi_1}, \dots$ therefore strictly increase (for the partial order of functions), so no policy appears twice; since there are finitely many deterministic policies, the algorithm stops. When it stops, π_k is greedy with respect to its own q_{π_k}, so $\mathcal{T}^\star v_{\pi_k} = \mathcal{T}^{\pi_k} v_{\pi_k} = v_{\pi_k}$: $v_{\pi_k} = v_\star$ and π_k is optimal. □

The bound $|\mathcal{A}|^{|\mathcal{S}|}$ is very pessimistic: in practice policy iteration converges in a handful of iterations, as the Pacman example below shows.

4.4 Value iteration

Policy iteration evaluates each policy completely before improving it. *Value iteration* goes to the other extreme: it applies the Bellman optimality operator directly, $v_{k+1} = \mathcal{T}^\star v_k$, which amounts to one sweep of evaluation interleaved with an improvement in every state.

Value iteration

Require: tolerance $\varepsilon > 0$
$v(s) \leftarrow 0$ for all s
repeat
  $v_{\text{old}} \leftarrow v$
  for each state s **do**
    $v(s) \leftarrow \max_a \left[r(s,a) + \gamma \sum_{s'} p(s' \mid s,a)\, v_{\text{old}}(s')\right]$
until $\|v - v_{\text{old}}\|_\infty < \varepsilon$
return v and a greedy policy with respect to v

Theorem 4.5. (Convergence of value iteration)

Let $v_{k+1} = \mathcal{T}^\star v_k$.
(i) $\|v_k - v_\star\|_\infty \leq \gamma^k \|v_0 - v_\star\|_\infty$.

(ii) If $\|v_{k+1} - v_k\|_\infty \le \varepsilon$, then $\|v_{k+1} - v_\star\|_\infty \le \gamma\varepsilon/(1-\gamma)$.
(iii) If $\|v - v_\star\|_\infty \le \delta$ and π is greedy with respect to v, then $\|v_\pi - v_\star\|_\infty \le 2\gamma\delta/(1-\gamma)$.

Proof. (i) and (ii) are proved exactly as Proposition 4.1, with $\mathcal{T}^\star$ and its fixed point $v_\star$ in place of $\mathcal{T}^\pi$ and v_π.

(iii) Since π is greedy with respect to v, $\mathcal{T}^\pi v = \mathcal{T}^\star v$. Then

$$\begin{aligned}\|v_\pi - v_\star\|_\infty &\le \|\mathcal{T}^\pi v_\pi - \mathcal{T}^\pi v\|_\infty + \|\mathcal{T}^\star v - \mathcal{T}^\star v_\star\|_\infty \le \gamma\,\|v_\pi - v\|_\infty + \gamma\,\|v - v_\star\|_\infty \\ &\le \gamma\,(\|v_\pi - v_\star\|_\infty + \|v_\star - v\|_\infty) + \gamma\,\delta \le \gamma\,\|v_\pi - v_\star\|_\infty + 2\gamma\,\delta,\end{aligned}$$

using $v_\pi = \mathcal{T}^\pi v_\pi$ and $v_\star = \mathcal{T}^\star v_\star$ in the first line. Solving for $\|v_\pi - v_\star\|_\infty$ gives the result. □

In words. Statement (iii) is what matters in practice: approximate values give good decisions. An error δ in the values can make the agent choose an action that is worse by about 2δ (one action overestimated by δ, the best one underestimated by δ), and such losses can occur at every step, hence the factor of the horizon $1/(1-\gamma)$. To act well, the values need not be exact; they need to rank the actions correctly.

Asynchronous DP. Nothing forces the backups to sweep all the states in order. *Asynchronous* value iteration updates the states in any order, one at a time, using the latest values (as the in-place policy evaluation above): it still converges to $v_\star$ as long as every state is updated infinitely often (Bertsekas and Tsitsiklis, 1989). This freedom allows focusing the computation on the states that matter, for example the states visited by the current policy (*real-time dynamic programming*), or the states whose value changed most recently (*prioritised sweeping*, Chapter 11).

4.5 Linear programming

The optimal value is also the solution of a linear program, which gives another exact method and the basis of several modern algorithms.

Theorem 4.6. (LP formulation)

Let μ be any distribution with $\mu(s) > 0$ for all s. Then $v_\star$ is the unique solution of

$$\min_{v \in \mathbb{R}^{\mathcal{S}}} \sum_s \mu(s)\, v(s) \quad \text{subject to} \quad v(s) \ge r(s,a) + \gamma \sum_{s'} p(s' \mid s,a)\, v(s') \quad \text{for all } s, a. \tag{4.3}$$

Proof. The constraints say exactly $v \ge \mathcal{T}^\star v$. Then by monotonicity of $\mathcal{T}^\star$ (Lemma 3.13), $v \ge \mathcal{T}^\star v \ge (\mathcal{T}^\star)^2 v \ge \dots$, and the iterates converge to $v_\star$ (Theorem 3.16): every feasible v satisfies $v \ge v_\star$. Conversely $v_\star$ is feasible ($v_\star = \mathcal{T}^\star v_\star$). Since $\mu > 0$, $\sum_s \mu(s)v(s) \ge \sum_s \mu(s)v_\star(s)$ with equality only if $v = v_\star$. □

In words. A feasible v is an *optimistic* value: in every state it is at least what any action gives if the future is valued by v. Such a function overestimates $v_\star$ everywhere, and the linear program looks for the lowest one: the tightest "roof" over the one-step values of all the actions, which touches them exactly at $v_\star$.

The program has $|\mathcal{S}|$ variables and $|\mathcal{S}|\,|\mathcal{A}|$ constraints. Its dual has one variable $x(s,a) \ge 0$ per state-action pair, which can be interpreted as the discounted frequency of playing a in s (an *occupancy measure*); optimal policies are read from the optimal dual solution as $\pi(a \mid s) \propto x(s,a)$. This dual view underlies several offline RL methods (Chapter 12).

Remark 4.7. (Generalised policy iteration)

Policy iteration and value iteration are two extremes of a common scheme, *generalised policy iteration*: a value function is pushed towards the value of the current policy (evaluation), while the policy is pushed towards greediness with respect to the value function (improvement). The two processes compete (each changes the target of the other) and cooperate (their only joint fixed point is optimal). All the algorithms of the next chapters follow this pattern: Q-learning and DQN are approximations of value iteration, actor-critic methods and PPO of policy iteration.

4.6 Pacman solved exactly

The full game is far too large for DP (Section 4.7), but a reduced version can be solved exactly, which shows what an optimal policy does.

Pacman 4.8. (Reaching the power cookie)

On the maze `pacman_map3` of game `v1`, Pacman and *one* ghost (aggressiveness 0.74, chasing along shortest paths), no cookies. Pacman must reach a power cookie in the bottom right corner of the maze: reaching it gives +10 and ends the episode, being eaten gives −35, and each move costs 0.15, with $\gamma = 0.99$. Pacman moves first, then the ghost. The state is the pair (Pacman's cell, ghost's cell): $77 \times 77 = 5929$ states. The script `Course/code/dp_escape.py` builds the transition probabilities and solves the MDP.

- Value iteration reaches $\|v_{k+1} - v_k\|_\infty < 10^{-8}$ after 57 sweeps; policy iteration stops after 11 iterations, far below the bound 5^{5929} of Theorem 4.4. Both give the same policy, and value functions that differ by 10^{-8}.
- The optimal policy reaches the power cookie *from every starting state* with probability 1 (to 10^{-12}). The policy that follows a shortest path to the cookie, ignoring the ghost, succeeds from only 67.9% of the states on average.
- From Pacman at $(5, 7)$ with the ghost at $(7, 12)$, between him and the cookie, the shortest path succeeds with probability 0.004. The optimal policy succeeds with probability 1, in 11.4 moves on average instead of the 10 moves of the shortest path: it makes a detour around the ghost. Its value is $v_\star = 7.39$.

The fact that the optimal success probability is 1 everywhere is a property of the maze: it has no dead ends, and a single ghost that moves at Pacman's speed, after him, can never corner him. This is why the game needs (at least) two ghosts.

The heart of the script is the Bellman backup and the value iteration loop; `T[k][a]` lists the triples (probability, reward, next state) of the transition from state `k` with action `a`, the next state being −1 for a terminal state.

Listing 4.1: Value iteration (`Course/code/dp_escape.py`, excerpt)

```
def backup(v, k, a):
    # r(s,a) + gamma * sum_s' p(s'|s,a) v(s'), terminal states worth 0
    return sum(pr * (r + (GAMMA * v[s2] if s2 >= 0 else 0.0))
               for pr, r, s2 in T[k][a])

def value_iteration(tol=1e-8):
    v = np.zeros(nS); it = 0
    while True:
        it += 1
        new = np.array([max(backup(v, k, a) for a in range(nA))
                        for k in range(nS)])      # v <- T* v
```

```
        delta = np.max(np.abs(new - v)); v = new
        if delta < tol:
            return v, it
```

4.7 The curse of dimensionality

DP needs two things that are missing in practice.

- **The model.** Each backup uses $p(s', r \mid s, a)$. For Pacman, these probabilities could be derived from the code of the ghosts, but in general the environment is a black box that can only be sampled.
- **A table of all states.** Each sweep visits every state.

Pacman 4.9. (How many states?)

In game `v1`, a state specifies Pacman's cell (77 choices), the cell of each ghost or the fact that it was eaten (78 choices each), which of the 77 cells still hold a cookie (at most 2^{77} choices), and the power credit (13 values). This gives at most

$$77 \times 78^2 \times 2^{77} \times 13 \approx 9 \times 10^{29}$$

states. Most of these configurations cannot occur, but even a tiny fraction of them is far beyond what a table can store: at one byte per state, 10^{20} states would already require 100 exabytes.

The two reduced games of this chapter illustrate the jump: one ghost and no cookies give 5929 states, solved in seconds; adding the cookies multiplies the number of states by up to 2^{77}. The way out, developed in the rest of the course, is twofold: *learn from sampled experience* instead of the model (Chapter 5), and *generalise* between states with a parametrised function, a neural network reading the image of the board (Chapter 7).

Chapter 5

Monte Carlo and temporal-difference learning

Dynamic programming needs the model p. This chapter replaces the model by *experience*: trajectories obtained by playing. Monte Carlo methods average complete returns; temporal-difference (TD) methods update an estimate from the next reward and the current estimate of the next state, which is called *bootstrapping*. Their control versions, SARSA and Q-learning, are the direct ancestors of DQN. Throughout the chapter the values are stored in a table, one entry per state (or state-action pair); Chapter 7 replaces the table by a neural network.

5.1 Monte Carlo prediction

Since $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$, the most direct estimate of $v_\pi(s)$ is the average of the returns observed after visits to s in episodes played with π.

First-visit Monte Carlo prediction

Require: policy π

$V(s) \leftarrow 0$ and $N(s) \leftarrow 0$ for all s
for each episode $S_0, A_0, R_1, \dots, S_T$ played with π **do**
  $G \leftarrow 0$
  for $t = T-1, T-2, \dots, 0$ **do**
    $G \leftarrow R_{t+1} + \gamma\, G$ ▷ $G = G_t$, by Proposition 3.5
    if S_t does not appear in $S_0, \dots, S_{t-1}$ **then**
      $N(S_t) \leftarrow N(S_t) + 1$; $V(S_t) \leftarrow V(S_t) + \frac{1}{N(S_t)}\,(G - V(S_t))$

Proposition 5.1. (Incremental averages)

(i) If $V_n = V_{n-1} + \frac{1}{n}(G_n - V_{n-1})$ with V_0 arbitrary, then $V_n = \frac{1}{n}\sum_{i=1}^{n} G_i$ for $n \geq 1$.
(ii) If $V_n = V_{n-1} + \alpha(G_n - V_{n-1})$ with a constant $\alpha \in (0, 1]$, then

$$V_n = (1-\alpha)^n V_0 + \sum_{i=1}^{n} \alpha(1-\alpha)^{n-i} G_i, \tag{5.1}$$

a weighted average in which the weight of a sample decreases geometrically with its age.

Proof. (i) By induction: $V_1 = G_1$, and if V_{n-1} is the mean of the first $n-1$ samples, then $V_n = \frac{n-1}{n} V_{n-1} + \frac{1}{n} G_n = \frac{1}{n}\left(\sum_{i=1}^{n-1} G_i + G_n\right)$. (ii) Write $V_n = (1-\alpha)V_{n-1} + \alpha G_n$ and unroll the

recursion. □

In words. The update reads: move the estimate a fraction of the way towards the new sample. With $1/n$ the estimate is the plain average, which never forgets. With a constant α it is an exponential moving average, which remembers roughly the last $1/\alpha$ samples. In RL the targets change as the policy improves, so forgetting old samples is usually what we want.

Each first-visit return is an independent sample of G_t given $S_t = s$ (episodes are independent), with mean $v_\pi(s)$ and finite variance; by the strong law of large numbers, $V(s) \to v_\pi(s)$ almost surely for every state visited infinitely often. Monte Carlo estimates are *unbiased*, but their variance can be large: a return sums all the random rewards of the rest of the episode. They also require complete episodes, so that nothing is learnt before the end of a game.

5.2 Temporal-difference learning

The Bellman equation $v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma\, v_\pi(S_{t+1}) \mid S_t = s]$ suggests replacing the return G_t by the *TD target* $R_{t+1} + \gamma V(S_{t+1})$, available after a single step.

Definition 5.2. (TD(0) and the TD error)

After each transition $(S_t, A_t, R_{t+1}, S_{t+1})$ played with π, TD(0) updates

$$V(S_t) \leftarrow V(S_t) + \alpha\, \delta_t, \qquad \delta_t = R_{t+1} + \gamma\, V(S_{t+1}) - V(S_t), \tag{5.2}$$

where $\alpha \in (0, 1]$ is the *step size* and δ_t the *TD error*. If S_{t+1} is terminal, $V(S_{t+1}) = 0$; if the episode was truncated (Remark 3.7), S_{t+1} is not terminal and its estimate is used.

In words. $V(S_t)$ was a prediction of the future made at time t; one step later, $R_{t+1} + \gamma V(S_{t+1})$ is a better prediction, since one real reward is now known. The TD error δ_t is the *surprise* between the two: positive if things turned out better than expected, negative otherwise. TD learning corrects each prediction towards the next one, as a forecaster corrects Thursday's forecast for Sunday using Friday's, without waiting for Sunday. It learns a guess from a guess, hence the name *bootstrapping*.

The TD target is *biased* (it uses the current estimate V instead of v_π) but has a much smaller variance than the return: it involves one random transition instead of a whole episode. In expectation, TD(0) moves V towards $\mathcal{T}^\pi V$:

Proposition 5.3. (The expected TD(0) update is a contraction)

For $\alpha \in (0, 1]$, define $U_\alpha V = V + \alpha(\mathcal{T}^\pi V - V)$, the update obtained by replacing, in every state at once, the sampled target by its expectation. Then U_α is a contraction of modulus $1 - \alpha(1 - \gamma) < 1$ for $\|\cdot\|_\infty$, and its unique fixed point is v_π.

Proof. Given $S_t = s$, the expectation of the target is $\sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)[r + \gamma V(s')] = (\mathcal{T}^\pi V)(s)$. Now $U_\alpha V = (1 - \alpha)V + \alpha \mathcal{T}^\pi V$, so

$$\|U_\alpha V - U_\alpha W\|_\infty \le (1 - \alpha)\, \|V - W\|_\infty + \alpha\gamma\, \|V - W\|_\infty = (1 - \alpha(1 - \gamma))\, \|V - W\|_\infty .$$

A fixed point satisfies $\alpha(\mathcal{T}^\pi V - V) = 0$, i.e. $V = \mathcal{T}^\pi V$, whose only solution is v_π. □

The real algorithm uses samples and updates one state at a time; its convergence is a result of *stochastic approximation*.

Theorem 5.4. (Convergence of TD(0) (stated without proof))

Suppose every state is visited infinitely often, and the step size used for the n-th update of each state, α_n, satisfies the Robbins–Monro conditions

$$\sum_n \alpha_n = \infty, \qquad \sum_n \alpha_n^2 < \infty \tag{5.3}$$

(for example $\alpha_n = 1/n$). Then TD(0) converges to v_π with probability 1.

In words. The steps must be large enough in total to carry the estimates from any starting point to the solution ($\sum \alpha_n = \infty$), and shrink fast enough for the noise of the samples to average out ($\sum \alpha_n^2 < \infty$). The step $\alpha_n = 1/n$, which computes plain averages, satisfies both; a constant step satisfies only the first.

The proof (Dayan, 1992; Jaakkola, Jordan and Singh, 1994; Tsitsiklis, 1994) combines Proposition 5.3 with martingale arguments that control the noise of the samples: the first condition of (5.3) lets the estimates travel arbitrarily far, the second makes the noise average out. In deep RL the step size is usually kept constant, and the estimates then fluctuate around the solution instead of converging.

5.3 n-step returns and λ-returns

Monte Carlo and TD(0) are two extremes: the target uses all the rewards of the episode, or only the first one followed by an estimate. Between them,

Definition 5.5. (n-step return)

The *n-step return* from time t is

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}), \tag{5.4}$$

with $G_{t:t+n} = G_t$ if the episode ends before $t + n$.

Proposition 5.6. (Error reduction of n-step returns)

For a fixed estimate V,

$$\max_s \left| \mathbb{E}_\pi \left[G_{t:t+n} \mid S_t = s \right] - v_\pi(s) \right| \leq \gamma^n \max_s \left| V(s) - v_\pi(s) \right|. \tag{5.5}$$

Proof. $\mathbb{E}_\pi[G_{t:t+n} \mid S_t = s] = ((\mathcal{T}^\pi)^n V)(s)$: this is the Bellman expectation operator applied n times (by induction on n, as in the proof of Proposition 3.10; a terminal state contributes 0 in both expressions). Since $v_\pi = (\mathcal{T}^\pi)^n v_\pi$ and $\mathcal{T}^\pi$ is a γ-contraction, the difference is at most $\gamma^n \left\| V - v_\pi \right\|_\infty$. □

In words. An n-step return contains n real rewards and only then a guess, which is discounted by γ^n: the more reality in the target, the less the error of the guess matters. The price is noise, since each real reward is random.

The larger n, the less the target depends on the (possibly wrong) estimate V, but the more random rewards it contains: n trades bias for variance. The *λ-return* averages all the n-step

returns with geometric weights,

$$G_t^\lambda = (1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}G_{t:t+n}, \qquad \lambda \in [0,1], \tag{5.6}$$

with $G_t^0 = G_{t:t+1}$ (TD(0)) and $G_t^1 = G_t$ (Monte Carlo). We will meet it again as the generalised advantage estimator of Chapter 8.

Lemma 5.7. (The λ-return error is a sum of TD errors)

For an episode ending at time T, a fixed estimate V (with $V = 0$ at the terminal state) and $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$,

$$G_t^\lambda - V(S_t) = \sum_{k=t}^{T-1}(\gamma\lambda)^{k-t}\delta_k. \tag{5.7}$$

Proof. The n-step return telescopes: $G_{t:t+n} - V(S_t) = \sum_{l=0}^{n-1}\gamma^l\delta_{t+l}$ (the value terms cancel in pairs; beyond T the TD errors are 0). Substituting into (5.6),

$$G_t^\lambda - V(S_t) = (1-\lambda)\sum_{n\geq 1}\lambda^{n-1}\sum_{l=0}^{n-1}\gamma^l\delta_{t+l} = \sum_{l\geq 0}\gamma^l\delta_{t+l}\,(1-\lambda)\sum_{n>l}\lambda^{n-1} = \sum_{l\geq 0}(\gamma\lambda)^l\delta_{t+l},$$

for $\lambda < 1$ (the sums are finite); for $\lambda = 1$ the identity is the telescoping of the Monte Carlo return. □

In words. The correction that the λ-return applies to $V(S_t)$ is the sum of all the future surprises, those further ahead counting less. The parameter λ says how much the agent trusts its own estimates: with $\lambda = 0$ it trusts them fully and looks at the next surprise only; with $\lambda = 1$ it trusts none and waits for the whole return.

5.3.1 Eligibility traces

The λ-return needs the future of the episode (the *forward view*). The lemma suggests an equivalent *backward view*: each TD error δ_t is credited to the past states, with a weight decaying by $\gamma\lambda$ per step. The *eligibility trace* $e_t(s)$ keeps track of these weights.

Tabular TD(λ) with accumulating traces

for each episode **do**
  $e(s) \leftarrow 0$ for all s
  for each step $S_t \to S_{t+1}$ with reward R_{t+1} **do**
    $\delta \leftarrow R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$
    $e(S_t) \leftarrow e(S_t) + 1$
    for all s **do**: $V(s) \leftarrow V(s) + \alpha\,\delta\,e(s)$; $\quad e(s) \leftarrow \gamma\lambda\,e(s)$

Proposition 5.8. (Forward and backward views)

If the updates of TD(λ) are accumulated during an episode and applied at its end (offline updating), the total update of each state is that of the λ-return algorithm, $\alpha\sum_t(G_t^\lambda - V(S_t))\mathbb{1}[S_t = s]$.

Proof. The trace at time t is $e_t(s) = \sum_{k \le t} (\gamma\lambda)^{t-k} \mathbb{1}[S_k = s]$. Exchanging the sums,

$$\sum_t \alpha\, \delta_t\, e_t(s) = \alpha \sum_k \mathbb{1}[S_k = s] \sum_{t \ge k} (\gamma\lambda)^{t-k} \delta_t = \alpha \sum_k \mathbb{1}[S_k = s] \left(G_k^\lambda - V(S_k) \right),$$

by Lemma 5.7, V being fixed during the episode. □

In words. The trace of a state measures how much it is "to blame" for the current surprise: it jumps when the state is visited and fades with every step. When a surprise occurs, all the recently visited states share the correction in proportion to their trace. Recent and frequent states get more credit, a simple answer to the *credit assignment* problem.

With online updates the equivalence is only approximate; the *true online* TD(λ) of van Seijen and Sutton (2014) makes it exact. Traces cost one memory cell per state (or per feature with function approximation, where $\mathbf{e} \leftarrow \gamma\lambda\mathbf{e} + \nabla\hat{v}(S_t)$), and give the benefits of λ-returns without waiting for the end of the episode.

Pacman 5.9. (n-step returns in the repository)

The DQN agent of the repository uses 3-step returns, and the A2C/PPO agent uses the advantage version of the λ-return with $\lambda = 0.95$. In Pacman the important rewards are rare and large (eating a ghost, dying, winning): with a 1-step target, the news of a death travels back one state per update; with a 3-step target it reaches three moves back at once.

5.4 Control: SARSA and Q-learning

To *improve* the policy without a model, we estimate action values: by Remark 3.17, acting greedily with respect to q needs no model. Two issues must be solved: exploration, and the choice of the target.

Exploration. A greedy agent may never try the action that would reveal itself to be the best. The simplest remedy is the *ε-greedy* policy: with probability $1 - \varepsilon$ play a greedy action of Q, otherwise an action chosen uniformly at random.

Proposition 5.10. (ε-greedy improvement)

Let π be *ε-soft* ($\pi(a \mid s) \ge \varepsilon / |\mathcal{A}|$ for all s, a) and let π' be ε-greedy with respect to q_π. Then $v_{\pi'} \ge v_\pi$.

Proof. Let $m = |\mathcal{A}|$. For every s,

$$\begin{aligned}
\sum_a \pi'(a \mid s)\, q_\pi(s,a) &= \frac{\varepsilon}{m} \sum_a q_\pi(s,a) + (1-\varepsilon) \max_a q_\pi(s,a) \\
&\ge \frac{\varepsilon}{m} \sum_a q_\pi(s,a) + (1-\varepsilon) \sum_a \frac{\pi(a \mid s) - \varepsilon/m}{1-\varepsilon}\, q_\pi(s,a) \\
&= \sum_a \pi(a \mid s)\, q_\pi(s,a) = v_\pi(s),
\end{aligned}$$

because the weights $(\pi(a \mid s) - \varepsilon/m)/(1-\varepsilon)$ are nonnegative and sum to 1, so the weighted average is at most the maximum. Theorem 4.2 concludes. □

SARSA (on-policy). The TD update for action values, using the action actually played next:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma\, Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right]. \tag{5.8}$$

Its name lists the quintuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ it uses. With an ε-greedy policy, SARSA estimates the values of the ε-greedy policy itself, exploration included.

Q-learning (off-policy). Watkins' Q-learning uses the best next action instead of the one played:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right]. \tag{5.9}$$

The target is a sample of the Bellman optimality operator for action values, $(\mathcal{T}^\star Q)(s,a) = \sum_{s',r} p(s', r \mid s, a)[r + \gamma \max_{a'} Q(s', a')]$, which is a γ-contraction with fixed point $q_\star$ (same proof as Lemma 3.13, using (3.12)). Q-learning therefore learns $q_\star$ *whatever the policy that generates the data*, provided it explores: it is *off-policy*. This property is what allows DQN to learn from a replay memory of old transitions (Chapter 7).

> **In words.** SARSA learns how good its actual behaviour is, exploration mistakes included: near a ghost, where a random move can be fatal, its values are lower, and it learns to keep a safety margin. Q-learning learns how good its behaviour *would* be if from the next step on it always played its best action: it values states as a perfect player would, even while it is still exploring. One learns about the agent it is, the other about the agent it is trying to become.

Expected SARSA. Replacing the sampled next action of SARSA by its expectation under the policy gives

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_{a'} \pi(a' \mid S_{t+1})\, Q(S_{t+1}, a') - Q(S_t, A_t) \right], \tag{5.10}$$

which removes the variance due to the choice of A_{t+1}. With π greedy with respect to Q, expected SARSA is Q-learning; with the ε-greedy behaviour policy, it is SARSA in expectation. It can also be used off-policy, with any target policy π.

Theorem 5.11. (Convergence of Q-learning (stated without proof))

If every state-action pair is updated infinitely often and the step sizes of each pair satisfy (5.3), tabular Q-learning converges to $q_\star$ with probability 1 (Watkins and Dayan, 1992).

Pacman 5.12. (Learning the reduced game from experience)

We return to the reduced game of Example 4.8 (one ghost, reach the power cookie), now *without* using the transition probabilities: the agents only sample transitions. Each episode starts from a random state; the agents use $\varepsilon = 0.1$ and a constant $\alpha = 0.2$ (script `Course/code/td_escape.py`). We measure the success probability of the *greedy* policy of Q, computed exactly as in Chapter 4, averaged over all the starting states:

Episodes	1 000	5 000	20 000	200 000
Q-learning	0.315	0.999	1.000	1.000
SARSA	0.257	0.999	1.000	1.000

Both learn an optimal behaviour within a few thousand games (about 12 moves each), against 0.679 for the shortest path. Their values differ, as the theory predicts: after 200 000

episodes the mean distance between $\max_a Q(s,a)$ and $v_\star(s)$ is 1.2 for Q-learning, which estimates $q_\star$, and 3.7 for SARSA, which estimates the value of its ε-greedy policy: with probability 0.1 that policy plays a random move, sometimes into the ghost, so its values are lower. Q-learning's values are not exact either: with a constant step size they keep fluctuating (Theorem 5.11 requires decreasing steps).

Listing 5.1: The SARSA and Q-learning updates (`Course/code/td_escape.py`, excerpt)

```
r, k2 = sample(k, a)                    # one transition of the game
if k2 < 0:                              # terminal: the target is r
    Q[k, a] += alpha * (r - Q[k, a])
    break
a2 = eps_greedy(Q, k2, eps)             # next action, actually played
if method == "q-learning":
    target = r + GAMMA * Q[k2].max()    # off-policy: best next action
else:
    target = r + GAMMA * Q[k2, a2]      # SARSA: the action played
Q[k, a] += alpha * (target - Q[k, a])
```

5.5 Off-policy Monte Carlo and importance sampling

Q-learning learns about the greedy policy while following another one thanks to the max in its target. Monte Carlo methods have no such shortcut: a return obtained with a *behaviour* policy b is a sample of the wrong expectation. Importance sampling corrects it.

Proposition 5.13. (Importance sampling of returns)

Let π (target) and b (behaviour) be two policies with $b(a \mid s) > 0$ whenever $\pi(a \mid s) > 0$, and for an episode ending at T let

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{b(A_k \mid S_k)}. \tag{5.11}$$

Then $\mathbb{E}_b\left[\rho_{t:T-1}\, G_t \mid S_t = s\right] = v_\pi(s)$.

Proof. Given $S_t = s$, the probability of a continuation $A_t, S_{t+1}, \dots, S_T$ under a policy μ is $\prod_{k=t}^{T-1} \mu(A_k \mid S_k)\, p(S_{k+1}, R_{k+1} \mid S_k, A_k)$. The ratio of these probabilities for π and b is $\rho_{t:T-1}$: the transition probabilities cancel. Hence $\mathbb{E}_b[\rho_{t:T-1} G_t \mid S_t = s] = \sum_{\text{cont.}} P_b(\text{cont.})\, \rho_{t:T-1}\, G_t = \sum_{\text{cont.}} P_\pi(\text{cont.})\, G_t = v_\pi(s)$, the sum running over the continuations of positive probability under b, which include those of positive probability under π. □

In words. Each observed trajectory is reweighted by how much more (or less) likely the target policy would have been to produce it than the behaviour policy. Trajectories that π would often take count more, those it would rarely take count less, and the average becomes that of π. The weight is a product of one ratio per step, so over long episodes it is either tiny or huge: the source of the variance.

Averaging $\rho_{t:T-1} G_t$ over the visits of s (*ordinary* importance sampling) is unbiased, but the products of ratios can be huge, and the variance even infinite. *Weighted* importance sampling divides by the sum of the ratios instead of the number of visits: it is biased (the bias vanishes asymptotically) but its variance is bounded, and it is usually preferred. The same ratios, truncated to a single step, appear in the surrogate objective of PPO (Chapter 9).

5.6 The maximisation bias

The target of Q-learning takes the maximum of noisy estimates, which is biased upwards.

Proposition 5.14. (Maximisation bias)

Let $X_1, \dots, X_m$ be random variables with finite means. Then $\mathbb{E}\left[\max_i X_i\right] \geq \max_i \mathbb{E}[X_i]$, with strict inequality as soon as, with positive probability, the maximum is attained by different indices.

Proof. For every j, $\max_i X_i \geq X_j$, so $\mathbb{E}[\max_i X_i] \geq \mathbb{E}[X_j]$; take the maximum over j. (The function $x \mapsto \max_i x_i$ is convex, and the inequality is an instance of Jensen's inequality.) □

In words. This is the "winner's curse": among many noisy estimates, the largest one is likely to be one that got lucky. Choosing an action because its estimate is the highest and then believing that estimate is over-optimistic. Double Q-learning separates the two roles: one estimate chooses, the other, with independent errors, evaluates, so the luck that made the choice does not inflate the value.

If the estimates $Q(s', a')$ are unbiased but noisy, $\max_{a'} Q(s', a')$ overestimates $\max_{a'} q(s', a')$, and Q-learning propagates the overestimation through bootstrapping. *Double Q-learning* (van Hasselt, 2010) keeps two estimates: one chooses the best action, the other evaluates it, which removes the bias when their errors are independent. Its deep version, double DQN, is one of the improvements of the DQN agent of the repository (Chapter 7).

5.7 Summary

- Monte Carlo methods average complete returns: unbiased, high variance, need complete episodes.
- TD methods bootstrap from the current estimate: biased, low variance, learn after every step; their expected update is a contraction towards the Bellman fixed point.
- n-step and λ-returns interpolate between the two.
- SARSA (on-policy) learns the value of the policy it follows; Q-learning (off-policy) learns $q_\star$ from any exploratory data, but overestimates through the maximisation bias.
- On a reduced Pacman game with 5929 states, both learn an optimal behaviour in a few thousand games. The full game, with its 10^{29} states, needs generalisation: the subject of the next chapter.

Chapter 6

Value function approximation

When the states are too many for a table, the value function is represented by a parametrised function $\hat{v}(s;\mathbf{w})$ with far fewer parameters than states. This chapter studies the simplest and best understood case, *linear* approximation, where the theory is complete: it shows what TD learning converges to, how far this is from the best possible approximation, and why off-policy learning can diverge. Deep networks (Chapter 7) have no such guarantees, but the phenomena identified here explain much of their behaviour.

6.1 The prediction objective

We approximate the value v_π of a fixed policy. Errors in different states cannot all be made small, so we weight them by a distribution d over states, usually the frequency with which the policy visits them.

Definition 6.1. (Linear approximation and weighted norm)

A *feature map* $\mathbf{x}:\mathcal{S}\to\mathbb{R}^m$ assigns to each state a vector of m features; the approximation is $\hat{v}(s;\mathbf{w})=\mathbf{w}^\top\mathbf{x}(s)$. Let $X\in\mathbb{R}^{|\mathcal{S}|\times m}$ be the matrix whose rows are the $\mathbf{x}(s)^\top$ (assumed of full column rank) and $D=\operatorname{diag}(d)$. The weighted norm and the *value error* are

$$\|u\|_D=\sqrt{\sum_s d(s)\,u(s)^2},\qquad \overline{\mathrm{VE}}(\mathbf{w})=\|X\mathbf{w}-v_\pi\|_D^2\,. \tag{6.1}$$

The *projection* Π onto the span of the features is $\Pi u = X\mathbf{w}_u$ with $\mathbf{w}_u = \arg\min_{\mathbf{w}}\|X\mathbf{w}-u\|_D$, i.e. $\Pi = X(X^\top DX)^{-1}X^\top D$.

In words. With far fewer parameters than states, the approximation cannot be right everywhere, so we must say where errors matter: the weights $d(s)$ make the frequently visited states count most. The features are all that the approximation "sees" of a state: two states with the same features get the same value. Πu is the best that can be done for u with these features, the closest point to u in the space they span.

A table is the special case of one-hot features ($m=|\mathcal{S}|$, $X=I$). Classical feature constructions include polynomials, Fourier bases, radial basis functions and *tile coding* (several overlapping grids, each state activating one tile per grid); a neural network can be seen as a feature map that is learnt together with the last linear layer.

6.2 Gradient Monte Carlo and semi-gradient TD

Stochastic gradient descent on $\overline{\text{VE}}$ would need $v_\pi(S_t)$. Monte Carlo replaces it by the return G_t, an unbiased sample:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \left(G_t - \mathbf{w}^\top \mathbf{x}(S_t)\right) \mathbf{x}(S_t), \tag{6.2}$$

a true stochastic gradient step, which converges (under the Robbins–Monro conditions) to the minimiser of $\overline{\text{VE}}$ when the states are sampled from d. Semi-gradient TD(0) uses the TD target instead, treating it as a constant:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha\, \delta_t\, \mathbf{x}(S_t), \qquad \delta_t = R_{t+1} + \gamma\, \mathbf{w}^\top \mathbf{x}(S_{t+1}) - \mathbf{w}^\top \mathbf{x}(S_t). \tag{6.3}$$

In words. The update moves the prediction for S_t towards the target, through the features of S_t. It is only a "semi"-gradient because the target also depends on $\mathbf{w}$ (through S_{t+1}), and that dependence is ignored: we move the prediction towards the target, never the target towards the prediction. A full gradient would also change the value of S_{t+1} to make it agree with S_t, letting the future be distorted to fit the past, which is not what we want.

Proposition 6.2. (Expected TD update)

Let P_π and r_π be as in Proposition 3.14. If $S_t \sim d$ and the transition follows π, then

$$\mathbb{E}\left[\delta_t\, \mathbf{x}(S_t)\right] = \mathbf{b} - A\mathbf{w}, \qquad A = X^\top D(I - \gamma P_\pi)X, \qquad \mathbf{b} = X^\top D r_\pi. \tag{6.4}$$

Proof. Conditioning on $S_t = s$,

$$\mathbb{E}[\delta_t \mathbf{x}(S_t)] = \sum_s d(s)\, \mathbf{x}(s) \left(r_\pi(s) + \gamma \sum_{s'} P_\pi(s, s')\, \mathbf{x}(s')^\top \mathbf{w} - \mathbf{x}(s)^\top \mathbf{w} \right),$$

which is the stated matrix expression. □

If TD converges, it is to a point where the expected update vanishes: the *TD fixed point* $\mathbf{w}_{\text{TD}} = A^{-1}\mathbf{b}$. It is generally *not* the minimiser of $\overline{\text{VE}}$.

6.3 The TD fixed point

Lemma 6.3. (P_π is non-expansive in $\|\cdot\|_D$)

If $dP_\pi \le d$ componentwise (true if d is the stationary distribution of P_π, or the expected number of visits of each state in an episodic task), then $\|P_\pi u\|_D \le \|u\|_D$ for every u.

Proof. Each row of P_π is a (sub-)probability vector, so by Jensen's inequality (convexity of $x \mapsto x^2$, and the missing mass counting as a value 0) $\left(\sum_{s'} P_\pi(s, s') u(s')\right)^2 \le \sum_{s'} P_\pi(s, s')\, u(s')^2$. Hence

$$\|P_\pi u\|_D^2 \le \sum_s d(s) \sum_{s'} P_\pi(s, s')\, u(s')^2 = \sum_{s'} (dP_\pi)(s')\, u(s')^2 \le \sum_{s'} d(s')\, u(s')^2. \qquad \square$$

Theorem 6.4. (Convergence of on-policy linear TD)

Under the assumption of Lemma 6.3, the matrix A is positive definite: $\mathbf{w}^\top A \mathbf{w} \ge (1 - \gamma)\, \|X\mathbf{w}\|_D^2 > 0$ for $\mathbf{w} \neq 0$. Consequently the expected update $\mathbf{w}_{k+1} = \mathbf{w}_k + \alpha(\mathbf{b} - A\mathbf{w}_k)$ converges to $\mathbf{w}_{\text{TD}}$ for every small enough $\alpha > 0$.

Proof. Let $\mathbf{y} = X\mathbf{w}$. By Cauchy–Schwarz in the inner product $\langle u, v\rangle_D = u^\top Dv$ and Lemma 6.3,

$$\mathbf{w}^\top A\mathbf{w} = \|\mathbf{y}\|_D^2 - \gamma\langle \mathbf{y}, P_\pi \mathbf{y}\rangle_D \geq \|\mathbf{y}\|_D^2 - \gamma \|\mathbf{y}\|_D \|P_\pi \mathbf{y}\|_D \geq (1-\gamma)\|\mathbf{y}\|_D^2 ,$$

and $\mathbf{y} \neq 0$ when $\mathbf{w} \neq 0$ (X has full column rank and we assume $d > 0$). Let $\lambda > 0$ be the smallest eigenvalue of the symmetric part $(A + A^\top)/2$, so that $\mathbf{e}^\top A\mathbf{e} \geq \lambda \|\mathbf{e}\|^2$. The error $\mathbf{e}_k = \mathbf{w}_k - \mathbf{w}_{\mathrm{TD}}$ satisfies $\mathbf{e}_{k+1} = (I - \alpha A)\mathbf{e}_k$, so

$$\|\mathbf{e}_{k+1}\|^2 = \|\mathbf{e}_k\|^2 - 2\alpha \mathbf{e}_k^\top A\mathbf{e}_k + \alpha^2 \|A\mathbf{e}_k\|^2 \leq \left(1 - 2\alpha\lambda + \alpha^2 \|A\|^2\right) \|\mathbf{e}_k\|^2 ,$$

and the factor is less than 1 for $0 < \alpha < 2\lambda/\|A\|^2$. □

In words. Positive definiteness of A means that, on average, the update always points towards the TD fixed point, like a ball rolling into a bowl. It depends on the states being weighted as the policy visits them: then a state whose value is overestimated is also visited as a *next* state often enough for the overestimate to be corrected.

The stochastic algorithm (6.3), with decreasing step sizes, converges to $\mathbf{w}_{\mathrm{TD}}$ with probability 1 (Tsitsiklis and Van Roy, 1997). How good is the fixed point?

Theorem 6.5. (Projected Bellman equation and error bound)

Under the same assumption, $v_{\mathrm{TD}} = X\mathbf{w}_{\mathrm{TD}}$ is the unique solution of the *projected Bellman equation* $v = \Pi\mathcal{T}^\pi v$, and

$$\|v_{\mathrm{TD}} - v_\pi\|_D \leq \frac{1}{\sqrt{1-\gamma^2}} \|\Pi v_\pi - v_\pi\|_D . \tag{6.5}$$

Proof. $\mathbf{b} - A\mathbf{w} = X^\top D(r_\pi + \gamma P_\pi X\mathbf{w} - X\mathbf{w}) = X^\top D(\mathcal{T}^\pi v - v)$ with $v = X\mathbf{w}$. It vanishes iff $\mathcal{T}^\pi v - v$ is D-orthogonal to the span of X, i.e. iff $\Pi(\mathcal{T}^\pi v - v) = 0$, i.e. $v = \Pi\mathcal{T}^\pi v$ since $\Pi v = v$. The map $\Pi\mathcal{T}^\pi$ is a γ-contraction in $\|\cdot\|_D$ (Π is an orthogonal projection, hence non-expansive, and $\|\mathcal{T}^\pi u - \mathcal{T}^\pi u'\|_D = \gamma \|P_\pi(u - u')\|_D \leq \gamma \|u - u'\|_D$), which gives uniqueness. For the bound, $v_\pi - \Pi v_\pi$ is orthogonal to the span, which contains $\Pi v_\pi - v_{\mathrm{TD}}$; by Pythagoras and the contraction, using $v_\pi = \mathcal{T}^\pi v_\pi$,

$$\begin{aligned} \|v_\pi - v_{\mathrm{TD}}\|_D^2 &= \|v_\pi - \Pi v_\pi\|_D^2 + \|\Pi\mathcal{T}^\pi v_\pi - \Pi\mathcal{T}^\pi v_{\mathrm{TD}}\|_D^2 \\ &\leq \|v_\pi - \Pi v_\pi\|_D^2 + \gamma^2 \|v_\pi - v_{\mathrm{TD}}\|_D^2 , \end{aligned}$$

and we solve for $\|v_\pi - v_{\mathrm{TD}}\|_D^2$. □

In words. TD never sees v_π: it only sees its own predictions one step ahead. Looking one step ahead ($\mathcal{T}^\pi$) produces a function that the features may not represent, so it is projected back (Π); the TD solution is the function that this look-then-project operation leaves unchanged. It is generally not the best fit Πv_π, because the approximation errors of the next states feed into the targets. The bound says that this compounding costs at most a factor $1/\sqrt{1-\gamma^2}$ (7 for $\gamma = 0.99$), and often much less.

Pacman 6.6. (Linear features for the reduced game)

We approximate the value $v_\star$ of the optimal policy of the reduced game of Example 4.8 (5776 non-terminal states) with 8 features of the board: a constant, the maze distances from Pacman to the power cookie, from Pacman to the ghost and from the ghost to the cookie, the square of the first, the distance to the ghost capped at 6, and two indicators

(ghost within 2 cells; ghost closer to the cookie than Pacman). The weighting d is the expected number of visits from a uniform start, which satisfies $dP_\pi \le d$. The script `Course/code/vfa_escape.py` computes exactly:

Quantity	Value
$\|v_\star\|_D$	8.261
best approximation $\|\Pi v_\star - v_\star\|_D$	0.245
TD fixed point $\|v_{\mathrm{TD}} - v_\star\|_D$	0.262
bound (6.5)	1.734

Eight numbers summarise 5776 values to within 3%, and the TD fixed point is almost as good as the best approximation: the bound, which must hold for every problem, is pessimistic here. The smallest eigenvalue of the symmetric part of A is 0.005, positive as Theorem 6.4 predicts.

6.4 Least-squares TD

Since $\mathbf{w}_{\mathrm{TD}} = A^{-1}\mathbf{b}$, one can estimate A and $\mathbf{b}$ from data and solve the linear system directly (LSTD, Bradtke and Barto, 1996):

$$\hat{A} = \sum_t \mathbf{x}(S_t)\left(\mathbf{x}(S_t) - \gamma\,\mathbf{x}(S_{t+1})\right)^\top + \varepsilon I, \qquad \hat{\mathbf{b}} = \sum_t R_{t+1}\,\mathbf{x}(S_t), \qquad \mathbf{w} = \hat{A}^{-1}\hat{\mathbf{b}}. \tag{6.6}$$

LSTD has no step size and uses the data optimally, at a cost of $O(m^2)$ per step (with incremental inverses) instead of $O(m)$; it is attractive for moderate numbers of features.

6.5 Off-policy divergence

The proof of Theorem 6.4 used $dP_\pi \le d$: the states must be weighted as the policy being evaluated visits them. With another weighting (off-policy learning, or a replay memory filled by other policies), A can fail to be positive definite and TD can diverge. Proposition 7.1 gives a two-state example; the classical one is *Baird's counterexample* (1995): seven states, linear features with more parameters than needed, uniform off-policy updates, and the weights diverge for every positive step size, although an exact representation exists. This is the *deadly triad* of function approximation, bootstrapping and off-policy learning.

> **In words.** In the off-policy case, some states are updated often while the states their targets depend on are rarely updated. Through the shared weights, raising the value of the first states also raises the value of the second, which raises the targets of the first: the updates chase a target that they push away. On-policy, every state that serves as a target is also updated in proportion, and this feedback is kept in check.

Two families of remedies exist. *Gradient TD* methods (GTD2, TDC, Sutton et al., 2009) perform true stochastic gradient descent on the *mean squared projected Bellman error*

$$\overline{\mathrm{PBE}}(\mathbf{w}) = \|\Pi\mathcal{T}^\pi X\mathbf{w} - X\mathbf{w}\|_D^2 = (\mathbf{b} - A\mathbf{w})^\top (X^\top D X)^{-1}(\mathbf{b} - A\mathbf{w}), \tag{6.7}$$

estimating the matrix product with a second set of weights; they converge off-policy. *Emphatic* TD reweights the updates so that the effective weighting satisfies the condition of Lemma 6.3. Deep RL mostly relies instead on the stabilisers of DQN (target networks, replay memories close to on-policy), which weaken the triad without removing it.

6.6 Control with function approximation

For control, $\hat{q}(s, a; \mathbf{w})$ is learnt with the semi-gradient versions of SARSA or Q-learning. Semi-gradient SARSA with linear features can oscillate but stays bounded (Gordon, 2001); Q-learning with linear features can diverge, being off-policy. In practice, the combination of Q-learning with deep networks requires the techniques of the next chapter.

6.7 Summary

- Linear approximation represents values as $\mathbf{w}^\top \mathbf{x}(s)$; the error is measured in the norm weighted by the state distribution.
- On-policy, semi-gradient TD converges to the TD fixed point, the solution of the projected Bellman equation, within a factor $1/\sqrt{1-\gamma^2}$ of the best approximation; LSTD computes it directly.
- Off-policy, TD can diverge (deadly triad); gradient TD methods converge, and deep RL relies on target networks and replay memories.

Chapter 7

Deep Q-networks

Tabular methods store one value per state; Pacman has about 10^{30} of them (Example 4.9). The way out is to represent q by a parametrised function Q_θ, here a convolutional network reading the image of the board, and to adjust the parameters θ so that Q_θ approximately satisfies the Bellman optimality equation. States that look alike then share what is learnt about them. This chapter shows why the naive combination of Q-learning and neural networks is unstable, how the deep Q-network (DQN) of Mnih et al. (2015) stabilises it, and the improvements used by the DQN agent of the repository.

7.1 Q-learning with function approximation

With a parametrised Q_θ, the update (5.9) becomes a gradient step on the squared TD error, the target being treated as a constant:

$$\theta \leftarrow \theta + \alpha \left[R_{t+1} + \gamma \max_{a'} Q_\theta(S_{t+1}, a') - Q_\theta(S_t, A_t) \right] \nabla_\theta Q_\theta(S_t, A_t). \tag{7.1}$$

This is not the gradient of any fixed objective (the target also depends on θ, and its dependence is ignored): it is called a *semi-gradient* method. With a table, $\nabla_\theta Q_\theta(s,a)$ is the indicator of the entry (s,a), and (7.1) is exactly tabular Q-learning. With a network, changing θ to fit one state changes the values of all the others, including the target.

7.1.1 The deadly triad

Sutton and Barto call the combination of *function approximation*, *bootstrapping* and *off-policy* learning the *deadly triad*: each is harmless alone, together they can make the parameters diverge. A two-state example shows the mechanism.

Proposition 7.1. (Divergence of semi-gradient TD)

Two states s_1, s_2 have features $x(s_1) = 1$ and $x(s_2) = 2$, and values $V_w(s) = w\,x(s)$ with a single parameter w. The transition $s_1 \to s_2$ gives reward 0. If the semi-gradient TD(0) update is applied repeatedly to this transition only (an off-policy distribution of updates), then

$$w_{k+1} = (1 + \alpha(2\gamma - 1))\, w_k,$$

and $|w_k| \to \infty$ for every $w_0 \neq 0$ as soon as $\gamma > 1/2$ and $\alpha > 0$.

Proof. The update is $w \leftarrow w + \alpha[0 + \gamma\, V_w(s_2) - V_w(s_1)]\, \nabla_w V_w(s_1) = w + \alpha(2\gamma w - w) \cdot 1$. The factor $1 + \alpha(2\gamma - 1)$ exceeds 1 when $\gamma > 1/2$. □

Each update tries to raise $V(s_1)$ towards $\gamma V(s_2)$; since s_2 shares the parameter and has a larger feature, its value rises even more, and so does the target. The transition from s_2, which would correct this, is never used. DQN does not remove the deadly triad, but its two key ingredients make such feedback loops much weaker.

7.2 DQN

Definition 7.2. (Experience replay and target network)

- The *replay memory* stores the last transitions (s, a, r, s', d) played ($d = 1$ if s' is terminal). Each update uses a mini-batch sampled uniformly from it.
- The *target network* Q_{θ^-} is a copy of Q_θ used to compute the targets; it is updated slowly (every C steps, or by Polyak averaging, see below).

The update minimises over the mini-batch the loss

$$\mathcal{L}(\theta) = \frac{1}{B}\sum_{i=1}^{B} \ell\left(y_i - Q_\theta(s_i, a_i)\right), \qquad y_i = r_i + \gamma(1 - d_i)\max_{a'} Q_{\theta^-}(s_i', a'), \tag{7.2}$$

with the targets y_i held constant, and ℓ the squared error or the Huber loss.

In words. The target y_i is the network's own, slightly better informed, opinion: one real reward plus the best value that the (frozen) target network sees at the next state, or nothing more if the game ended ($d_i = 1$). DQN thus turns RL into a sequence of supervised regression problems in which the network learns to predict a one-step-improved version of itself. Without the frozen copy it would be chasing its own tail; the target network holds the tail still for a while.

Why a replay memory. Consecutive transitions of a game are strongly correlated: a mini-batch made of them describes one situation and pushes the network towards it, then the next situation pushes it elsewhere. Sampling from a large memory gives nearly independent samples covering many situations, closer to the assumptions of stochastic gradient descent. It also lets each transition be used several times, which matters when playing is expensive. This is only possible because Q-learning is off-policy: the transitions were generated by older policies, but the target is the Bellman optimality target, valid whatever the policy that produced the data.

Why a target network. With the online network in the target, each gradient step moves the target (Proposition 7.1). With a frozen copy, each phase between two copies is an ordinary regression problem towards fixed targets, approximately one application of $\mathcal{T}^\star$: DQN behaves like an approximate value iteration.

The Huber loss. The Huber loss $\ell(u) = u^2/2$ for $|u| \le 1$ and $|u| - 1/2$ otherwise has derivative $\ell'(u) = \max(-1, \min(1, u))$: errors larger than 1 contribute a bounded gradient. Large TD errors, frequent at the beginning of training or after a rare event (a death worth -35), then do not destabilise the network.

DQN (with the improvements of Section 7.3)

1: initialise θ; $\theta^- \leftarrow \theta$; empty replay memory
2: **for** each step t **do**
3: play a_t, ε-greedy with respect to $Q_\theta(s_t, \cdot)$; observe r_{t+1}, s_{t+1}

store the transition (as an n-step transition) in the memory
sample a mini-batch; compute the targets with Q_{θ^-} (double DQN target (7.3))
one gradient step on (7.2) (Huber loss, gradient clipping)
$\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$
decrease ε (linear schedule)

7.3 Improvements

7.3.1 Double DQN

The target (7.2) takes a maximum of noisy estimates, which overestimates (Proposition 5.14). Double DQN (van Hasselt et al., 2016) uses the online network to *choose* the next action and the target network to *evaluate* it:

$$y = r + \gamma(1-d)\, Q_{\theta^-}\left(s', \arg\max_{a'} Q_\theta(s', a')\right). \tag{7.3}$$

When the errors of the two networks are not too correlated, a noisy overestimate by the online network is not confirmed by the target network.

7.3.2 Dueling architecture

In many states the choice of action barely matters (in an empty corridor far from the ghosts, going on or back changes little), while the value of the state matters a lot. The *dueling* network (Wang et al., 2016) computes a state value $V(s)$ and advantages $A(s,a)$ with two heads on a common trunk, and combines them as

$$Q(s,a) = V(s) + A(s,a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a'). \tag{7.4}$$

Proposition 7.3. (Identifiability of the dueling decomposition)

For every function Q, the decomposition (7.4) with the constraint $\sum_a A(s,a) = 0$ exists and is unique: $V(s) = \frac{1}{|\mathcal{A}|}\sum_a Q(s,a)$ and $A(s,a) = Q(s,a) - V(s)$. Without the mean subtraction, $(V + c, A - c)$ gives the same Q for every constant c.

Proof. Averaging (7.4) over a gives $\frac{1}{|\mathcal{A}|}\sum_a Q(s,a) = V(s)$: the advantage terms cancel. Then $A(s,a) - \bar{A}(s) = Q(s,a) - V(s)$, where $\bar{A}(s)$ is the mean of the advantages; with the constraint $\bar{A}(s) = 0$ this determines A. The last claim is immediate. □

In words. $Q(s,a)$ = "how good is the situation" + "how much better is this action than the average action here". The first term is shared by all the actions and learnt from every visit of the state; the second only needs to capture differences between actions, often small. Subtracting the mean fixes how Q is split between the two heads, which could otherwise drift in opposite directions.

Every transition updates $V(s)$, whatever the action played, so the value of a state is learnt from all the actions taken in it.

7.3.3 n-step targets

The repository's DQN uses n-step targets (Proposition 5.6): for a transition from s_t,

$$y = \sum_{k=0}^{m-1} \gamma^k r_{t+k+1} + \gamma^m (1-d)\, Q_{\theta^-}\left(s_{t+m}, \arg\max_{a'} Q_\theta(s_{t+m}, a')\right), \tag{7.5}$$

with $m = n = 3$, or $m < n$ if the game ended (or was aborted) before. The replay memory assembles these transitions on the fly. The n-step target is not exactly off-policy any more (the intermediate actions were chosen by an older, exploratory policy); with small n this bias is outweighed by the faster propagation of rewards.

The replay memory keeps, for each game, the last n steps not yet stored. When it has n of them, or when the game ends (or is aborted), it stores the oldest one as an n-step transition: discounted sum of the rewards, state reached, `done` flag, and the discount γ^m to apply to the bootstrap.

Listing 7.1: Assembling n-step transitions (`memories.py`, `NStepReplayBuffer`)

```
def _emit_first(self, env_idx, s_next, done):
    q = self.pending[env_idx]
    ret = 0.
    for k, (_, _, r) in enumerate(q):      # sum_k gamma^k r_{t+k+1}
        ret += (self.gamma ** k) * r
    s, a, _ = q.popleft()
    self._store(s, a, ret, s_next, done, self.gamma ** (len(q) + 1))

def push(self, env_idx, state, action, reward, next_state, done,
         truncated=False):
    q = self.pending[env_idx]
    q.append((state, action, reward))
    if done or truncated:      # flush: shorter horizons; done only if
        while q:               # the game really ended (not truncated)
            self._emit_first(env_idx, next_state, float(done))
    elif len(q) == self.n_step:
        self._emit_first(env_idx, next_state, 0.)
```

7.3.4 Soft target updates and other details

Instead of copying θ into θ^- every C steps, the repository uses *Polyak averaging* $\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$ after every update, with $\tau = 0.005$: θ^- is an exponential average of the recent parameters, with a time constant of $1/\tau = 200$ updates. The rewards are multiplied by 0.1 (so that the values are of order 1 to 10), the gradients are clipped, and the exploration rate ε decreases linearly from 1 to 0.02 over the first quarter of the training.

7.4 Rainbow and beyond

Rainbow (Hessel et al., 2018) combines six extensions of DQN: the three above (double, dueling, n-step) and the three of this section. On the Atari benchmark, each contributes; prioritised replay, n-step targets and the distributional loss contribute most.

7.4.1 Prioritised experience replay

Uniform sampling spends most updates on transitions that are already well predicted. Prioritised replay (Schaul et al., 2016) samples transition i with probability

$$P(i) = \frac{p_i^\alpha}{\sum_j p_j^\alpha}, \qquad p_i = |\delta_i| + \epsilon, \tag{7.6}$$

where δ_i is its last TD error and $\alpha \in [0,1]$ interpolates between uniform ($\alpha = 0$) and fully prioritised sampling. Sampling from P instead of the uniform distribution changes the expected gradient; the bias is corrected by weighting the loss of transition i by

$$w_i = \left(\frac{1}{N\,P(i)}\right)^\beta, \tag{7.7}$$

normalised by $\max_j w_j$. With $\beta = 1$ the weighted gradient is unbiased: $\mathbb{E}_{i\sim P}[w_i g_i] = \sum_i P(i)\frac{1}{NP(i)} g_i = \frac{1}{N}\sum_i g_i$. In practice β grows from about 0.4 to 1 during training. In Pacman, deaths are rare and their TD errors large: prioritisation would replay them far more often than uniform sampling.

> **In words.** Prioritised replay is studying the exam questions one got wrong more often than those one already knows. Done naively, this distorts the answer: the rare, surprising transitions would weigh as much in the learnt values as if they were frequent. The weights w_i undo the distortion: a transition replayed k times more often counts k times less each time, so the transitions are revisited more often without being over-represented in the result.

7.4.2 Distributional reinforcement learning

Instead of the expected return $q(s,a)$, distributional RL learns the whole distribution of the return $Z(s,a)$, which satisfies the *distributional Bellman equation*

$$Z(s,a) \overset{D}{=} R + \gamma\, Z(S', A'), \tag{7.8}$$

an equality of distributions, with (R, S') drawn from $p(\cdot \mid s,a)$ and $A' \sim \pi(\cdot \mid S')$. For policy evaluation, the corresponding operator is a γ-contraction in the maximal Wasserstein distance (Bellemare, Dabney and Munos, 2017). Two parametrisations are common.

- **C51** represents $Z(s,a)$ as a categorical distribution on 51 fixed atoms $z_1 < \cdots < z_{51}$; the target distribution of $r + \gamma Z(s', a^\star)$, whose atoms $r + \gamma z_j$ fall between the fixed atoms, is projected back onto them (each mass is split between the two neighbouring atoms in proportion to the distances), and the network minimises the cross-entropy to it.
- **QR-DQN** (Dabney et al., 2018) represents $Z(s,a)$ by N values $\theta_1, \dots, \theta_N$, meant to be its quantiles at levels $\tau_i = (2i-1)/(2N)$, learnt with the quantile regression loss below.

Proposition 7.4. (Quantile regression)

Let Z be a real random variable with a continuous distribution function F, and $\rho_\tau(u) = u\,(\tau - \mathbb{1}[u < 0])$ for $\tau \in (0,1)$. Then $\theta \mapsto \mathbb{E}[\rho_\tau(Z - \theta)]$ is convex and minimised at the τ-quantile $F^{-1}(\tau)$.

Proof. ρ_τ is convex (it is piecewise linear with slopes $\tau - 1 < 0 < \tau$), hence so is the expectation. Its derivative with respect to θ is $\mathbb{E}[-\tau\,\mathbb{1}[Z > \theta] + (1-\tau)\,\mathbb{1}[Z < \theta]] = -\tau(1 - F(\theta)) + (1-\tau)F(\theta) = F(\theta) - \tau$, which vanishes at $F(\theta) = \tau$. □

In words. The loss ρ_τ charges τ per unit when the sample is above θ and $1-\tau$ when it is below. Moving θ up gains τ on the samples above and loses $1-\tau$ on those below; the balance is reached when a fraction τ of the samples lies below θ, which defines the τ-quantile. With $\tau = 1/2$ the loss is the absolute error, minimised by the median.

The action values are recovered as means of the distributions, and the agent still acts greedily with respect to them. Why does learning the distribution help, if only its mean is used? The extra targets act as auxiliary tasks that shape the network's representation, and they keep apart situations with the same mean but different risks; in Pacman, a move with a small chance of death and a safe move of equal mean value have very different return distributions.

7.4.3 Noisy networks

ε-greedy exploration is the same in every state. NoisyNets (Fortunato et al., 2018) replace the last linear layers by layers with learnt noise, $y = (\mu^w + \sigma^w \odot \varepsilon^w)x + \mu^b + \sigma^b \odot \varepsilon^b$, the noise ε being resampled regularly; the network learns how much to explore through σ, and can explore differently in different states.

7.4.4 Beyond Rainbow

Recurrent networks (R2D2, Kapturowski et al., 2019) handle partial observability, as in game `v2` (Remark 3.3), by giving the agent a memory; distributed actors (Ape-X) collect experience at a much larger scale; Agent57 (2020) adds learnt exploration bonuses (Chapter 13) and surpassed the human benchmark on all 57 Atari games. MuZero (Chapter 11) replaces the replay of real transitions by planning with a learnt model.

7.5 The DQN agent of the repository

Pacman 7.5. (Network and training)

- **Input:** the feature planes of the last two images ($14 \times 9 \times 13$, Section 1.6).
- **Network** (`DuelingCNN`): three 3×3 convolutions with 32, 64 and 32 channels, stride 1 and padding 1, a fully connected layer of 256 units, and the two dueling heads; about 10^6 parameters. The convolutions keep the full 9×13 resolution: on a board this small, a convolution of stride 2 would blur the exact cells of Pacman and the ghosts, which decide everything.
- **Training:** 16 games are played in parallel; at each step the network chooses the actions of the 16 games in a single batched forward pass, and performs one gradient step on a mini-batch of 128 transitions from a memory of 300 000. Double DQN, 3-step targets, Huber loss, $\tau = 0.005$, learning rate decreasing from 2.5×10^{-4} to 5×10^{-5}.

Listing 7.2: One DQN update (`algorithms.py`, `DQN.learn`, excerpt)

```
states, actions, returns, next_states, dones, discounts = \
    self.memory.sample(self.batch_size)
q = self.policy_net(states).gather(1, actions.unsqueeze(1)).squeeze(1)

with torch.no_grad():                       # targets held constant
    next_q_target = self.target_net(next_states)
    # double DQN: the online net chooses, the target net evaluates
    next_actions = self.policy_net(next_states).argmax(dim=1,
                                                       keepdim=True)
```

```
    next_q = next_q_target.gather(1, next_actions).squeeze(1)
    targets = (returns * self.reward_scale
               + discounts * (1. - dones) * next_q)  # n-step target

loss = F.smooth_l1_loss(q, targets)        # Huber loss
self.optimizer.zero_grad(set_to_none=True)
loss.backward()
nn.utils.clip_grad_norm_(self.policy_net.parameters(),
                         self.max_grad_norm)
self.optimizer.step()

with torch.no_grad():                      # Polyak averaging
    for p_t, p in zip(self.target_net.parameters(),
                      self.policy_net.parameters()):
        p_t.lerp_(p, self.tau)             # p_t <- p_t + tau (p - p_t)
```

Remark 7.6. (The replay ratio)

The *replay ratio* is the average number of times each transition is used in a mini-batch. With the settings above, each step of the 16 games adds 16 transitions and uses 128: each transition is replayed about 8 times. The first version of the repository played complete games in 32 environments (up to about 32 000 transitions) and then made a *single* gradient step on 32 transitions: a replay ratio of about 0.001. Its network barely learnt anything, whatever its architecture: in deep RL the amount of learning per transition matters as much as the algorithm.

Remark 7.7. (Results)

The results of the DQN agent trained for 40 million steps on Kaggle will be added here, and compared with the A2C/PPO agent of Chapter 9.

7.6 Summary

- With function approximation, Q-learning becomes a semi-gradient method, which can diverge (deadly triad).
- DQN stabilises it with a replay memory (decorrelated, reused samples, possible because Q-learning is off-policy) and a slowly moving target network (each phase is a regression towards fixed targets).
- Double DQN removes most of the maximisation bias; the dueling architecture learns state values from every transition; n-step targets propagate rare rewards faster.
- In practice, the amount of learning per transition (replay ratio) is as important as these refinements.

Chapter 8

Policy gradients and actor-critic methods

Value-based methods learn $q_\star$ and act greedily. Policy-based methods take the other road: they parametrise the policy itself, $\pi_\theta(a \mid s)$, and improve it by gradient ascent on the expected return. Combined with a learnt value function (the *critic*) that reduces the variance of the gradient estimates, this gives the actor-critic methods, A2C and its successor PPO (Chapter 9), which produced the best agent of the repository.

8.1 Parametrised policies

The agents of the repository use a *softmax policy*: a network outputs one score (*logit*) $z_\theta(s, a)$ per action, and

$$\pi_\theta(a \mid s) = \frac{\exp z_\theta(s, a)}{\sum_b \exp z_\theta(s, b)}. \tag{8.1}$$

Compared with an ε-greedy policy, a softmax policy changes smoothly with θ, can represent any stochastic policy, and explores by itself in the states where it is uncertain.

Proposition 8.1. (Score of a softmax policy)

The gradient of $\log \pi_\theta(a \mid s)$ with respect to the logits $z = (z(s, b))_b$ of state s is $e_a - \pi_\theta(\cdot \mid s)$, where e_a is the indicator vector of action a.

Proof. $\log \pi_\theta(a \mid s) = z(s, a) - \log \sum_b \exp z(s, b)$, whose derivative with respect to $z(s, c)$ is $\mathbb{1}[c = a] - \exp z(s, c) / \sum_b \exp z(s, b) = \mathbb{1}[c = a] - \pi_\theta(c \mid s)$. □

Increasing $\log \pi_\theta(a \mid s)$ thus raises the logit of a and lowers the others in proportion to their probabilities.

8.2 The policy gradient theorem

We maximise the expected return from the initial state distribution μ,

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G_0] = \sum_s \mu(s)\, v_{\pi_\theta}(s). \tag{8.2}$$

The difficulty is that θ influences the distribution of the trajectories, through the actions, and the dynamics p that also shape it are unknown. The policy gradient theorem shows that the gradient nevertheless has an expression that can be estimated from trajectories, without p. We

prove it for episodes of bounded length H, which covers Pacman: a game is aborted after at most 1000 moves.

Theorem 8.2. (Policy gradient theorem)

Suppose every episode ends (or is aborted) within H steps, and π_θ is differentiable in θ. Then

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{H-1} \nabla_\theta \log \pi_\theta(A_t \mid S_t)\, \gamma^t\, G_t\right] = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{H-1} \nabla_\theta \log \pi_\theta(A_t \mid S_t)\, \gamma^t\, q_{\pi_\theta}(S_t, A_t)\right], \tag{8.3}$$

where, after the end of an episode, the terms are 0.

Proof. Step 1 (likelihood ratio). A trajectory $\tau = (s_0, a_0, r_1, \ldots, s_H)$ has probability

$$P_\theta(\tau) = \mu(s_0) \prod_{t=0}^{H-1} \pi_\theta(a_t \mid s_t)\, p(s_{t+1}, r_{t+1} \mid s_t, a_t),$$

and $G_0(\tau) = \sum_t \gamma^t r_{t+1}$ does not depend on θ. There are finitely many trajectories, so we may differentiate under the sum: using $\nabla P = P \nabla \log P$ where $P > 0$,

$$\nabla_\theta J = \sum_\tau G_0(\tau)\, \nabla_\theta P_\theta(\tau) = \mathbb{E}_{\pi_\theta}\left[G_0\, \nabla_\theta \log P_\theta(\tau)\right] = \mathbb{E}_{\pi_\theta}\left[G_0 \sum_{t=0}^{H-1} \nabla_\theta \log \pi_\theta(A_t \mid S_t)\right],$$

because μ and p do not depend on θ and disappear from the gradient of the logarithm.

Step 2 (causality). Write $\psi_t = \nabla_\theta \log \pi_\theta(A_t \mid S_t)$. For $k \le t$, the reward R_k is determined by the history up to S_t, and given that history,

$$\mathbb{E}\left[\psi_t \mid S_0, \ldots, S_t\right] = \sum_a \pi_\theta(a \mid S_t)\, \nabla_\theta \log \pi_\theta(a \mid S_t) = \nabla_\theta \sum_a \pi_\theta(a \mid S_t) = \nabla_\theta 1 = 0.$$

Hence $\mathbb{E}[R_k \psi_t] = 0$ for $k \le t$: an action cannot influence the rewards received before it. Removing these terms from $G_0 \psi_t$ leaves $\sum_{k>t} \gamma^{k-1} R_k\, \psi_t = \gamma^t G_t\, \psi_t$, which proves the first equality.

Step 3 (conditioning). Given (S_t, A_t), ψ_t is fixed and, by the Markov property, $\mathbb{E}[G_t \mid S_0, A_0, \ldots, S_t, A_t] = q_{\pi_\theta}(S_t, A_t)$. The tower property gives the second equality. □

In words. The gradient is "try, then reinforce". $\nabla_\theta \log \pi_\theta(A_t \mid S_t)$ is the direction in parameter space that makes the action actually taken more likely; each such direction is weighted by how well things went afterwards. Good outcomes make their actions more likely, bad ones less likely. The logarithm divides by the probability of the action, so that actions are not favoured merely because they are often sampled. Two things are remarkable: the dynamics do not appear (the agent needs to know how an action turned out, not why), and an action is judged only by what happened *after* it (step 2).

Remark 8.3. (The factor γ^t)

The factor γ^t in (8.3) gives little weight to the late steps of an episode. Practical implementations, including the repository's, drop it and weight all the steps equally; they then follow a direction that is not exactly the gradient of J (Thomas, 2014), but which treats every state as a possible starting point, which works well in practice.

The simplest algorithm built on (8.3) is REINFORCE (Williams, 1992): play episodes and take the step $\theta \leftarrow \theta + \alpha \sum_t \psi_t G_t$. It is unbiased but very noisy, since each G_t is a Monte Carlo return.

8.3 Baselines and advantages

Proposition 8.4. (Baselines)

For any function $b : \mathcal{S} \to \mathbb{R}$ (a *baseline*), $\mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(A_t \mid S_t)\, b(S_t)\right] = 0$. Consequently (8.3) remains true with G_t replaced by $G_t - b(S_t)$, or $q_{\pi_\theta}(S_t, A_t)$ by $q_{\pi_\theta}(S_t, A_t) - b(S_t)$.

Proof. Conditionally on S_t, $b(S_t)$ is a constant and $\mathbb{E}[\psi_t \mid S_t] = 0$ (step 2 of the previous proof). □

A baseline does not change the expected gradient but can greatly reduce its variance. The natural choice is $b = v_{\pi_\theta}$, which gives the *advantage*

$$A_\pi(s, a) = q_\pi(s, a) - v_\pi(s), \tag{8.4}$$

the amount by which action a is better than the average action of the policy in s. The gradient step then increases the probability of the actions that did better than expected and decreases the others. The advantage of an action is often much smaller than its value: in Pacman, the value of a state may be 100, while the difference between going east and going west is a few points.

> **In words.** Without a baseline, an action is judged by its absolute outcome: if all the returns are positive, every action taken is reinforced, and only the differences in the amount of reinforcement carry information, hidden in a lot of noise. With the value as baseline, an action is judged *relative to what was expected* in that state: better than usual, reinforce; worse, discourage. The expected gradient is the same, but the signal no longer has to be extracted from the noise of large returns.

8.4 Actor-critic methods

The advantage is unknown; an *actor-critic* method learns a value function $V_w \approx v_{\pi_\theta}$ (the *critic*) alongside the policy (the *actor*), and uses it to estimate the advantages.

Proposition 8.5. (The TD error estimates the advantage)

Let $\delta_t = R_{t+1} + \gamma\, v_\pi(S_{t+1}) - v_\pi(S_t)$, with the true value function. Then $\mathbb{E}_\pi[\delta_t \mid S_t = s, A_t = a] = A_\pi(s, a)$.

Proof. By (3.7), $\mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a] = q_\pi(s, a)$; subtract $v_\pi(s)$. □

> **In words.** The critic grades the actor. Before the move it predicted $v_\pi(S_t)$; after one step it can say $R_{t+1} + \gamma v_\pi(S_{t+1})$. If the move led somewhere better than predicted, the TD error is positive and the move was better than average. One step of experience is enough for a grade, at the cost of relying on the critic's judgement of S_{t+1}.

With the critic V_w instead of v_π, δ_t is a biased but low-variance estimate of the advantage. Monte Carlo advantages $G_t - V_w(S_t)$ are unbiased (given a baseline) but noisy; the generalised advantage estimator interpolates between the two.

8.4.1 Generalised advantage estimation

Definition 8.6. (GAE)

Let $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ be the TD errors of a critic V (with $V = 0$ after a terminal state). For $\lambda \in [0, 1]$, the *generalised advantage estimate* (Schulman et al., 2016) is

$$\hat{A}_t^\lambda = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}, \tag{8.5}$$

where the sum stops at the end of the episode. It satisfies the recursion $\hat{A}_t^\lambda = \delta_t + \gamma\lambda \, \hat{A}_{t+1}^\lambda$, used to compute it backwards along a trajectory.

Proposition 8.7. (GAE interpolates between TD and Monte Carlo)

For an episode ending at time T (so that the sums are finite):
(i) $\hat{A}_t^0 = \delta_t$;
(ii) $\hat{A}_t^1 = G_t - V(S_t)$;
(iii) $\hat{A}_t^\lambda + V(S_t) = G_t^\lambda$, the λ-return (5.6).

Proof. (i) is immediate. (ii) The sum telescopes:

$$\sum_{l=0}^{T-t-1} \gamma^l \left(R_{t+l+1} + \gamma V(S_{t+l+1}) - V(S_{t+l})\right) = \sum_{l=0}^{T-t-1} \gamma^l R_{t+l+1} + \gamma^{T-t} V(S_T) - V(S_t) = G_t - V(S_t),$$

since $V(S_T) = 0$ at the terminal state. (iii) The same telescoping over the first n terms gives $\sum_{l=0}^{n-1} \gamma^l \delta_{t+l} = G_{t:t+n} - V(S_t)$. Hence

$$(1-\lambda) \sum_{n \geq 1} \lambda^{n-1} \left(G_{t:t+n} - V(S_t)\right) = (1-\lambda) \sum_{n \geq 1} \lambda^{n-1} \sum_{l=0}^{n-1} \gamma^l \delta_{t+l} = \sum_{l \geq 0} \gamma^l \delta_{t+l} (1-\lambda) \sum_{n > l} \lambda^{n-1},$$

and $(1-\lambda)\sum_{n>l} \lambda^{n-1} = \lambda^l$. The left-hand side is $G_t^\lambda - V(S_t)$ because the weights $(1-\lambda)\lambda^{n-1}$ sum to 1 (for $\lambda < 1$; the case $\lambda = 1$ is (ii)). All sums are finite, which justifies the exchange. □

> **In words.** The advantage of an action is estimated by the surprises that follow it, the later ones counting less. The factor $\gamma\lambda$ says for how long an action is held responsible for what happens next: with $\lambda = 0$, only for the next surprise (and the critic is trusted for the rest); with $\lambda = 1$, for everything until the end of the game. With $\gamma\lambda = 0.99 \times 0.95 \approx 0.94$, an action is held responsible for about the next 17 moves.

The parameter λ trades bias (small λ, which relies on the critic) against variance (large λ, which relies on the sampled rewards). The repository uses $\lambda = 0.95$, and the critic is trained towards the targets $\hat{A}_t^\lambda + V(S_t)$, i.e. the λ-returns.

Listing 8.1: GAE by the backward recursion (`algorithms.py`, `A2C.compute_gae`)

```
# rewards, values, dones: arrays [n_steps, n_envs]; dones[t] = 1 if the
# game ended after step t; last_values: value of the states reached
advantages = np.zeros_like(rewards)
last_gae = 0.
for t in reversed(range(n_steps)):
    next_values = last_values if t == n_steps - 1 else values[t + 1]
    not_done = 1. - dones[t]
    delta = rewards[t] + self.gamma * next_values * not_done - values[t]
```

```
    last_gae = delta + self.gamma * self.gae_lambda * not_done * last_gae
    advantages[t] = last_gae
return advantages, advantages + values    # value targets: lambda-returns
```

A game that is aborted (not ended) after step t is marked as done, but its reward is first increased by $\gamma V(S_{t+1})$: the bootstrap of Remark 3.7.

8.5 Continuous actions: Gaussian policies

Policy gradients extend directly to continuous actions $a \in \mathbb{R}^k$, where value-based methods would need a maximisation over a continuum. The standard choice is a Gaussian policy whose mean is computed by a network, with a diagonal covariance (state-independent, or also computed by the network):

$$\pi_\theta(a \mid s) = \mathcal{N}\left(a;\, \mu_\theta(s),\, \mathrm{diag}(\sigma_\theta(s)^2)\right). \tag{8.6}$$

Proposition 8.8. (Score of a Gaussian policy)

In one dimension, with mean μ and standard deviation $\sigma = e^\eta$,

$$\frac{\partial \log \pi(a)}{\partial \mu} = \frac{a-\mu}{\sigma^2}, \qquad \frac{\partial \log \pi(a)}{\partial \eta} = \frac{(a-\mu)^2}{\sigma^2} - 1. \tag{8.7}$$

Proof. $\log \pi(a) = -\frac{(a-\mu)^2}{2e^{2\eta}} - \eta - \frac{1}{2}\log(2\pi)$; differentiate with respect to μ and η. □

An action better than expected ($\hat{A} > 0$) moves the mean towards it; the standard deviation grows if the action was far from the mean ($|a-\mu| > \sigma$) and shrinks otherwise. Actions bounded in an interval are usually obtained by squashing a Gaussian sample with tanh, with the corresponding correction of $\log \pi$ (Chapter 10).

8.6 The natural policy gradient

The gradient $\nabla_\theta J$ is the direction of steepest ascent for the Euclidean distance between *parameters*, which has no meaning for the *policies*: rescaling a parameter changes the gradient, and a small change of a parameter can change the policy a lot in some states and not at all in others. The natural gradient measures steps by the change of the policy itself, with the KL divergence.

Definition 8.9. (Fisher information matrix)

For a policy π_θ and a state distribution d,

$$F(\theta) = \mathbb{E}_{s\sim d,\, a\sim\pi_\theta(\cdot|s)}\left[\nabla_\theta \log \pi_\theta(a \mid s)\, \nabla_\theta \log \pi_\theta(a \mid s)^\top\right]. \tag{8.8}$$

Proposition 8.10. (KL divergence to second order)

Let $\bar{K}(\theta') = \mathbb{E}_{s\sim d}[\mathrm{KL}(\pi_\theta(\cdot \mid s) \,\|\, \pi_{\theta'}(\cdot \mid s))]$. For smooth policies with finitely many actions, $\bar{K}(\theta+\Delta) = \frac{1}{2}\Delta^\top F(\theta)\Delta + O(\|\Delta\|^3)$.

Proof. Fix s and write $\pi_\theta(a)$ for $\pi_\theta(a \mid s)$. The function $\theta' \mapsto \mathrm{KL}(\pi_\theta \| \pi_{\theta'}) = \sum_a \pi_\theta(a)(\log \pi_\theta(a) - \log \pi_{\theta'}(a))$ vanishes at $\theta' = \theta$. Its gradient there is $-\sum_a \pi_\theta(a) \nabla \log \pi_\theta(a) = -\nabla \sum_a \pi_\theta(a) = 0$. Its Hessian is $-\sum_a \pi_\theta(a) \nabla^2 \log \pi_\theta(a)$, and differentiating $\sum_a \pi_\theta(a) \nabla \log \pi_\theta(a) = 0$ once more

gives $\sum_a \pi_\theta(a)\left(\nabla^2 \log \pi_\theta(a) + \nabla \log \pi_\theta(a) \nabla \log \pi_\theta(a)^\top\right) = 0$: the Hessian is the Fisher matrix of state s. Average over s and apply Taylor's theorem. □

Proposition 8.11. (Natural gradient direction)

If F is positive definite, the maximiser of the linear approximation $\nabla J^\top \Delta$ under the quadratic constraint $\frac{1}{2}\Delta^\top F \Delta \le \delta$ is

$$\Delta^\star = \sqrt{\frac{2\delta}{g^\top F^{-1} g}}\, F^{-1} g, \qquad g = \nabla_\theta J. \tag{8.9}$$

Proof. With $u = F^{1/2}\Delta$ the problem is to maximise $(F^{-1/2}g)^\top u$ subject to $\|u\|^2 \le 2\delta$. By Cauchy–Schwarz the maximiser is u colinear with $F^{-1/2}g$ and of norm $\sqrt{2\delta}$, i.e. $\Delta = F^{-1/2}u \propto F^{-1}g$, with the stated scaling. □

In words. The Fisher matrix measures how sensitive the policy is to each direction of change of the parameters. The natural gradient takes large steps in the directions that barely change the behaviour and small steps in the sensitive ones: every step changes the *policy* by the same amount, whatever the parametrisation. It is like walking a fixed distance on the ground rather than a fixed number of millimetres on a map whose scale varies from place to place.

The *natural policy gradient* (Kakade, 2001) follows $F^{-1}\nabla J$. It is invariant to reparametrisations of the policy, and the size of its steps is measured in KL divergence, the quantity that controls how much the state distribution can change (Chapter 9). Computing $F^{-1}g$ exactly is impossible for large networks; TRPO approximates it with conjugate gradients, and PPO replaces the constraint by clipping.

8.7 A2C

Advantage actor-critic (A2C, the synchronous version of A3C, Mnih et al., 2016) runs N environments in parallel for n steps, then performs one update on the Nn collected transitions.

A2C with GAE

1: **for** each iteration **do**
2: play n steps in each of the N environments with π_θ; store $s_t, a_t, r_{t+1}, d_{t+1}$ and $V_w(s_t)$
3: compute $\hat{A}_t$ by the backward recursion $\hat{A}_t = \delta_t + \gamma\lambda(1-d_{t+1})\hat{A}_{t+1}$, bootstrapping the last step with V_w
4: normalise the advantages (mean 0, standard deviation 1)
5: one gradient step on the loss

$$\mathcal{L} = -\frac{1}{Nn}\sum_t \hat{A}_t \log \pi_\theta(a_t \mid s_t) + \frac{c_v}{2Nn}\sum_t \left(\hat{A}_t + V_{w,\text{old}}(s_t) - V_w(s_t)\right)^2 - c_e\,\bar{\mathcal{H}}$$

The last term is an *entropy bonus*: $\bar{\mathcal{H}}$ is the average entropy $-\sum_a \pi_\theta(a \mid s_t)\log \pi_\theta(a \mid s_t)$ of the policy over the batch. It keeps the policy from becoming deterministic too early, which would stop exploration. In the repository the actor and the critic share the same network (two heads on a common trunk), with $c_v = 0.5$ and $c_e = 0.03$.

Pacman 8.12. (The entropy coefficient)

With $c_e = 0.01$, short test trainings of the repository's agent saw the entropy of the policy fall below 0.85 (out of a maximum of $\log 5 \approx 1.61$) while the agent still lost almost every game: the policy had become nearly deterministic before finding good behaviours. With $c_e = 0.03$ the entropy stabilised around 1.2 at the same stage and the reward improved faster, which is why the repository uses 0.03.

8.8 The actor-critic network of the repository

Pacman 8.13. (Input, network, symmetry, curriculum)

- **Input:** the feature planes of the last two images ($14 \times 9 \times 13$ in game `v1`).
- **Network** (`ActorCriticCNN`): the same full-resolution convolutional trunk as the DQN agent, a fully connected layer of 256 units, a policy head (five logits) and a value head.
- **Symmetry:** the maze is symmetric with respect to its vertical axis, and the network is made exactly equivariant to this symmetry (Proposition 8.14).
- **Curriculum:** the aggressiveness of the ghosts starts at 30% of its value and increases linearly to its full value over the first 20% of the training, so that the agent first survives long enough to discover the rewards.

Let M be the horizontal mirror of an input (reverse the order of the columns of every plane) and P the permutation of the actions that exchanges `E` and `W` and fixes `N`, `S`, `I`. Both are involutions: $M^2 = \mathrm{id}$ and $P^2 = \mathrm{id}$. A network f giving logits and a value, $f(s) = (z(s), V(s))$, is *equivariant* if $z(Ms) = Pz(s)$ and $V(Ms) = V(s)$: the mirror image of a situation gets the mirrored policy and the same value.

Proposition 8.14. (Symmetrisation)

For any network $f = (z, V)$, the symmetrised network

$$z_{\mathrm{sym}}(s) = \tfrac{1}{2}\left(z(s) + Pz(Ms)\right), \qquad V_{\mathrm{sym}}(s) = \tfrac{1}{2}\left(V(s) + V(Ms)\right) \tag{8.10}$$

is equivariant. If f is already equivariant, $f_{\mathrm{sym}} = f$.

Proof. Using $M^2 = \mathrm{id}$ and $P^2 = \mathrm{id}$,

$$z_{\mathrm{sym}}(Ms) = \tfrac{1}{2}\left(z(Ms) + Pz(s)\right) = \tfrac{1}{2}P\left(Pz(Ms) + z(s)\right) = Pz_{\mathrm{sym}}(s),$$

and $V_{\mathrm{sym}}(Ms) = \frac{1}{2}(V(Ms) + V(s)) = V_{\mathrm{sym}}(s)$. If $z(Ms) = Pz(s)$ then $Pz(Ms) = z(s)$ and $z_{\mathrm{sym}} = z$; similarly for V. □

The symmetrised network is evaluated on each state and its mirror image (in a single batch), and trained as is. The agent can then never learn different behaviours on the two halves of the maze, and every game teaches it about both halves at once: this is equivalent to doubling the data. The same construction works for any group of symmetries of the board, by averaging over the group.

Listing 8.2: The symmetrised network (`neural_net.py`, `ActorCriticCNN.forward`)

```
# action_mirror[i] = index of the mirror of action i (E <-> W)
logits, values = self._forward(
    torch.cat([state, torch.flip(state, dims=[-1])]))  # s and Ms
b = state.shape[0]
```

```
logits = (logits[:b] + logits[b:][:, self.action_mirror]) / 2  # z_sym
values = (values[:b] + values[b:]) / 2                         # V_sym
```

8.9 Summary

- The policy gradient theorem expresses ∇J as an expectation over trajectories that does not involve the dynamics.
- A baseline leaves the gradient unchanged and reduces its variance; with the value function it gives the advantage.
- Actor-critic methods learn the value function to estimate the advantages; GAE interpolates between the TD error and the Monte Carlo advantage with the parameter λ.
- A2C runs several environments in parallel and makes one update per batch of transitions; the entropy bonus preserves exploration.

A2C makes one gradient step per batch and throws the batch away: the next chapter shows how PPO reuses each batch several times without letting the policy change too much.

Chapter 9

Trust regions: TRPO and PPO

A2C makes one gradient step per batch of experience and discards the batch. Making several steps on the same batch would use the data better, but after the first step the batch was generated by an older policy, and a large step can ruin a policy that took millions of games to learn. This chapter derives a surrogate objective that is accurate near the current policy, and the clipping mechanism of proximal policy optimisation (PPO, Schulman et al., 2017) that keeps the updates within the region where the surrogate can be trusted. PPO is the algorithm of the best agent of the repository.

Throughout the chapter, π is the current policy (the one that played the batch) and π' a candidate new policy. We use the discounted objective $J(\pi) = \mathbb{E}_\pi[G_0]$ of Chapter 8, episodes of bounded length and finitely many states and actions.

9.1 How good is a new policy?

Lemma 9.1. (Performance difference lemma)

For any two policies π and π',

$$J(\pi') - J(\pi) = \mathbb{E}_{\pi'}\left[\sum_{t\geq 0} \gamma^t \, A_\pi(S_t, A_t)\right], \tag{9.1}$$

where the trajectory is generated by π' and the advantages are those of π (Kakade and Langford, 2002).

Proof. Given (S_t, A_t), $\mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t, A_t] = q_\pi(S_t, A_t)$ whatever the policy that generated the trajectory, since this involves only the dynamics. Hence, by the tower property,

$$\mathbb{E}_{\pi'}\left[\sum_t \gamma^t A_\pi(S_t, A_t)\right] = \mathbb{E}_{\pi'}\left[\sum_t \gamma^t \left(R_{t+1} + \gamma v_\pi(S_{t+1}) - v_\pi(S_t)\right)\right].$$

The sum is finite (bounded episodes, $v_\pi = 0$ at the end), and the value terms telescope: $\sum_t \gamma^t(\gamma v_\pi(S_{t+1}) - v_\pi(S_t)) = -v_\pi(S_0)$. What remains is $\mathbb{E}_{\pi'}[G_0] - \mathbb{E}[v_\pi(S_0)] = J(\pi') - J(\pi)$, the initial distribution being the same for both policies. □

> **In words.** The gain of a new policy is the sum, along its own games, of how much better each of its moves is than what the old policy would have done on average, as judged by the old policy's own values. It is like evaluating a new player's moves with an old coach's judgement: every move the coach rates above average adds to the score.

The lemma says that π' is better than π if it plays actions of positive advantage (for π) in the states *it* visits. We cannot evaluate the right-hand side without playing π', but we can evaluate the actions of π' on the states visited by π, using importance sampling.

Proposition 9.2. (Importance sampling of actions)

For a state s and any function f of the action, if $\pi(a \mid s) > 0$ whenever $\pi'(a \mid s) > 0$,

$$\mathbb{E}_{a\sim\pi'(\cdot|s)}\left[f(a)\right] = \mathbb{E}_{a\sim\pi(\cdot|s)}\left[\rho(a)\, f(a)\right], \qquad \rho(a) = \frac{\pi'(a \mid s)}{\pi(a \mid s)}. \tag{9.2}$$

Proof. $\sum_a \pi(a \mid s)\frac{\pi'(a|s)}{\pi(a|s)} f(a) = \sum_a \pi'(a \mid s) f(a)$, the sum running over the actions with $\pi(a \mid s) > 0$, which include all those with $\pi'(a \mid s) > 0$. □

Definition 9.3. (Surrogate objective)

The *surrogate objective* of π' around π is

$$L_\pi(\pi') = J(\pi) + \mathbb{E}_\pi\left[\sum_{t\geq 0} \gamma^t\, \rho_t\, A_\pi(S_t, A_t)\right], \qquad \rho_t = \frac{\pi'(A_t \mid S_t)}{\pi(A_t \mid S_t)}, \tag{9.3}$$

in which the states *and* the actions come from π: it can be estimated from a batch played with π.

By Proposition 9.2, $L_\pi(\pi')$ is exactly the right-hand side of (9.1) plus $J(\pi)$, except that the states are visited with the frequencies of π instead of π'. When π' is close to π, the two frequencies are close and so are $L_\pi(\pi')$ and $J(\pi')$.

Proposition 9.4. (First-order agreement)

Let π_θ be a differentiable family of policies and $\pi = \pi_{\theta_0}$. Then $L_\pi(\pi_{\theta_0}) = J(\pi_{\theta_0})$ and $\nabla_\theta L_\pi(\pi_\theta)\big|_{\theta=\theta_0} = \nabla_\theta J(\pi_\theta)\big|_{\theta=\theta_0}$.

Proof. At $\theta = \theta_0$ all ratios equal 1, and $\mathbb{E}_\pi[A_\pi(S_t, A_t) \mid S_t] = \sum_a \pi(a \mid S_t)(q_\pi(S_t, a) - v_\pi(S_t)) = 0$, which gives the first equality. Differentiating under the (finite) expectation, $\nabla_\theta \rho_t|_{\theta_0} = \nabla_\theta \pi_\theta(A_t \mid S_t)/\pi_{\theta_0}(A_t \mid S_t) = \nabla_\theta \log \pi_\theta(A_t \mid S_t)|_{\theta_0}$, so

$$\nabla_\theta L_\pi(\pi_\theta)\big|_{\theta_0} = \mathbb{E}_\pi\left[\sum_t \gamma^t \nabla_\theta \log \pi_\theta(A_t \mid S_t)\, A_\pi(S_t, A_t)\right],$$

which is $\nabla_\theta J$ by the policy gradient theorem with the baseline v_π (Theorem 8.2 and Proposition 8.4). □

A small step that improves L_π therefore improves J; a large one may not. How large may the step be?

9.2 Trust region policy optimisation

Theorem 9.5. (Policy improvement bound (stated without proof))

For any policies π, π' (Schulman et al., 2015, Theorem 1, and Achiam et al., 2017),

$$J(\pi') \geq L_\pi(\pi') - C\,\mathrm{KL}^{\max}(\pi, \pi'), \qquad C = \frac{4\epsilon\gamma}{(1-\gamma)^2}, \tag{9.4}$$

where $\epsilon = \max_{s,a} |A_\pi(s,a)|$ and $\mathrm{KL}^{\max}(\pi, \pi') = \max_s \mathrm{KL}(\pi(\cdot \mid s) \,\|\, \pi'(\cdot \mid s))$.

The proof bounds the change of the state distribution by the distance between the policies (a coupling argument). Its consequence is a *monotonic improvement* scheme.

Corollary 9.6. (Monotonic improvement)

Let $M_\pi(\pi') = L_\pi(\pi') - C\,\mathrm{KL}^{\max}(\pi, \pi')$ and $\pi_{k+1} \in \arg\max_{\pi'} M_{\pi_k}(\pi')$. Then $J(\pi_{k+1}) \geq J(\pi_k)$ for all k.

Proof. By (9.4), $J(\pi_{k+1}) \geq M_{\pi_k}(\pi_{k+1})$. Since π_{k+1} maximises M_{π_k}, $M_{\pi_k}(\pi_{k+1}) \geq M_{\pi_k}(\pi_k) = L_{\pi_k}(\pi_k) - 0 = J(\pi_k)$ by Proposition 9.4. □

In words. The surrogate pretends that the new policy visits the same states as the old one; the KL term is the price of that pretence, larger when the policies differ more. Its constant grows like $1/(1-\gamma)^2$: a small change of behaviour at every step accumulates over the horizon into a different distribution of states, and each state's value is itself of order $1/(1-\gamma)$. The scheme is like climbing a hill in the fog with a lower bound that touches the hill where one stands: climbing the bound can only lift one higher on the hill.

This is a *minorise–maximise* algorithm: M_{π_k} is a lower bound of J that touches it at π_k, and improving the bound improves J. The constant C is so large that the steps would be tiny; TRPO (Schulman et al., 2015) keeps the idea but replaces the penalty by a constraint on the *average* KL divergence, a *trust region*:

$$\max_\theta \hat{L}_{\theta_{\text{old}}}(\theta) \quad \text{subject to} \quad \mathbb{E}_s\left[\mathrm{KL}(\pi_{\theta_{\text{old}}}(\cdot \mid s) \,\|\, \pi_\theta(\cdot \mid s))\right] \leq \delta, \tag{9.5}$$

with $\delta \approx 0.01$. Linearising the objective and using the quadratic approximation of the KL divergence (Proposition 8.10) turns it into the natural gradient step of Proposition 8.11. TRPO computes $F^{-1}g$ by conjugate gradients, using only products Fv (computed by automatic differentiation without forming F), then performs a backtracking line search that checks the KL constraint and the improvement of $\hat{L}$ on the actual policy. TRPO is robust but complex and expensive; PPO obtains a similar effect with a first-order method.

9.3 The clipped objective

Definition 9.7. (PPO's clipped surrogate)

For a clipping parameter $\epsilon > 0$ (the repository uses $\epsilon = 0.2$), the per-sample objective is

$$\ell^{\text{CLIP}}(\rho, \hat{A}) = \min\left(\rho\hat{A},\ \mathrm{clip}(\rho, 1-\epsilon, 1+\epsilon)\,\hat{A}\right), \tag{9.6}$$

where $\rho = \pi_\theta(a \mid s)/\pi_{\theta_{\text{old}}}(a \mid s)$ and $\hat{A}$ is the (normalised GAE) advantage computed with the policy that played the batch. PPO maximises the average of ℓ^{CLIP} over the batch.

Proposition 9.8. (Properties of the clipped objective)

For every $\rho > 0$ and $\hat{A}$:

(i) $\ell^{\text{CLIP}}(\rho, \hat{A}) \le \rho\hat{A}$: the clipped objective is a lower bound of the unclipped surrogate;
(ii) $\ell^{\text{CLIP}}(\rho, \hat{A}) = \rho\hat{A}$ when $\rho \in [1-\epsilon, 1+\epsilon]$;
(iii) if $\hat{A} > 0$ and $\rho > 1+\epsilon$, or $\hat{A} < 0$ and $\rho < 1-\epsilon$, then ℓ^{CLIP} is constant in ρ near ρ: the sample no longer contributes to the gradient;
(iv) at $\theta = \theta_{\text{old}}$, the gradient of the average of ℓ^{CLIP} is the A2C policy gradient $\frac{1}{N}\sum_i \hat{A}_i \nabla_\theta \log \pi_\theta(a_i \mid s_i)$.

Proof. (i) The minimum of two numbers is at most the first. (ii) The clip does nothing. (iii) If $\hat{A} > 0$ and $\rho > 1+\epsilon$, then $\text{clip}(\rho)\hat{A} = (1+\epsilon)\hat{A} < \rho\hat{A}$, so the minimum is the constant $(1+\epsilon)\hat{A}$ on a neighbourhood of ρ. If $\hat{A} < 0$ and $\rho < 1-\epsilon$, then $\text{clip}(\rho)\hat{A} = (1-\epsilon)\hat{A} < \rho\hat{A}$ (multiplying by a negative number reverses the inequality $1-\epsilon > \rho$), and the minimum is the constant $(1-\epsilon)\hat{A}$. (iv) At $\theta = \theta_{\text{old}}$ every $\rho_i = 1$, in the interior of the clipping interval, so by (ii) the objective coincides near θ_{old} with $\frac{1}{N}\sum_i \rho_i \hat{A}_i$, whose gradient there is $\frac{1}{N}\sum_i \hat{A}_i \nabla_\theta \log \pi_\theta(a_i \mid s_i)$, as in the proof of Proposition 9.4. □

In words. An action with a positive advantage has its probability increased, but once it has been increased by a factor $1+\epsilon$ there is nothing more to gain; an action with a negative advantage is made less likely, down to a factor $1-\epsilon$. In the other directions (an action of positive advantage that became *less* likely, for example) the objective is not clipped: by (i) it stays a pessimistic bound, so a step that went too far can be corrected. Property (iv) shows that A2C is the special case of PPO with a single gradient step per batch.

PPO also has a *penalty* variant, which maximises $\hat{L} - \beta\,\overline{\text{KL}}$ and adapts β after each iteration (doubled if the KL divergence exceeded a target, halved if it was much smaller). The clipped version is usually preferred; the penalty form reappears in RLHF, where the KL divergence to a reference model is part of the objective (Chapter 15).

9.4 The algorithm

PPO (as implemented in the repository)

for each iteration **do**
  play $n = 128$ steps in each of $N = 16$ environments with $\pi_{\theta_{\text{old}}} = \pi_\theta$; store s, a, r, d, $\log \pi_{\theta_{\text{old}}}(a \mid s)$ and $V_{\text{old}}(s)$
  compute the GAE advantages $\hat{A}$ ($\gamma = 0.99$, $\lambda = 0.95$) and the value targets $\hat{A} + V_{\text{old}}$
  for $K = 4$ epochs **do**
    for each of 4 mini-batches of the 2048 transitions **do**
      normalise the advantages of the mini-batch
      gradient step on $-\overline{\ell^{\text{CLIP}}} + c_v\overline{(\hat{A} + V_{\text{old}} - V_\theta)^2}/2 - c_e\bar{\mathcal{H}}$, with gradient norm clipped to 0.5

The learning rate decreases linearly from 2.5×10^{-4} to 2.5×10^{-5}, $c_v = 0.5$ and $c_e = 0.03$. The loss of one mini-batch:

Listing 9.1: PPO loss of a mini-batch (`algorithms.py`, `A2C.update`, excerpt)

```
logits, values = self.a2c_net(states[idx])
dist = torch.distributions.Categorical(logits=logits)
log_probs = dist.log_prob(actions[idx])
```

```
entropy = dist.entropy().mean()

adv = advantages[idx]
adv = (adv - adv.mean()) / (adv.std() + 1e-8)  # normalised advantages
ratio = (log_probs - old_log_probs[idx]).exp()   # rho
policy_loss = -torch.min(
    ratio * adv,
    ratio.clamp(1 - self.clip_eps, 1 + self.clip_eps) * adv).mean()
value_loss = 0.5 * (returns[idx] - values).pow(2).mean()
loss = (policy_loss + self.value_loss_coef * value_loss
        - self.entropy_coef * entropy)
```

Two quantities monitor the size of the updates; the repository prints both during training.

Proposition 9.9. (An estimator of the KL divergence)

With $\rho = \pi_\theta(a \mid s)/\pi_{\text{old}}(a \mid s)$ and $a \sim \pi_{\text{old}}(\cdot \mid s)$, the quantity $k = (\rho - 1) - \log \rho$ is nonnegative and $\mathbb{E}[k] = \mathrm{KL}(\pi_{\text{old}}(\cdot \mid s) \,\|\, \pi_\theta(\cdot \mid s))$.

Proof. $x - 1 - \log x \geq 0$ for $x > 0$, since log is concave and $x - 1$ is its tangent at 1. Next, $\mathbb{E}[\rho] = \sum_a \pi_{\text{old}}(a \mid s)\pi_\theta(a \mid s)/\pi_{\text{old}}(a \mid s) = 1$ (assuming $\pi_{\text{old}} > 0$, true for a softmax), and $-\mathbb{E}[\log \rho] = \sum_a \pi_{\text{old}}(a \mid s) \log \frac{\pi_{\text{old}}(a|s)}{\pi_\theta(a|s)}$ is the KL divergence. □

In words. The plain estimate $-\log \rho$ has the right mean but can be negative on individual samples, which makes it noisy. Adding $\rho - 1$, whose mean is zero, changes nothing on average but makes every term nonnegative: each sample now says "this action's probability changed by this much", whichever the direction of the change.

Its batch average is the `kl` column of the training log, and the fraction of samples outside the clipping interval is the `clip` column. Typical values for the repository's agent are a KL of a few thousandths and a clip fraction of a few percent: the policy changes gently at each iteration.

9.5 Results on Pacman

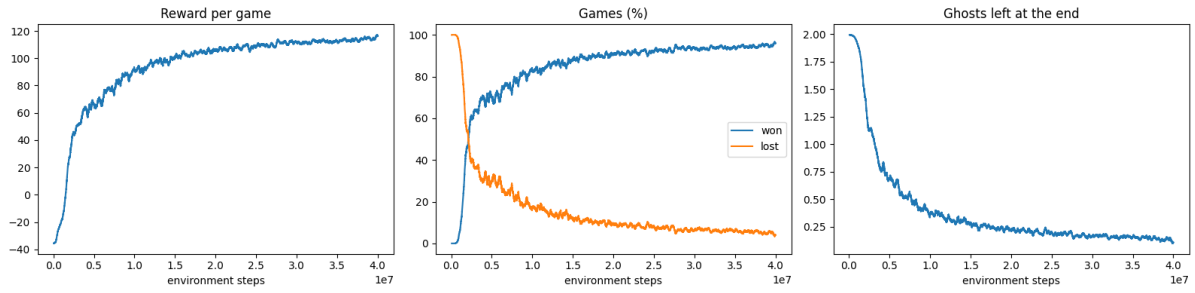


Figure 9.1: Training of the agent `Trainings/A2C/PPO_1` on a Kaggle GPU, game `v1`: reward per game, games won and lost, and ghosts left at the end, against the number of environment steps (averages over 2000 games).

Pacman 9.10. (From 79% to 97%)

Fig. 9.1 shows the training of the PPO agent over 40 million steps. The agent wins about half of its games after 2 million steps, 80% after 10 million, and about 95% at the end, while the number of ghosts left at the end falls from 2 to about 0.1. Evaluated on 1000 games:

	Original A2C	PPO agent (`PPO_1`)
Games won	79.4%	96.6%
Games ending with no ghost left	51.4%	93.6%
Games with both power cookies eaten	83.4%	96.8%
Average number of ghosts left	0.69	0.095

The original A2C agent of the repository used a stride-2 network on a single RGB image, n-step returns and one gradient step per batch. The PPO agent differs in several ways at once: feature planes and two frames, a full-resolution symmetric network, GAE, PPO updates, a curriculum and a larger entropy bonus. Since all were changed together, the experiment does not tell how much each one contributes; it shows that together they divide the losing rate by six.

9.6 Summary

- The performance difference lemma measures a new policy with the advantages of the old one; with importance sampling it gives a surrogate objective that can be estimated from the old policy's data and agrees with J to first order.
- PPO maximises a clipped version of the surrogate, a pessimistic bound that removes the incentive to change the probability of an action by more than a factor $1 \pm \epsilon$; this allows several epochs of mini-batch updates on each batch. A2C is the case of a single step.
- The KL estimator $(\rho - 1) - \log \rho$ and the clip fraction monitor the size of the updates.
- On Pacman, PPO with a better input and network wins 96.6% of the games, against 79.4% for the original A2C agent.

Chapter 10

Continuous actions: DDPG, TD3 and SAC

In robotics and control the actions are vectors of real numbers (torques, forces, steering angles). Pacman's five moves are not, so this chapter uses the classical continuous control benchmarks: swinging up a pendulum, making a simulated robot walk (the MuJoCo tasks). Policy gradients with Gaussian policies (Section 8.5) work there, and PPO is a standard baseline; but they are on-policy and need many samples. The algorithms of this chapter are *off-policy actor-critic* methods: like DQN they learn a Q-function from a replay memory, and they replace the maximisation over actions, impossible on a continuum, by a learnt actor.

10.1 Deterministic policy gradients

Let $\mu_\theta : \mathcal{S} \to \mathbb{R}^k$ be a deterministic policy and Q^μ its action-value function. The idea of DDPG is to move the action $\mu_\theta(s)$ in the direction that increases $Q^\mu(s, \cdot)$.

Theorem 10.1. (Deterministic policy gradient (stated without proof))

Under regularity conditions (Silver et al., 2014), with $J(\theta) = \mathbb{E}[\sum_t \gamma^t R_{t+1}]$ for the policy μ_θ and ρ^μ its discounted state distribution,

$$\nabla_\theta J(\theta) = \mathbb{E}_{s\sim\rho^\mu}\left[\nabla_\theta \mu_\theta(s)\, \nabla_a Q^\mu(s,a)\big|_{a=\mu_\theta(s)}\right]. \tag{10.1}$$

The formula is the chain rule applied to $Q^\mu(s, \mu_\theta(s))$, with the remarkable fact (as in the stochastic policy gradient theorem) that the effect of θ on the state distribution does not appear. It is also the limit of the stochastic policy gradient for Gaussian policies whose variance tends to 0. Since the gradient involves no expectation over actions, it can be estimated from states drawn from a replay memory filled by an exploratory policy: the deterministic policy gradient is naturally off-policy.

> **In words.** The actor proposes an action; the critic, being a differentiable function of the action, can say in which direction the action should be nudged to get a higher value ($\nabla_a Q$). The chain rule then tells the actor how to change its parameters to produce the nudged action. The actor learns to climb the critic's landscape, which is why everything depends on the critic's landscape being accurate.

10.2 DDPG

Deep deterministic policy gradient (DDPG, Lillicrap et al., 2016) is "DQN for continuous actions". A critic Q_ϕ is trained with the Bellman target of the deterministic policy, using target networks for both the critic and the actor:

$$y = r + \gamma(1-d)\, Q_{\phi^-}(s', \mu_{\theta^-}(s'))\,, \tag{10.2}$$

and the actor follows (10.1) with the critic in place of Q^μ: it maximises $\mathbb{E}_s[Q_\phi(s, \mu_\theta(s))]$ by gradient ascent through the critic. Exploration adds noise to the actions (Gaussian, or an Ornstein–Uhlenbeck process in the original paper), and the target networks are updated by Polyak averaging. DDPG works but is brittle: the actor exploits the errors of the critic, seeking actions where Q_ϕ is overestimated, and the overestimation is propagated by the targets.

10.3 TD3

Twin delayed DDPG (TD3, Fujimoto et al., 2018) adds three corrections.

1. **Clipped double Q-learning.** Two critics Q_{ϕ_1}, Q_{ϕ_2} are trained, and the target uses the smaller of their target versions:

$$y = r + \gamma(1-d)\, \min_{i=1,2} Q_{\phi_i^-}(s', \tilde{a}'), \tag{10.3}$$

   a pessimistic estimate that counters the maximisation bias.

2. **Target policy smoothing.** $\tilde{a}' = \mu_{\theta^-}(s') + \mathrm{clip}(\varepsilon, -c, c)$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$: the target is averaged over nearby actions, so that a narrow peak of the critic cannot be exploited.

3. **Delayed policy updates.** The actor and the target networks are updated once every two critic updates, so that the actor follows a critic that has had time to settle.

Proposition 10.2. (The minimum of two estimates is not optimistic)

Let $Q_1 = q + \varepsilon_1$ and $Q_2 = q + \varepsilon_2$ be two estimates of a value q with errors of mean zero. Then $\mathbb{E}[\min(Q_1, Q_2)] \le q$, with strict inequality as soon as $\mathbb{P}(\varepsilon_1 \neq \varepsilon_2) > 0$.

Proof. $\min(Q_1, Q_2) = \frac{Q_1+Q_2}{2} - \frac{|Q_1-Q_2|}{2}$, whose expectation is $q - \frac{1}{2}\mathbb{E}\,|\varepsilon_1 - \varepsilon_2|$. □

The bias is now downwards. An underestimated action is avoided by the actor and is not explored further, which limits the damage, whereas an overestimated one is sought and its error propagated: pessimism is the safer error.

10.4 Soft actor-critic

Soft actor-critic (SAC, Haarnoja et al., 2018) learns a stochastic policy that maximises the reward *and* its own entropy.

Definition 10.3. (Maximum entropy objective)

For a temperature $\alpha > 0$,

$$J_\alpha(\pi) = \mathbb{E}_\pi\left[\sum_t \gamma^t \left(R_{t+1} + \alpha\, \mathcal{H}(\pi(\cdot \mid S_t))\right)\right], \tag{10.4}$$

where $\mathcal{H}(\pi(\cdot \mid s)) = -\mathbb{E}_{a\sim\pi(\cdot|s)}[\log \pi(a \mid s)]$. The *soft* action values of π satisfy

$$Q^\pi(s,a) = r(s,a) + \gamma\, \mathbb{E}_{s'}\left[V^\pi(s')\right], \qquad V^\pi(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}\left[Q^\pi(s,a) - \alpha \log \pi(a \mid s)\right]. \tag{10.5}$$

In words. The agent is paid for keeping its options open: at each step it receives, besides the reward, a bonus equal to the entropy of its policy, converted into reward units by the temperature α. Among behaviours with the same reward it prefers the most random one, and it becomes deterministic only where determinism pays more than α times the entropy given up. In the soft value, $-\alpha \log \pi(a \mid s)$ is the bonus earned by playing an unlikely action.

The entropy term rewards the policy for keeping several actions possible: it explores by construction, it learns several ways of solving a task, and it is more robust to perturbations. We prove the two steps of soft policy iteration for finitely many actions (the continuous case is analogous with densities).

Proposition 10.4. (Soft policy evaluation)

For a fixed policy π, the operator $(\mathcal{T}^\pi_{\text{soft}} Q)(s,a) = r(s,a) + \gamma\, \mathbb{E}_{s'}\mathbb{E}_{a'\sim\pi}[Q(s',a') - \alpha \log \pi(a' \mid s')]$ is a γ-contraction in the sup norm; its fixed point is the soft action-value function Q^π.

Proof. The entropy terms do not depend on Q: they are an additional reward $r'(s,a) = -\alpha\gamma\, \mathbb{E}_{s'}\mathbb{E}_{a'}[\log \pi(a' \mid s')]$, bounded for finitely many actions. The operator is therefore the Bellman expectation operator for action values with reward $r + r'$, a γ-contraction (Lemma 3.13). □

Lemma 10.5. (Soft policy improvement)

Let $\pi_{\text{new}}(\cdot \mid s) \propto \exp(Q^\pi(s,\cdot)/\alpha)$ for every s (the minimiser of $\mathrm{KL}(\pi'(\cdot \mid s) \,\|\, \exp(Q^\pi(s,\cdot)/\alpha)/Z(s))$). Then $Q^{\pi_{\text{new}}} \geq Q^\pi$.

Proof. For a fixed state s, let $f(\pi') = \mathbb{E}_{a\sim\pi'}[Q^\pi(s,a) - \alpha \log \pi'(a \mid s)]$. Writing $\pi^\star(a) = \exp(Q^\pi(s,a)/\alpha)/Z(s)$, we have $Q^\pi(s,a) = \alpha \log \pi^\star(a) + \alpha \log Z(s)$, hence $f(\pi') = \alpha \log Z(s) - \alpha\, \mathrm{KL}(\pi' \,\|\, \pi^\star)$, maximal at $\pi' = \pi^\star = \pi_{\text{new}}(\cdot \mid s)$. In particular $f(\pi_{\text{new}}) \geq f(\pi) = V^\pi(s)$. Then, as in the proof of Theorem 4.2,

$$\begin{aligned} Q^\pi(s,a) &= r(s,a) + \gamma \mathbb{E}_{s'}[V^\pi(s')] \\ &\leq r(s,a) + \gamma \mathbb{E}_{s'}\mathbb{E}_{a'\sim\pi_{\text{new}}}\left[Q^\pi(s',a') - \alpha \log \pi_{\text{new}}(a' \mid s')\right] = (\mathcal{T}^{\pi_{\text{new}}}_{\text{soft}} Q^\pi)(s,a), \end{aligned}$$

and iterating the monotone contraction $\mathcal{T}^{\pi_{\text{new}}}_{\text{soft}}$ gives $Q^\pi \leq Q^{\pi_{\text{new}}}$. □

In words. The improved policy is a *Boltzmann* distribution over the actions: each is played with a probability proportional to $e^{Q/\alpha}$. The temperature α sets how sharply the better actions are preferred: an action worse by α is played $e \approx 2.7$ times less often; as $\alpha \to 0$ only the best actions remain, and as $\alpha \to \infty$ the choice becomes uniform.

As $\alpha \to 0$, π_{new} concentrates on the greedy actions and soft policy iteration becomes policy iteration. In practice:

- two soft critics are trained (clipped double Q, as in TD3) towards $y = r + \gamma(1 - d)\left(\min_i Q_{\phi_i^-}(s', \tilde{a}') - \alpha \log \pi_\theta(\tilde{a}' \mid s')\right)$ with $\tilde{a}' \sim \pi_\theta(\cdot \mid s')$;

- the actor minimises $\mathbb{E}_s \mathbb{E}_{a\sim\pi_\theta}[\alpha \log \pi_\theta(a \mid s) - \min_i Q_{\phi_i}(s,a)]$, the KL divergence of the lemma up to a constant, using the *reparametrisation trick*: $a = \tanh(\mu_\theta(s) + \sigma_\theta(s) \odot \xi)$ with $\xi \sim \mathcal{N}(0, I)$, so that gradients flow through the sampled action;
- the temperature is adjusted automatically so that the entropy of the policy stays near a target (for example $-k$ for k-dimensional actions), by gradient descent on $\alpha\, \mathbb{E}[-\log \pi(a \mid s) - \bar{\mathcal{H}}]$.

Proposition 10.6. (Density of a squashed Gaussian)

If $u \in \mathbb{R}^k$ has density ν and $a = \tanh(u)$ componentwise, then

$$\log \pi(a) = \log \nu(u) - \sum_{i=1}^{k} \log\left(1 - \tanh^2(u_i)\right). \tag{10.6}$$

Proof. The map $u \mapsto \tanh(u)$ is a bijection from $\mathbb{R}^k$ to $(-1,1)^k$ with diagonal Jacobian of entries $1 - \tanh^2(u_i) > 0$. By the change of variables formula, $\pi(a) = \nu(u) / \prod_i (1 - \tanh^2(u_i))$. □

In words. tanh squeezes the whole real line into $(-1, 1)$, compressing more and more towards the ends: a large interval of values of u near the tails is mapped to a tiny interval near ± 1, where the probability therefore piles up. The density of a is that of u divided by the local compression factor $1 - \tanh^2(u)$. Forgetting this correction would make the entropy term of SAC wrong precisely at the bounds of the actions.

SAC is sample-efficient, stable across hyperparameters, and one of the standard algorithms for continuous control; a discrete-action variant also exists, in which the expectations over actions are computed exactly.

10.5 Summary

- With continuous actions, a learnt actor replaces the maximisation over actions; the deterministic policy gradient moves the action along the gradient of the critic, and is naturally off-policy.
- DDPG is DQN for continuous actions; TD3 makes it reliable with clipped double critics (a downward bias, the safer one), target policy smoothing and delayed actor updates.
- SAC maximises reward plus entropy; soft policy evaluation and improvement mirror their classical versions, and with the reparametrisation trick and automatic temperature it is a robust default for continuous control.

Chapter 11

Model-based reinforcement learning and planning

Model-free methods learn values or policies directly from experience. Model-based methods also use a *model* of the environment, given or learnt, to *plan*: to compute what to do by simulating rather than by acting. Planning can happen in the background, generating simulated experience to train the values or the policy (Dyna, MBPO), or at decision time, searching from the current state before each move (rollouts, MCTS, AlphaZero, MuZero). This chapter covers both, then returns to Pacman: the look-ahead search of the repository, the question of what the search may know about the ghosts, and expert iteration, which teaches the search's decisions back to the network.

11.1 Models and background planning

A *model* predicts the next state and reward, $\hat{p}(s', r \mid s, a)$, either as a distribution or as a sampler. It can be known (the rules of a board game, a physics simulator, a copy of the Pacman environment) or learnt from the transitions observed, by supervised learning.

Dyna-Q (Sutton, 1990)

for each real step **do**
    in state s, play a (ε-greedy w.r.t. Q), observe r, s'
    Q-learning update with (s, a, r, s')
    model update: Model$(s, a) \leftarrow (r, s')$
    for n planning steps **do**
        pick a previously visited $(\tilde{s}, \tilde{a})$; $(\tilde{r}, \tilde{s}') \leftarrow$ Model$(\tilde{s}, \tilde{a})$
        Q-learning update with $(\tilde{s}, \tilde{a}, \tilde{r}, \tilde{s}')$

Each real transition is reused through the model many times, propagating values much faster than with Q-learning alone. *Prioritised sweeping* chooses the simulated updates where they matter: when a value changes, the predecessors of the state are queued with a priority equal to the size of the change they would undergo.

A learnt model is wrong, and planning amplifies its errors: the planner seeks the actions whose predicted outcome is best, including those where the model is optimistically wrong. The following lemma quantifies the effect of model errors on values.

Lemma 11.1. (Simulation lemma)

Let $M = (p, r)$ and $\hat{M} = (\hat{p}, \hat{r})$ be two MDPs on the same states and actions with rewards bounded by $R_{\max}$, and suppose $|r(s,a) - \hat{r}(s,a)| \le \varepsilon_r$ and $\sum_{s'} |p(s' \mid s,a) - \hat{p}(s' \mid s,a)| \le \varepsilon_p$ for all s, a. Then for every policy π,

$$\left\| v^\pi_M - v^\pi_{\hat{M}} \right\|_\infty \le \frac{\varepsilon_r}{1-\gamma} + \frac{\gamma\, \varepsilon_p\, R_{\max}}{(1-\gamma)^2}. \tag{11.1}$$

Proof. With the matrix notation of Proposition 3.14, $v = r_\pi + \gamma P_\pi v$ and $\hat{v} = \hat{r}_\pi + \gamma \hat{P}_\pi \hat{v}$, so

$$v - \hat{v} = (r_\pi - \hat{r}_\pi) + \gamma(P_\pi - \hat{P}_\pi)v + \gamma \hat{P}_\pi (v - \hat{v}).$$

Each row of $P_\pi - \hat{P}_\pi$ has ℓ_1 norm at most ε_p and $\|v\|_\infty \le R_{\max}/(1-\gamma)$ (Proposition 3.5), and $\hat{P}_\pi$ is stochastic. Taking sup norms, $\|v - \hat{v}\|_\infty \le \varepsilon_r + \gamma\varepsilon_p R_{\max}/(1-\gamma) + \gamma \|v - \hat{v}\|_\infty$; solve. □

In words. A model that is wrong about the rewards by ε_r makes that error at every simulated step, hence $\varepsilon_r/(1-\gamma)$ over the horizon. A model that is wrong about the transitions is worse: at each step it sends the simulation to slightly wrong states, and landing in the wrong place can cost as much as the whole range of values, $R_{\max}/(1-\gamma)$; this too happens at every step, hence the square. Imagined trajectories drift away from reality the longer they run.

The factor $1/(1-\gamma)^2$ reflects the compounding of errors over long simulated trajectories. Modern methods therefore keep the simulated trajectories short: MBPO (Janner et al., 2019) branches rollouts of a few steps from real states with an ensemble of learnt models, and trains SAC on the mixture of real and simulated transitions. World-model methods (Dreamer, Hafner et al., 2020–2023) learn a latent dynamics model from images and train an actor-critic entirely in imagination.

11.2 Decision-time planning

Background planning improves the policy everywhere; decision-time planning concentrates the computation on the current state. It is how game-playing programs have always worked, and it pays off when the policy alone is imperfect: a trained policy decides each move in one glance at the board, and when the PPO agent of Chapter 9 loses, it is usually because it walks into a trap that only closes a few moves later.

11.2.1 Rollouts

Suppose we can simulate the game from the current state: here, a copy of the environment (`PacmanEnvironment.clone()`). To evaluate a move a in state s, play a in a copy, then let a *rollout policy* π play for at most H moves, and score the simulation by

$$\hat{G} = \sum_{k=0}^{m-1} \gamma^k R_{k+1} + \gamma^m V(S_m), \tag{11.2}$$

where $m \le H+1$ is the number of moves simulated and $V(S_m) = 0$ if the game ended. Averaging N independent simulations gives an estimate $\hat{q}(s,a)$.

Proposition 11.2. (What a rollout estimates)

If $V = v_\pi$, then $\mathbb{E}[\hat{G}] = q_\pi(s,a)$ for every horizon H, and the average of N independent simulations has variance $\mathrm{Var}(\hat{G})/N$. Playing in s an action maximising $q_\pi(s,\cdot)$, and following π afterwards, is at least as good as π.

Proof. $\mathbb{E}[\hat{G}]$ is $(\mathcal{T}^\pi)^H V$ applied after the first move, as in Proposition 5.6; with $V = v_\pi$ every application of $\mathcal{T}^\pi$ leaves v_π unchanged, so $\mathbb{E}[\hat{G}] = \mathbb{E}[R_1 + \gamma v_\pi(S_1) \mid s,a] = q_\pi(s,a)$. The variance of a mean of N independent copies is the variance divided by N. The last claim is the policy improvement theorem (Corollary 4.3) applied in the single state s. □

In the repository the rollout policy is the trained agent itself (greedy moves), and V its value head; the rollouts therefore estimate what happens if the agent plays a and then goes on as usual.

Common random numbers. The moves of the ghosts are random. To compare two candidate moves, the k-th simulation of each move uses the same random numbers. For two estimates X and Y,

$$\mathrm{Var}(X - Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) - 2\,\mathrm{Cov}(X,Y), \tag{11.3}$$

and simulations that share the ghosts' random draws are positively correlated, so their *difference*, which decides the move, is much less noisy than each of them.

11.2.2 Two ways of choosing the move

Definition 11.3. (Value mode and safety mode)

Let M be the set of candidate moves (the moves leading to different cells), a_0 the agent's move, and, for each $a \in M$, $\hat{q}(a)$ the average score and $\hat{d}(a)$ the fraction of its N simulations in which Pacman dies.

- *Value mode:* play $\arg\max_{a \in M} \hat{q}(a)$.
- *Safety mode:* keep a_0 unless $\hat{d}(a_0) - \min_a \hat{d}(a) \geq \kappa$; otherwise play, among the moves of minimal death rate, the one of best average score. The repository uses $\kappa = 0.15$.

The search is only used when a ghost is at most 5 cells away (Manhattan distance), about 33 moves per game.

Pacman 11.4. (Why the value mode fails)

With $N = 20$ simulations, the value mode *lowered* the win rate (about 95.5%, against 96.1% without search) and changed about a quarter of the agent's moves. The reason is noise: the returns of a simulation range from -35 (death) to $+20$ and more (a ghost eaten), so the standard deviation of a mean of 20 of them is of the order of a few points, while the difference in value between two reasonable moves is often a fraction of a point. The maximum of noisy estimates then picks a random move (Proposition 5.14). The death rate is a much more robust signal: a trap shows up as many deaths for one move and few for another. The safety mode changes the agent's move about 0.6 times per game, only when it clearly walks into a trap.

In words. The safety mode does not ask the search which move is best, a question its 20 noisy simulations per move cannot answer reliably. It asks a simpler one: is the move the agent wants to play clearly more dangerous than another? The trained policy keeps the

initiative, and the search acts as a safety net that vetoes a move only when the evidence is strong.

Listing 11.1: Choosing the move (`pacman_search.py`, `LookaheadSearch.select_action`, excerpt)

```
scores = {m: np.mean([s["score"] for s in sims if s["move"] == m])
          for m in moves}
if self.mode == "value":
    best = max(moves, key=lambda m: scores[m])
    if scores[agent_move] >= scores[best] - 1e-9:
        return agent_action
else:  # safety mode
    deaths = {m: np.mean([s["env"].dead for s in sims
                          if s["move"] == m]) for m in moves}
    safest = min(deaths.values())
    if deaths[agent_move] - safest < self.margin:   # kappa = 0.15
        return agent_action
    best = max((m for m in moves if deaths[m] == safest),
               key=lambda m: scores[m])
return best
```

11.3 Monte Carlo tree search

The search of the repository is the simplest form of Monte Carlo planning: a tree of depth one with the same number of simulations for every move. Monte Carlo tree search (MCTS) grows a deeper tree, asymmetrically, and allocates the simulations to the moves that deserve them. Each node of the tree stores, for each action, a visit count $N(s,a)$ and a mean value $\hat{q}(s,a)$.

Monte Carlo tree search (one simulation)

1. **selection:** from the root, descend the tree choosing at each node the action maximising a selection rule (UCT below), until a leaf
2. **expansion:** add one child of the leaf to the tree
3. **evaluation:** estimate the value of the new node, by a rollout with a default policy, or by a value network
4. **backup:** along the path, increment $N(s,a)$ and update $\hat{q}(s,a)$ with the observed return

After the simulations, the move played is the most visited action of the root. The UCT selection rule (Kocsis and Szepesvári, 2006) treats each node as a bandit and applies UCB1 (Definition 2.5):

$$a = \arg\max_a \left(\hat{q}(s,a) + c\sqrt{\frac{\ln N(s)}{N(s,a)}} \right), \tag{11.4}$$

with $N(s) = \sum_a N(s,a)$. Kocsis and Szepesvári proved that, with suitable c, the probability of choosing a wrong move at the root tends to 0 as the number of simulations grows. In a stochastic game such as Pacman, the tree is built over sequences of the agent's moves ("open-loop" MCTS), each simulation replaying the sequence on a fresh copy of the game, or the tree also branches on chance outcomes.

In words. MCTS places a bandit at every node of the tree. Moves that look good are simulated more and their subtrees grow deeper; moves that look bad are revisited only now and then, in case their first results were unlucky. The tree grows towards the promising

lines, as a chess player analyses the critical variations deeply and dismisses the others after a glance.

Remark 11.5. (MCTS for two-player games)

In a game between two opponents, such as Hex, chess or Go, the players alternate along each path of the tree, and the statistics of a node are kept from the point of view of the player who made the move leading to it. During the backup the result is therefore flipped at each level: a win for one player is a loss for the other. Each player then selects, with UCT, the moves that are best *for itself*, which is the minimax principle obtained by sampling. The move finally played can be the most visited child of the root (the usual, most robust choice), the child with the highest mean value (optimistic when it has few visits), or the child with the highest lower confidence bound (pessimistic).
A C++ implementation for Hex, with brute force, flat Monte Carlo and MCTS players, and a step-by-step account of the algorithm on a small example, is in the repository CL-BZH/Hex_game_AI (`hex_machine_mcts.h` and `Docs/mcts.pdf`). The MCTS Visualizer (repository CL-BZH/MCTS-Visualizer) shows the tree growing iteration by iteration on a simple two-player game, with the four phases and the UCB1 values of each node.

11.3.1 AlphaZero

AlphaZero (Silver et al., 2018) learnt chess, shogi and Go from the rules alone by combining MCTS with a network $f_\theta(s) = (\mathbf{p}, v)$ that outputs a prior policy and a value. MCTS uses them through the PUCT rule

$$a = \arg\max_a \left(\hat{q}(s,a) + c\, p_a(s) \frac{\sqrt{N(s)}}{1 + N(s,a)} \right), \tag{11.5}$$

which explores first the moves the network finds promising, and evaluates the leaves with v instead of rollouts. The games of self-play provide the training data: the network is trained to predict the visit distribution of the search, $\boldsymbol{\pi}_t \propto N(s_t, \cdot)$, and the final result z of the game,

$$\ell(\theta) = (z - v)^2 - \boldsymbol{\pi}_t^\top \log \mathbf{p} + c \left\| \theta \right\|^2 . \tag{11.6}$$

The search is a *policy improvement operator*: its visit distribution is better than the prior it started from, and training the network on it is *expert iteration* (Anthony et al., 2017). The network then makes the next searches better, and so on.

In words. In PUCT the prior $p_a(s)$ plays the role of intuition: the moves the network likes are explored first, and the bonus, decreasing like $1/(1 + N(s,a))$, lets the search's own statistics take over as they accumulate. In the loss, the value head learns to predict who wins, and the policy head learns to imitate the search, which plays better than the raw network because it looks ahead. The program pulls itself up: a better network makes a better search, which makes a better teacher.

11.3.2 MuZero

MuZero (Schrittwieser et al., 2020) plans without knowing the rules. It learns three functions: a representation h mapping observations to a latent state, a dynamics function $g(z, a) = (\hat{r}, z')$, and a prediction function $f(z) = (\mathbf{p}, v)$. The latent model is not trained to reconstruct observations, only to predict, along real trajectories unrolled for a few steps, the rewards, the values and the search policies: it models only what matters for planning. MuZero matched AlphaZero in board games and set the state of the art on Atari at the time.

11.4 What does the search know about the ghosts?

A copy of the environment simulates the ghosts with the game's own code, which the agent never sees. Using it would give the search a *perfect model* of the opponents: legitimate in a board game whose rules are known, debatable for Pacman, where the behaviour of the ghosts is part of what the agent should discover. The search therefore offers three *ghost models*:

- `exact`: the game's code (perfect model), used only as a reference;
- `chase` (default): worst case; when Pacman has no power each ghost moves along a shortest path towards him, and randomly when he has power;
- `estimated`: a ghost chases with probability p_{chase} and moves randomly otherwise; when Pacman has power it moves away from him with probability p_{flee}. Both probabilities are estimated for each ghost from its observed moves.

With `chase` and `estimated`, the search uses only what the agent observes (positions, power) and the rules for Pacman himself.

Proposition 11.6. (Estimating a ghost's chase probability)

Model each observed move of a ghost as: towards Pacman with probability p, otherwise uniform among its n reachable cells, of which m go towards Pacman ($0 < m < n$). Then the probability that the move goes towards Pacman is $p+(1-p)\,m/n$, and after observing moves with ratios $q_i = m_i/n_i$ the posterior density of p under a prior $\text{Beta}(2,2)$ is proportional to

$$p(1-p)\prod_{i\in\mathcal{T}}(p+(1-p)q_i)\prod_{i\notin\mathcal{T}}(1-p)(1-q_i), \tag{11.7}$$

where $\mathcal{T}$ is the set of moves that went towards Pacman.

Proof. A move goes towards Pacman either because the ghost chases (probability p) or because it moves randomly (probability $1-p$) and picks one of the m cells out of n. The moves are independent given p, and the posterior is the prior density $\propto p(1-p)$ times the likelihood. □

In words. A move towards Pacman is evidence that the ghost chases, but weak evidence when most of the ghost's possible moves go towards Pacman anyway (q_i close to 1): a random ghost would have done the same. A move away from Pacman is strong evidence of randomness, since a chasing ghost never makes it. The posterior weighs each observation by how much it discriminates between the two behaviours.

The repository takes the maximum of this posterior on a grid. Over many games, it recovers $p \approx 0.73$ and 0.64 for the two ghosts (their configured aggressiveness is 0.74 and 0.64), and a flee probability of about 0.08: edible ghosts move almost at random. The first version of the models assumed that edible ghosts always flee; the simulated hunts then failed, the search judged eating a ghost dangerous and made the agent hesitate in front of an edible ghost. Estimating the flee probability fixed it: a model that is wrong in the optimistic or the pessimistic direction can both mislead a planner.

Pacman 11.7. (The search on 1000 games)

With 20 simulations, horizon 15, safety mode, the search was evaluated on 1000 games: all the games lost without search, and a sample of the won ones, were replayed with search from the same starting conditions.

Ghost model	Estimated win rate
no search	96.1%
`exact`	$97.1\% \pm 0.5$
`chase`	$97.5\% \pm 0.5$
`estimated`	$97.7\% \pm 0.3$

The models that do not know the ghosts' code do at least as well as the exact one: the gain comes from seeing a trap a few moves ahead, not from knowing the opponents' rules.

11.5 Expert iteration

The search is expensive (up to about 100 simulated games per searched move). If the network could learn to make the search's decisions by itself, the gain would be free at play time. Simply training the network on games played *by the search* (behaviour cloning) has a known flaw: the network only sees the situations that the search gets into, and when it makes a mistake of its own, it reaches situations it has never seen, where its errors compound (Ross and Bagnell, 2010). The remedy, DAgger (Ross, Gordon and Bagnell, 2011), lets the *learner* play and asks the *expert* what it would have done in the situations the learner reached.

Expert iteration (DAgger form), `pacman_expert.py`

1: start from the PPO agent π_θ
2: **for** each iteration **do**
3: the agent plays 2000 games alone; whenever a ghost is close, record the state s and the search's move $a^\star(s)$
4: record also a fraction of the other states, labelled with the agent's own move, and the discounted returns observed
5: train on the data of the last iterations: weighted cross-entropy $-w(s)\log \pi_\theta(a^\star(s) \mid s)$ with $w(s) = 10$ where the search disagreed with the agent and 1 otherwise, and squared error of the value head

The labels of the states where the search agrees with the agent keep the rest of its behaviour unchanged; the weight 10 makes the few disagreements count. Because the games are played by the agent alone, each iteration also measures its progress.

Pacman 11.8. (Results of expert iteration)

Ten iterations of 2000 games on Kaggle (search with 10 simulations, horizon 10, `chase` model), about one hour each, almost all of it spent in the search (on CPU). The search disagreed with the agent 0.27 times per game at the first iteration and 0.12 at the last, and the network reproduced 76% of the corrections of its training data. On the same 1000 games:

Same 1000 games	Without search	With search (20)
PPO agent (`PPO_1`)	97.1%	98.1%
After expert iteration (`Expert_1`)	97.7%	97.9%

Expert iteration and the search correct the same mistakes: without search the expert agent recovers about 60% of the search's gain (and no longer loses games by the "off-by-one" of Section 1.4), but with the search both agents reach about 98%.

11.6 What is left?

The repository classifies the lost games by cause (`pacman_play.py`). For the PPO agent alone, out of about 38 losses in 1000 games, 2 come from the off-by-one, 1 from a ghost catching Pacman just after his power ended, 6 from *bad starts* (Pacman caught in the first five moves), and the others from traps. With the search, the same three bad starts are lost in every configuration, which suggests that they cannot be won. The 15 to 18 remaining losses resist both the search and expert iteration: they are probably traps from which the agent's own policy, which plays the simulations, cannot escape either. Going further would require a deeper search (MCTS), or, following AlphaZero, alternating the improvement of the search and of the network over many more iterations.

11.7 Summary

- A model can be used to generate simulated experience (Dyna, MBPO, Dreamer) or to search at decision time (rollouts, MCTS); the errors of a learnt model compound with the length of the simulations.
- MCTS treats each node as a bandit (UCT); AlphaZero guides it with a policy and value network and trains the network on the search's decisions; MuZero learns a latent model for planning.
- On Pacman, a depth-one search with a safety rule raises the win rate from 96% to about 98%, without knowing the ghosts' code; expert iteration transfers part of this gain to the network alone.

Chapter 12

Imitation learning and offline reinforcement learning

The previous chapters assumed that the agent learns by interacting with the environment and receiving rewards. In many applications this is expensive or dangerous (a car, a patient, a power grid), and the reward is hard to write down, while *data* are available: demonstrations by an expert, or logs of past decisions. This chapter studies two ways of using them. *Imitation learning* learns from demonstrations, without rewards; *offline reinforcement learning* learns from a fixed dataset of transitions with rewards, without further interaction.

12.1 Behaviour cloning

The simplest approach treats demonstrations $\mathcal{D} = \{(s_i, a_i)\}$ of an expert policy $\pi^\star$ as a supervised learning problem: fit π_θ by maximum likelihood,

$$\min_\theta \; \mathbb{E}_{(s,a)\sim\mathcal{D}} \left[-\log \pi_\theta(a \mid s) \right]. \tag{12.1}$$

Behaviour cloning (BC) is simple and often works well with enough data (it was used to drive a car as early as 1989, by ALVINN). Its weakness is that the states of the training set are those visited by the expert. When the learner makes a mistake, it reaches states the expert never visited, where it has no data and makes more mistakes: the errors compound.

Theorem 12.1. (Compounding errors of behaviour cloning)

Consider episodes of T steps with costs $c(s,a) \in [0,1]$, a deterministic expert $\pi^\star$, and let $J(\pi)$ be the expected total cost of π. Let d_t be the distribution of the expert's state at step t, and suppose that the learner π disagrees with the expert with probability at most ε on these distributions: $\mathbb{P}_{s\sim d_t,\, a\sim\pi(\cdot|s)}(a \neq \pi^\star(s)) \le \varepsilon$ for all t. Then

$$J(\pi) \le J(\pi^\star) + \varepsilon\, T^2. \tag{12.2}$$

Proof. Run the expert and the learner with the same random numbers for the environment, so that their trajectories coincide as long as their actions coincide. Let D_t be the event that the trajectories have diverged at or before step t. If they have not diverged before step t, the shared state is the expert's state, distributed as d_t, so the probability of a first disagreement at step t is at most ε, and $\mathbb{P}(D_t) \le t\varepsilon$. At step t the costs of the two trajectories are equal outside D_t and differ by at most 1 on D_t. Summing, $J(\pi) - J(\pi^\star) \le \sum_{t=1}^{T} t\varepsilon \le \varepsilon T^2$. □

In words. A learner who has only watched an expert drive has never seen the car on the shoulder of the road. After a small mistake it is in a situation it never trained on, where it makes more mistakes, and it may never return to familiar ground. A mistake at step t can spoil all the remaining steps, so the cost of a per-step error rate ε is multiplied by the horizon *twice*.

The bound is tight up to constants: there are problems where a single mistake leads to states from which the learner never recovers, and the cost is then of order εT^2, not the εT of a supervised learning error accumulated over T steps.

12.2 DAgger

The cure is to train the learner on the states *it* visits. DAgger (dataset aggregation, Ross, Gordon and Bagnell, 2011) requires an expert that can be queried in any state.

DAgger

1: $\mathcal{D} \leftarrow$ demonstrations of the expert; train π_1 on $\mathcal{D}$
2: **for** $i = 1, 2, \ldots, N$ **do**
3: run π_i (optionally mixed with the expert) to collect states s
4: ask the expert for its actions $\pi^\star(s)$; $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s, \pi^\star(s))\}$
5: train π_{i+1} on $\mathcal{D}$

DAgger is an online learning algorithm in disguise: training on the aggregated data is "follow the leader" on the sequence of losses defined by the learner's own state distributions. Ross et al. show, with a no-regret learner, that some policy of the sequence has a cost $J(\pi) \leq J(\pi^\star) + u\,T\varepsilon_N + o(1)$, where ε_N is the best training error on the aggregated data and u bounds the increase of the expert's cost-to-go after one wrong action: the error grows linearly in T when the expert can recover from the learner's mistakes.

In words. DAgger is a driving instructor in the passenger seat: the learner drives, and in the situations *it* gets into, including those caused by its own mistakes, the instructor says what it should have done. The training data then cover the states the learner actually visits, and the learner learns how to recover.

Pacman 12.2. (Expert iteration is DAgger)

In Chapter 11, the expert is the look-ahead search and the learner the PPO network: `pacman_expert.py` lets the network play and asks the search for its move in the states the network reached, which is DAgger. The search can be queried in any state because it only needs a copy of the game. Training on games played *by the search* would have been behaviour cloning, exposed to Theorem 12.1.

12.3 Inverse reinforcement learning

Behaviour cloning copies actions; inverse reinforcement learning (IRL) recovers the *reward* that the expert optimises, which transfers to new dynamics and explains the behaviour. The problem is ill-posed: the reward $r = 0$ makes every policy optimal, and adding to the reward a *shaping* term $\gamma\Phi(s') - \Phi(s)$, for any function Φ, changes the reward without changing the optimal policies (Ng, Harada and Russell, 1999). A principle is needed to choose among the rewards that explain the data.

Maximum entropy IRL. Ziebart et al. (2008) take a linear reward $r_\theta(\tau) = \theta^\top \phi(\tau)$ of the features $\phi(\tau) = \sum_t \phi(s_t, a_t)$ of a trajectory, and model the expert as choosing trajectories with probability $p_\theta(\tau) = \exp(\theta^\top \phi(\tau))/Z(\theta)$ (for deterministic dynamics): the maximum entropy distribution that matches the feature expectations. The parameters are fitted by maximum likelihood on the demonstrations.

Proposition 12.3. (Gradient of maximum entropy IRL)

The average log-likelihood $L(\theta) = \frac{1}{N}\sum_{i=1}^{N} \log p_\theta(\tau_i)$ of N demonstrations is concave, with gradient

$$\nabla L(\theta) = \frac{1}{N}\sum_{i=1}^{N} \phi(\tau_i) - \mathbb{E}_{\tau \sim p_\theta}\left[\phi(\tau)\right]. \tag{12.3}$$

Proof. $L(\theta) = \theta^\top \bar{\phi} - \log Z(\theta)$ with $\bar{\phi}$ the empirical mean of the features, and $\nabla \log Z(\theta) = \sum_\tau \phi(\tau) e^{\theta^\top \phi(\tau)}/Z(\theta) = \mathbb{E}_{p_\theta}[\phi]$. The Hessian of $\log Z$ is the covariance of ϕ under p_θ, positive semi-definite, so L is concave. □

In words. The gradient compares what the expert does with what the current model of the expert does. A feature the expert's trajectories show more often (staying far from the ghosts, say) gets a higher reward weight; a feature the model overproduces gets a lower one. Learning stops when the model's trajectories have the same feature statistics as the expert's.

At the optimum the model matches the expert's feature expectations. The expectation under p_θ requires solving a soft (maximum entropy) RL problem for the current reward, as in SAC (Chapter 10): IRL is an outer loop around RL, and therefore expensive.

Adversarial imitation. GAIL (Ho and Ermon, 2016) skips the reward: a discriminator $D_\psi(s, a)$ is trained to distinguish the expert's pairs from the learner's, and the policy is trained by RL (TRPO or PPO) with the reward $-\log(1 - D_\psi(s, a))$, so that it produces pairs the discriminator takes for the expert's. As in generative adversarial networks, at the equilibrium the occupancy measure of the learner matches that of the expert (the procedure minimises their Jensen–Shannon divergence). Unlike BC, GAIL matches distributions of states, not only actions in the expert's states, and does not suffer from compounding errors; but it needs interaction with the environment.

12.4 Offline reinforcement learning

In offline (or batch) RL, the agent receives a dataset $\mathcal{D} = \{(s_i, a_i, r_i, s'_i)\}$ collected by one or several *behaviour* policies β, and must produce the best possible policy without collecting more data. Q-learning and DQN are off-policy, so it is tempting to run them on $\mathcal{D}$. This fails in practice, often badly.

Extrapolation error. The target $r + \gamma \max_{a'} Q(s', a')$ evaluates Q at actions a' that may never appear in the data for s'. There the network's values are arbitrary, and the maximisation selects precisely the actions whose values are overestimated. Online, the agent would try these actions and correct the error; offline, nothing corrects it, and the errors propagate through the targets. The *distribution shift* between the learnt policy and β is the central problem of offline RL. Three families of methods address it.

Policy constraints. Keep the policy close to the behaviour policy. BCQ (Fujimoto et al., 2019) only maximises over actions proposed by a generative model of β. TD3+BC (Fujimoto and Gu, 2021) adds a behaviour cloning term to the actor of TD3 (Chapter 10):

$$\max_{\theta} \; \mathbb{E}_{(s,a)\sim\mathcal{D}} \left[\lambda \, Q_\phi(s, \mu_\theta(s)) - \|\mu_\theta(s) - a\|^2 \right], \tag{12.4}$$

a one-line change that is a strong baseline.

Conservative values. CQL (Kumar et al., 2020) learns a Q-function that underestimates the values of the actions absent from the data. For discrete actions it adds to the TD loss the term

$$\alpha \, \mathbb{E}_{s\sim\mathcal{D}} \left[\log \sum_a \exp Q(s,a) - \mathbb{E}_{a\sim\mathcal{D}(\cdot|s)}[Q(s,a)] \right], \tag{12.5}$$

which pushes down the values of all actions (through a soft maximum) and pushes up those of the actions in the data.

Listing 12.1: CQL loss for discrete actions (PyTorch)

```
q = q_net(states)                                      # (B, |A|)
q_data = q.gather(1, actions[:, None]).squeeze(1)  # Q(s, a), a in data
with torch.no_grad():
    target = rewards + gamma * (1 - dones) * \
        target_net(next_states).max(1).values
td_loss = F.smooth_l1_loss(q_data, target)
cql_loss = (torch.logsumexp(q, dim=1) - q_data).mean()
loss = td_loss + alpha * cql_loss
```

The following idealised, tabular version shows why the values become conservative. Let $\mu(\cdot \mid s)$ be the policy whose values are pushed down (in (12.5), the softmax of Q), and consider the iteration

$$\hat{Q}_{k+1} = \arg\min_{Q} \; \alpha \left(\mathbb{E}_{s,a\sim\mu}[Q(s,a)] - \mathbb{E}_{s,a\sim\beta}[Q(s,a)] \right) + \frac{1}{2} \mathbb{E}_{s,a\sim\beta} \left[\left(Q(s,a) - (\mathcal{T}^\pi \hat{Q}_k)(s,a) \right)^2 \right],$$

where the states are drawn from the data and the pairs weighted by $\beta(a \mid s) > 0$.

Proposition 12.4. (CQL underestimates the value)

The minimiser is $\hat{Q}_{k+1}(s,a) = (\mathcal{T}^\pi \hat{Q}_k)(s,a) - \alpha \left(\frac{\mu(a|s)}{\beta(a|s)} - 1 \right)$, and

$$\mathbb{E}_{a\sim\mu(\cdot|s)} \left[\hat{Q}_{k+1}(s,a) \right] \le \mathbb{E}_{a\sim\mu(\cdot|s)} \left[(\mathcal{T}^\pi \hat{Q}_k)(s,a) \right]. \tag{12.6}$$

Proof. The objective separates over the pairs (s,a). For one pair, the derivative with respect to $Q(s,a)$ is $\alpha(\mu(a \mid s) - \beta(a \mid s)) + \beta(a \mid s)(Q(s,a) - (\mathcal{T}^\pi \hat{Q}_k)(s,a))$ (up to the common factor of the state's weight); setting it to zero gives the minimiser. For the inequality, $\mathbb{E}_\mu[\mu/\beta - 1] = \sum_a \mu(a)^2/\beta(a) - 1 \ge 0$, by the Cauchy–Schwarz inequality $1 = (\sum_a \mu(a))^2 \le \sum_a \mu(a)^2/\beta(a) \cdot \sum_a \beta(a)$. □

The penalty is large where μ puts much more weight than the data: exactly the actions whose values cannot be trusted.

> **In words.** CQL tells the Q-function: "be pessimistic about what you have not seen". An action that the policy would choose much more often than the data contain it gets its value pushed down, in proportion to that imbalance; actions well covered by the data are left

almost unchanged. The policy then prefers actions whose good value is backed by evidence.

In-sample learning. IQL (Kostrikov et al., 2022) never evaluates Q at actions outside the data. It fits $V(s)$ to an upper expectile of $Q(s,a)$ over the actions of the data, minimising $\mathbb{E}_{(s,a)\sim\mathcal{D}}[L_\tau(Q(s,a)-V(s))]$ with $L_\tau(u)=|\tau-\mathbb{1}(u<0)|\,u^2$ and $\tau\approx 0.7$–0.9, which approximates a maximum over the actions supported by the data; Q is fitted to $r+\gamma V(s')$; and the policy is extracted by advantage-weighted regression, maximising $\mathbb{E}_{\mathcal{D}}[e^{b(Q(s,a)-V(s))}\log\pi(a\mid s)]$: behaviour cloning weighted by the advantage.

In words. An upper expectile of $Q(s,a)$ over the actions of the data is a soft version of "the best action that was actually tried in this state". IQL thus computes a maximum over the data's actions only, never asking the network about actions it has no data for, and then imitates the data, copying the good actions more than the bad ones.

Remark 12.5. (Offline RL and Pacman)

The replay memory of a DQN training run is an offline dataset of millions of transitions of Pacman. Running DQN on it from scratch, without interaction, is an instructive experiment: the values of the rare actions (walking into a ghost) are poorly constrained by the data, and the conservative loss above is a few lines to add to `DQN.learn`.

12.5 Summary

- Behaviour cloning is supervised learning on demonstrations; its errors compound quadratically with the horizon, because the learner leaves the states covered by the data.
- DAgger queries the expert in the learner's own states and reduces the error to linear in the horizon; expert iteration with a search as expert is DAgger.
- Inverse RL recovers a reward (maximum entropy IRL matches feature expectations); GAIL matches occupancy measures adversarially.
- Offline RL suffers from extrapolation error at actions absent from the data; policy constraints (TD3+BC), conservative values (CQL) and in-sample learning (IQL) avoid it.

Chapter 13

Exploration

An agent learns only about what it tries. The bandits of Chapter 2 showed the dilemma in its simplest form and the main answers: random exploration (ε-greedy), optimism (UCB) and posterior sampling (Thompson). In an MDP the dilemma is harder, because information is delayed: reaching an informative state may take many deliberate steps, and random actions rarely string them together. This chapter shows why, and presents the methods that scale the bandit ideas to deep RL.

13.1 Why dithering fails

ε-greedy, Boltzmann exploration and the entropy bonus of PPO are *dithering* strategies: they perturb each action independently. The following example, a "combination lock", shows that they can need a time exponential in the size of the problem.

Proposition 13.1. (Random exploration in a chain)

Consider states $0, 1, \dots, n$, starting in 0, where action *right* moves from k to $k+1$ and action *left* returns to 0; the only reward is at state n. Under the uniform random policy, the expected number of steps to reach n from 0 is $2^{n+1} - 2$.

Proof. Let T_k be the expected number of steps to reach n from k, so that $T_n = 0$ and $T_k = 1 + \frac{1}{2}T_{k+1} + \frac{1}{2}T_0$ for $k < n$. We show by induction that $T_k = T_0 - (2^{k+1} - 2)$. It holds for $k = 0$, and if it holds for k, then $T_{k+1} = 2T_k - 2 - T_0 = T_0 - 2^{k+2} + 2$. With $k = n$, $0 = T_0 - 2^{n+1} + 2$. □

> **In words.** To reach the reward, the agent must play *right* n times in a row, and a random agent does so with probability 2^{-n}; any *left* sends it back to the start. Random exploration tries combinations at random, and the number of combinations grows exponentially with the depth at which the reward is hidden.

A greedy agent with optimistic initial values solves the same chain in $O(n^2)$ steps: every unvisited action looks attractive, so the agent tries the actions of the states it reaches systematically. This is the principle of the provably efficient algorithms.

13.2 Optimism in the face of uncertainty

Provably efficient exploration. R-max (Brafman and Tennenholtz, 2002) treats every state-action pair visited fewer than m times as leading to a fictitious state of maximal reward $R_{\max}$, and plans in the resulting model; either the plan collects near-optimal reward, or it

visits an under-explored pair, which can happen only finitely often. R-max and its successors (E^3, MBIE, UCRL2) find a near-optimal policy with a number of mistakes polynomial in the numbers of states and actions, in $1/(1-\gamma)$ and in the accuracy: the MDP analogue of the logarithmic regret of UCB.

Exploration bonuses. A simpler form of optimism adds a bonus to the reward,

$$\tilde{r}(s,a) = r(s,a) + \frac{\beta}{\sqrt{N(s,a)}}, \tag{13.1}$$

where $N(s,a)$ is the number of visits (MBIE-EB, Strehl and Littman, 2008). The square root has the same origin as in UCB1 (Definition 2.5): the confidence interval of a mean estimated from N samples has a width of order $1/\sqrt{N}$. The bonus is propagated by the Bellman equations, so the agent is attracted to states from which under-explored pairs can be reached, not only to these pairs themselves.

> **In words.** The bonus is the error bar of UCB turned into a reward: the agent treats what it knows little about as if it were good. It is drawn to the unknown until the unknown has become known, and the Bellman equations carry this attraction back along the paths that lead there.

13.3 Counts in large state spaces

With images as states, every state is new, and $N(s) \in \{0,1\}$. Deep RL methods replace counts by quantities that generalise.

Pseudo-counts. Bellemare et al. (2016) derive a count from a density model $\rho_n(s)$ trained on the states seen so far. Let $\rho'_n(s)$ be the density the model would give to s after one more training step on s. If the model were an empirical frequency, $\rho_n(s) = N/n$ and $\rho'_n(s) = (N+1)/(n+1)$. Solving these two equations for N defines the *pseudo-count*

$$\hat{N}_n(s) = \frac{\rho_n(s)\,(1-\rho'_n(s))}{\rho'_n(s) - \rho_n(s)}, \tag{13.2}$$

used in the bonus (13.1). (From $\rho = N/n$ and $\rho' = (N+1)/(n+1)$: $n = N/\rho$, so $\rho'(N/\rho+1) = N+1$, which gives $N(\rho'-\rho) = \rho(1-\rho')$.) With a pixel density model, the method made the first real progress on *Montezuma's Revenge*, an Atari game where rewards are rare and DQN scored zero.

> **In words.** The pseudo-count asks: how much would seeing s once more change the model's belief in s? For a state seen thousands of times, one more sighting changes almost nothing; for a state seen once, it changes a lot. The size of the change reveals how familiar the state is, even when no state ever repeats exactly.

Prediction error as novelty. Random network distillation (RND, Burda et al., 2019) trains a network $\hat{f}_\psi$ to predict the output of a fixed, randomly initialised network f, on the states visited. The error $\left\|\hat{f}_\psi(s) - f(s)\right\|^2$ is small on states similar to those seen often, and large on new states: it is used as the bonus. The target is deterministic, so the error does not stay large on states whose future is merely random, a failure of the earlier *curiosity* methods, which use the error of a forward model of the dynamics. The intrinsic curiosity module (ICM, Pathak et al., 2017) avoids part of the problem by predicting in a feature space learnt to predict the agent's

own actions (an inverse model), which ignores what the agent cannot influence: a television showing random images is not interesting to it.

> **In words.** RND is a familiarity test: the predictor can only imitate the random network on states similar to those it has been trained on. A large error means "I have not been here before". Because the random network is fixed and deterministic, the error goes down once a state is familiar, even if what happens in that state is random.

Listing 13.1: Random network distillation bonus (PyTorch)

```
target = make_net().requires_grad_(False)   # fixed random network
predictor = make_net()
opt = torch.optim.Adam(predictor.parameters(), lr=1e-4)

def bonus(states):                          # also trains the predictor
    err = (predictor(states) - target(states)).pow(2).sum(1)
    opt.zero_grad(); err.mean().backward(); opt.step()
    return err.detach()                      # intrinsic reward
```

In practice the intrinsic rewards are normalised, combined with the extrinsic rewards with their own discount and value head, and used with PPO.

Remembering where to return. Go-Explore (Ecoffet et al., 2021) observed that novelty bonuses fade before the agent has learnt to return to the frontier of the explored region. It keeps an archive of visited "cells" (downscaled images), returns to a promising cell (deterministically in a simulator, or with a goal-conditioned policy), and explores from there; the trajectories found are then made robust by imitation. It solved *Montezuma's Revenge* and *Pitfall*.

13.4 Posterior sampling and deep exploration

Thompson sampling also extends to MDPs. Posterior sampling for RL (PSRL, Osband et al., 2013) keeps a posterior over MDPs, samples one at the start of each episode, and follows its optimal policy for the whole episode. Committing to one hypothesis for an episode produces *deep* exploration: if the sampled MDP is optimistic about a distant state, the agent goes there deliberately, unlike dithering. PSRL has a regret of order $\tilde{O}(HS\sqrt{AT})$ for episodes of length H.

> **In words.** PSRL behaves like a scientist who picks one plausible hypothesis about the world and tests it for a whole episode, rather than flipping a coin at every step. If the hypothesis says that a distant state is rewarding, the agent goes all the way there and finds out. Over time the hypotheses that keep being refuted lose their posterior probability.

Bootstrapped DQN (Osband et al., 2016) approximates the posterior with an ensemble of K Q-heads on a shared torso, each trained on its own bootstrap sample of the data (each transition is used by each head with probability 1/2). At each episode one head is drawn at random and followed greedily. NoisyNets (Section 7.4) are a cheaper relative: the noise of the weights is sampled once for many steps, which also gives temporally consistent exploration.

Pacman 13.2. (Exploration in Pacman)

Exploration is not the bottleneck in Pacman: the cookies are a dense reward, and the random ghosts produce a wide variety of situations by themselves. ε-greedy (DQN) and the entropy bonus with $c_e = 0.03$ (PPO) suffice. Where exploration matters is in the rare, dangerous situations (being trapped between two ghosts), which the agent meets too seldom

to learn well: this is why the look-ahead search of Chapter 11 helps most there. A maze with a single, distant reward and no cookies would be a hard exploration problem.

13.5 Summary

- Dithering (ε-greedy, entropy bonuses) explores in time exponential in the depth of the reward, even in a chain.
- Optimism (R-max, bonuses $\beta/\sqrt{N}$) gives polynomial guarantees; in large spaces, counts are replaced by pseudo-counts from density models or by prediction errors (RND, ICM).
- Posterior sampling (PSRL, bootstrapped DQN) explores deeply by committing to a sampled hypothesis for a whole episode.

Chapter 14

Multi-agent reinforcement learning

Pacman shares its maze with ghosts. In this course they follow fixed rules, so from Pacman's point of view they are part of the environment, and the problem is an MDP. If the ghosts learned too, each agent would face an environment that changes as the others learn: the problem would become a *game*. Multi-agent RL covers competitive games (chess, Go, poker, StarCraft), cooperative teams (robots, traffic lights, network routing) and mixed situations (markets, negotiation).

14.1 Markov games

Definition 14.1. (Markov game)

A Markov (or stochastic) game with n agents is a tuple $(\mathcal{S}, \mathcal{A}_1, \dots, \mathcal{A}_n, p, r_1, \dots, r_n, \gamma)$: in state s, every agent i chooses an action a_i, the joint action $\mathbf{a} = (a_1, \dots, a_n)$ determines the transition $p(s' \mid s, \mathbf{a})$, and agent i receives $r_i(s, \mathbf{a})$. Each agent maximises its own expected discounted return.

With one agent it is an MDP; with one state it is a repeated matrix game. The game is *fully cooperative* if all the rewards are equal, *zero-sum* if $n = 2$ and $r_1 = -r_2$. In general there is no single optimal policy: the best policy of an agent depends on those of the others. A joint policy $(\pi_1, \dots, \pi_n)$ is a *Nash equilibrium* if no agent can increase its return by changing its policy alone; every finite discounted Markov game has a Nash equilibrium in stationary stochastic policies (Fink, 1964).

> **In words.** A Nash equilibrium is a situation in which nobody regrets their own choice, given what the others do: it is stable, not necessarily good. In the prisoner's dilemma both players betray each other at the equilibrium, although both would be better off cooperating. In multi-agent RL "optimal" therefore has no single meaning, and the answer depends on what the other agents do.

14.2 Two-player zero-sum games

Zero-sum games are the multi-agent case where the single-agent theory extends most completely. In a matrix game with payoff matrix A (the first player receives $x^\top A y$ for mixed strategies x, y), von Neumann's minimax theorem states that $\max_x \min_y x^\top A y = \min_y \max_x x^\top A y$; this common *value* $\mathrm{val}(A)$ is computed by linear programming. Shapley (1953) extended the Bellman equations: the value of a zero-sum Markov game satisfies

$$v(s) = \mathrm{val}\Big[r(s, a_1, a_2) + \gamma \sum_{s'} p(s' \mid s, a_1, a_2)\, v(s') \Big]_{a_1, a_2}, \tag{14.1}$$

the value of the matrix game indexed by the two actions.

Theorem 14.2. (Shapley)

The operator $\mathcal{T}$ defined by the right-hand side is a γ-contraction in the sup norm. Its unique fixed point is the value of the game, and the optimal mixed strategies of the matrix games at the fixed point form a stationary equilibrium.

Proof. We prove the contraction. First, val is non-expansive: if $|A_{jk} - B_{jk}| \le \delta$ for all j, k, then $x^\top A y \le x^\top B y + \delta$ for all mixed strategies, since $x^\top (A - B) y$ is an average of entries of $A - B$; taking $\min_y$ then $\max_x$ gives $\mathrm{val}(A) \le \mathrm{val}(B) + \delta$, and symmetrically. For two functions v, w, the matrices defining $(\mathcal{T}v)(s)$ and $(\mathcal{T}w)(s)$ differ entrywise by $\gamma \sum_{s'} p(s' \mid s, a_1, a_2)(v(s') - w(s'))$, at most $\gamma \|v - w\|_\infty$ in absolute value. Hence $\|\mathcal{T}v - \mathcal{T}w\|_\infty \le \gamma \|v - w\|_\infty$. The rest follows as for MDPs (Theorem 3.16). □

In words. Shapley's equation is the Bellman equation of a game: the value of a position, if both players play perfectly from then on, is the value of the one-shot game in which the payoffs are the immediate rewards plus the discounted values of the positions reached. Each player picks the (possibly random) strategy whose worst case is best. Randomness matters: in rock–paper–scissors any deterministic strategy is exploited, and only the uniform mixture is safe.

Minimax-Q (Littman, 1994) is the corresponding Q-learning: it learns $Q(s, a_1, a_2)$ with the target $r + \gamma\, \mathrm{val}[Q(s', \cdot, \cdot)]$, solving a small linear program at each update, and converges to the value of the game under the same conditions as Q-learning. Nash-Q extends the idea to general-sum games, but needs restrictive assumptions: equilibria are no longer unique or interchangeable.

Self-play. In symmetric games, an agent can learn by playing against copies of itself. TD-Gammon (Tesauro, 1995), AlphaZero (Chapter 11) and OpenAI Five learnt this way. Pure self-play may cycle (as in rock–paper–scissors, where each best response is beaten by the next) or overfit to its own style. *Fictitious play* best responds to the *average* of the past opponents, and converges to equilibrium in zero-sum games; population methods keep a league of past and exploiting agents (AlphaStar, Vinyals et al., 2019) or compute equilibria of the meta-game between policies (PSRO, Lanctot et al., 2017). Imperfect-information games such as poker require different tools (counterfactual regret minimisation, which beat professional players at heads-up and six-player no-limit hold'em).

14.3 Learning in cooperative teams

Independent learners. The simplest approach lets each agent run its own single-agent algorithm (independent Q-learning, or independent PPO), treating the others as part of the environment. It often works, but the environment of each agent is non-stationary while the others learn, so the convergence guarantees are lost, and the replay memory of DQN holds transitions generated by obsolete policies of the others.

Centralised training, decentralised execution. In simulation, training can use information that will not be available at execution: the global state and the actions of all the agents. MADDPG (Lowe et al., 2017) gives each agent a centralised critic $Q_i(s, a_1, \ldots, a_n)$ and a decentralised actor $\mu_i(o_i)$ of its own observation, trained as in DDPG (Chapter 10); since the critic sees all the actions, its environment is stationary. MAPPO does the same with PPO and a centralised value function, and is a strong baseline for cooperative tasks.

In words. This is a coach who sees the whole field during training, while the players, during the match, act on what each of them sees. The critic, which knows everything, is needed only to learn; the actors, which must act alone, are all that is kept.

Value decomposition. For a team with a common reward, a joint Q-function $Q_{\text{tot}}(s, \mathbf{a})$ cannot be maximised by enumeration (the number of joint actions is exponential in n), and the agents must act on their own. VDN (Sunehag et al., 2018) writes $Q_{\text{tot}} = \sum_i Q_i(o_i, a_i)$; QMIX (Rashid et al., 2018) combines the Q_i with a mixing network whose weights, produced by a hypernetwork from the state, are non-negative.

Proposition 14.3. (Monotonic mixing)

If $Q_{\text{tot}}(s, \mathbf{a}) = f_s(Q_1(o_1, a_1), \dots, Q_n(o_n, a_n))$ with f_s non-decreasing in each argument, then the joint action made of the individual greedy actions $a_i^\star = \arg\max_{a_i} Q_i(o_i, a_i)$ maximises $Q_{\text{tot}}(s, \cdot)$.

Proof. For any joint action $\mathbf{a}$, $Q_i(o_i, a_i) \le Q_i(o_i, a_i^\star)$ for every i, and replacing the arguments of f_s one at a time by larger ones does not decrease it: $Q_{\text{tot}}(s, \mathbf{a}) \le Q_{\text{tot}}(s, \mathbf{a}^\star)$. □

In words. Monotonic mixing means that no agent can hurt the team by improving its own value: if each agent's own estimate goes up, the team's goes up too. Each agent can then simply pick its own best action, and the team's best joint action follows, without any agent knowing what the others chose.

The joint Q-function is trained by the DQN loss on the common reward, its maximum in the target is computed agent by agent, and each agent acts greedily on its own Q_i. The price is representational: monotonic mixtures cannot express every joint Q-function (for instance, when the best action of an agent depends on what another does).

Pacman 14.4. (Learning ghosts)

The ghosts of `ghost_agents.py` follow fixed rules. Making them learn turns Pacman into a two-team zero-sum Markov game: Pacman against a cooperative team of ghosts with the opposite reward. The obvious scheme alternates: train the ghosts (with QMIX or MAPPO) against a frozen PPO Pacman, then retrain Pacman against the new ghosts, and so on. Each side then learns to beat only the opponent it faces now, and the training can cycle. For example, the ghosts learn to guard the power cookies, so Pacman learns to clear the far corners first; the ghosts then spread out to cover the corners, so Pacman learns to rush the power cookies, which the ghosts learn to guard again… Each version beats the previous one and forgets how to beat the older ones, as in rock–paper–scissors.

The standard remedy is a *league*: a copy of each side is saved regularly, and each side trains against opponents drawn from all the saved copies, not only the latest:

1. train Pacman against the rule-based ghosts; save it as P_0;
2. train ghosts against P_0; save them as G_0;
3. train Pacman against opponents drawn from {rule-based ghosts, G_0}; save it as P_1;
4. train ghosts against opponents drawn from $\{P_0, P_1\}$; save them as G_1; and so on, the pools growing at each round.

A Pacman that forgets how to beat the ghosts of an earlier generation loses when they are drawn, and relearns it. The opponents can be drawn uniformly, or more often among those the current agent loses against (as in AlphaStar's league); the rule-based ghosts stay in the pool, so that Pacman keeps mastering the original game.

In words. Training against the latest opponent only is chasing a moving target: the agent adapts to the opponent's current habits, not to the game. Training against a growing collection of past opponents makes forgetting costly, and pushes each side towards strategies that are good against everything seen so far. In the limit this is fictitious play, which converges to an equilibrium in zero-sum games where plain alternation can cycle forever.

14.4 Summary

- Markov games generalise MDPs to several agents; the solution concept is an equilibrium, not an optimal policy.
- In two-player zero-sum games, Shapley's operator is a contraction and minimax-Q learns the value; self-play, fictitious play and leagues train agents against themselves.
- For cooperative teams, centralised training with decentralised execution (MADDPG, MAPPO) restores stationarity, and monotonic value decomposition (VDN, QMIX) makes the joint maximisation decentralised.

Chapter 15

Reinforcement learning from human feedback

Large language models are trained first to predict text. To turn them into helpful assistants, they are then fine-tuned with reinforcement learning, with a reward that comes from human (or AI) judgements of their answers. This last chapter shows that the tools of the course, PPO above all, apply directly; and it derives direct preference optimisation, which removes the RL loop altogether.

15.1 A language model is a policy

A language model $\pi_\theta(y \mid x)$ generates an answer $y = (y_1, \dots, y_T)$ to a prompt x token by token, with $\pi_\theta(y \mid x) = \prod_t \pi_\theta(y_t \mid x, y_{<t})$. This is a policy in an MDP whose state is the text so far, whose actions are the tokens of the vocabulary, and whose transitions are deterministic (appending the token). The reward is usually given only at the end, for the whole answer, so the problem can also be seen as a contextual bandit (Chapter 2) whose context is x and whose "arm" is the whole answer y.

The standard pipeline (Ouyang et al., 2022, building on Christiano et al., 2017) has three stages:

1. **Supervised fine-tuning:** behaviour cloning (Chapter 12) on demonstrations of good answers, giving the reference policy π_{ref};
2. **Reward modelling:** learning a reward $r_\phi(x, y)$ from comparisons between answers;
3. **RL fine-tuning:** maximising r_ϕ with PPO, while staying close to π_{ref}.

15.2 Reward models from preferences

Humans compare answers more reliably than they score them. For a prompt x, two answers are sampled and a labeller says which is better, y_w (winner) or y_l (loser). The Bradley–Terry model (1952) assumes that

$$\mathbb{P}(y_w \succ y_l \mid x) = \sigma\left(r(x, y_w) - r(x, y_l)\right), \qquad \sigma(u) = \frac{1}{1 + e^{-u}}, \tag{15.1}$$

for a latent reward r. The reward model, usually the language model with a scalar head, is fitted by maximum likelihood:

$$\mathcal{L}(\phi) = -\mathbb{E}_{(x, y_w, y_l)}\left[\log \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)\right]. \tag{15.2}$$

The reward is only defined up to an additive function of x, which cancels in (15.1); this is harmless, since the policy is compared on the same prompt.

In words. This is the logic of Elo ratings in chess: each answer has a hidden score, and the probability that one is preferred to another depends only on the difference of their scores. Equal scores give 50%, a difference of 2 gives $\sigma(2) \approx 88\%$. Fitting the model finds the scores that best explain which answers won their comparisons.

15.3 RL fine-tuning with a KL penalty

The policy is trained to maximise the learnt reward, regularised by a Kullback–Leibler penalty towards the reference policy:

$$\max_{\pi} \; \mathbb{E}_x \left[\mathbb{E}_{y \sim \pi(\cdot|x)}[r(x,y)] - \beta \, \mathrm{KL}\left(\pi(\cdot \mid x) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid x)\right) \right]. \tag{15.3}$$

The penalty matters: the reward model is accurate only near the answers it was trained on, and an unconstrained policy finds answers where it is wrong (*reward hacking*, or overoptimisation: the true quality rises, then falls as the learnt reward keeps rising). It also preserves the fluency and the diversity of the reference model.

Proposition 15.1. (Optimal policy of the KL-regularised objective)

For each prompt x, the objective (15.3) is maximised by

$$\pi^\star(y \mid x) = \frac{1}{Z(x)} \, \pi_{\mathrm{ref}}(y \mid x) \exp\left(\frac{r(x,y)}{\beta}\right), \qquad Z(x) = \sum_y \pi_{\mathrm{ref}}(y \mid x) \, e^{r(x,y)/\beta}. \tag{15.4}$$

Proof. For a fixed x (omitted from the notation),

$$\begin{aligned} \mathbb{E}_\pi[r(y)] - \beta \, \mathrm{KL}(\pi \,\|\, \pi_{\mathrm{ref}}) &= -\beta \, \mathbb{E}_{y \sim \pi}\left[\log \frac{\pi(y)}{\pi_{\mathrm{ref}}(y) e^{r(y)/\beta}}\right] \\ &= -\beta \, \mathrm{KL}(\pi \,\|\, \pi^\star) + \beta \log Z, \end{aligned}$$

maximal, since a KL divergence is non-negative and zero only between equal distributions, at $\pi = \pi^\star$. □

In words. The optimal policy keeps the reference model's judgement of what is a plausible answer and tilts it towards rewarded answers: an answer whose reward is higher by β becomes $e \approx 2.7$ times more likely. It cannot invent answers that the reference considers impossible ($\pi_{\mathrm{ref}} = 0$ stays 0). The parameter β sets how far the tilt may go, and so how much the learnt reward is trusted.

This is the same computation as soft policy improvement (Lemma 10.5), with the reference policy as prior. The normalising constant $Z(x)$ is a sum over all possible answers, so (15.4) cannot be used directly, and the objective is optimised with PPO (Chapter 9). In practice the KL penalty is spread over the tokens: the reward of token t is $-\beta \log(\pi_\theta(y_t \mid \cdot)/\pi_{\mathrm{ref}}(y_t \mid \cdot))$, plus $r_\phi(x,y)$ at the last token. A value head estimates the values, and GAE (Definition 8.6) computes the advantages; apart from the size of the networks, this is the algorithm that trained the Pacman agent.

Without a critic. The value network is as large as the policy, which doubles the memory. Since the reward is given per answer, a baseline can instead be computed by sampling several answers per prompt: GRPO (Shao et al., 2024) samples G answers, and uses as advantage of each answer its reward standardised within the group, $A_i = (r_i - \mathrm{mean}(r))/\mathrm{std}(r)$, in the clipped PPO objective. It is the baseline of Chapter 8 estimated by Monte Carlo instead of a

network. With rewards that can be checked automatically (a mathematical answer, a program that passes its tests), this RL with verifiable rewards trains models to reason at length before answering.

> **In words.** GRPO grades each answer against the other attempts at the same question: an answer is reinforced if it did better than its siblings, discouraged if it did worse. The group plays the role of the critic, without a network: "was this answer better than what I usually produce for this prompt?"

15.4 Direct preference optimisation

Rafailov et al. (2023) observed that (15.4) can be inverted: it expresses the reward in terms of the optimal policy,

$$r(x, y) = \beta \log \frac{\pi^\star(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x). \tag{15.5}$$

Theorem 15.2. (Direct preference optimisation)

Under the Bradley–Terry model (15.1), with the reward parametrised by a policy as $r_\theta(x, y) = \beta \log(\pi_\theta(y \mid x)/\pi_{\text{ref}}(y \mid x))$, the reward-model loss (15.2) becomes

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right], \tag{15.6}$$

and π_θ is the optimal policy (15.4) for the reward r_θ. Minimising (15.6) therefore fits a reward model and solves the KL-regularised RL problem for it at the same time.

Proof. Adding $\beta \log Z(x)$ to r_θ does not change the Bradley–Terry probabilities, since it is the same for y_w and y_l; hence the loss (15.2) for r_θ is (15.6). For the reward r_θ, the normalising constant of (15.4) is $Z(x) = \sum_y \pi_{\text{ref}}(y \mid x)\pi_\theta(y \mid x)/\pi_{\text{ref}}(y \mid x) = 1$, and the optimal policy is $\pi_{\text{ref}} \cdot \pi_\theta/\pi_{\text{ref}} = \pi_\theta$. Conversely, any reward is equivalent, up to a function of x, to one of this form, namely the one of its optimal policy, so no generality is lost. □

> **In words.** In DPO the policy *is* the reward model: the reward it implicitly assigns to an answer is how much more likely it makes that answer than the reference model does. Training raises this likelihood ratio for the preferred answer and lowers it for the rejected one. The gradient of (15.6) is weighted by $\sigma(-\text{margin})$, so the pairs that the policy currently ranks wrongly count most, and those already ranked correctly with a large margin hardly count.

Listing 15.1: DPO loss (PyTorch); `logp` are summed token log-probabilities of the answers

```
def dpo_loss(logp_w, logp_l, ref_logp_w, ref_logp_l, beta=0.1):
    margin = beta * ((logp_w - ref_logp_w) - (logp_l - ref_logp_l))
    return -F.logsigmoid(margin).mean()
```

DPO is supervised learning on a fixed dataset of comparisons: no sampling during training, no reward model, no value network. It is an *offline* method (Chapter 12), and shares their limit: the data are answers of another policy, whereas PPO with a reward model can evaluate the new answers of the current policy. Online and iterative variants sample fresh pairs from the current policy and label them.

15.5 Feedback from AI

Human labels are expensive and inconsistent. In RL from AI feedback (RLAIF) the comparisons are made by a language model, guided by a written set of principles (constitutional AI, Bai et al., 2022). Reward models can also be trained to judge each step of a reasoning rather than the final answer (process rewards). The difficulties are those this course met throughout: the reward is a proxy, and an optimiser exploits its flaws; the policy must explore to improve, but stay close to what is known to be good; and the data distribution moves with the policy.

15.6 Summary

- A language model is a policy over tokens; RLHF fine-tunes it with a reward model learnt from pairwise comparisons (Bradley–Terry).
- The objective is the reward minus a KL penalty to the reference model; its optimum is the reference reweighted by $e^{r/\beta}$, and it is optimised with PPO and GAE, or with group baselines (GRPO).
- DPO inverts the optimal-policy formula and turns the problem into a classification loss on comparisons, with no RL loop.

Conclusion

The course followed one thread: the Bellman equations (Chapter 3), solved exactly when the model is known (Chapter 4), from samples when it is not (Chapter 5), with function approximation when the states are too many (Chapters 6 and 7); then the direct optimisation of the policy (Chapters 8 and 9), extended to continuous actions (Chapter 10); and planning with a model (Chapter 11). The last chapters removed, one at a time, the assumptions of the standard setting: rewards replaced by demonstrations, interaction replaced by a fixed dataset (Chapter 12), exploration made deliberate (Chapter 13), several learning agents (Chapter 14), and rewards learnt from human preferences (Chapter 15).

A few ideas came back in almost every chapter: bootstrapping, and the contraction that makes it converge; the bias–variance trade-off between Monte Carlo returns and bootstrapped targets, controlled by n-step returns and λ; the danger of maximising over estimates (the maximisation bias of DQN, the actor exploiting the critic in DDPG, extrapolation in offline RL, reward hacking in RLHF), countered by pessimism or by keeping close to a reference; and the value of keeping the policy close to the one that produced the data (trust regions, KL penalties, behaviour constraints).

On Pacman, these ideas took the agent from 79% to about 98% of games won: PPO with feature planes, a mirror-symmetric network and GAE, then a look-ahead search that knows nothing of the ghosts' code, and expert iteration to teach the search's decisions to the network. Game `v2` (a bigger maze, three ghosts that come back after being eaten, a shorter power) opens the next round, with a new difficulty: as discussed in Remark 3.3, it is only partially observable from one image. The directions left open in Chapter 11 (MCTS with a value network, an AlphaZero-style loop) and the learning ghosts of Chapter 14 are natural next projects.

Further reading. Sutton and Barto, *Reinforcement Learning: An Introduction* (2nd edition, 2018) for the foundations; Szepesvári, *Algorithms for Reinforcement Learning* (2010) and Bertsekas, *Reinforcement Learning and Optimal Control* (2019) for the theory; Lattimore and Szepesvári, *Bandit Algorithms* (2020) for bandits; Agarwal, Jiang, Kakade and Sun, *Reinforcement Learning: Theory and Algorithms* (online notes) for sample complexity; and the original papers cited in each chapter. For Monte Carlo tree search in two-player games, see the Hex implementation and the interactive visualiser cited in Remark 11.5.